

What is tesseract planner

If you mean **Tesseract Motion Planning**, it is an **open-source, ROS-friendly motion-planning and trajectory-optimization framework for industrial robots**, originally developed around ROS-Industrial.

It is especially useful for tasks where the robot must follow a **process path**, rather than simply move from point A to point B. Examples include welding, painting, grinding, inspection, additive manufacturing, and dispensing.

For example, if you provide a sequence of Cartesian TCP targets

$T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow \dots \rightarrow T_n$

Tesseract can solve the robot configurations for those targets while considering joint limits, collisions, kinematics, and trajectory quality. It supports planning approaches such as **TrajOpt trajectory optimization**, sampling-based planning through OMPL, Cartesian/process planning, and time parameterization.

For your type of industrial robotics work, a useful distinction is:

Framework	Main strength
Tesseract	Industrial process/tool-path planning and trajectory optimization
Movelt 2	General-purpose ROS 2 manipulation and motion planning
HPP	Constraint-heavy planning, complex kinematics and manipulation
NVIDIA cuMotion	GPU-accelerated motion generation/planning

So conceptually, for something like sealant dispensing:

CAD/process path → TCP waypoints → Tesseract planner → collision-free joint trajectory → robot controller

Tesseract is particularly interesting when you already know **where the TCP should travel along a surface** and need to convert that process path into a feasible, optimized robot trajectory.

If you want, I can also explain **Tesseract vs HPP vs Movelt for your NDT/dispensing applications**, including which one is better for coverage/tool-path planning.

What is the method or approach in finding inverse kinematics solutions for a redundant robotic arm

For a **redundant robotic arm**, inverse kinematics (IK) is different from a standard 6-DOF robot because there can be **infinitely many joint configurations** that achieve the same TCP pose.

For example, with a 7-DOF robot:

$$q=[q_1,\dots,q_7]$$

but the Cartesian TCP pose has only 6 DOF:

$$x=[x,y,z,R_x,R_y,R_z]$$

so there is one extra degree of freedom that can be exploited.

Main approaches

Approach	Basic idea	Typical methods
Analytical IK	Derive closed-form joint solutions	Geometry, polynomial methods
Numerical IK	Iteratively minimize TCP pose error	Newton-Raphson, Jacobian pseudoinverse
Damped Least Squares	Numerical IK with better singularity behavior	DLS / Levenberg-Marquardt
Null-space optimization	Use redundancy to satisfy secondary objectives	Pseudoinverse + null-space projection
Optimization-based IK	Formulate IK as constrained optimization	SQP, nonlinear programming
Sampling/search-based IK	Sample redundant DOFs and solve remaining joints	Discretization, random sampling
Differential IK	Compute joint velocities from desired TCP velocity	Jacobian-based velocity IK

For redundant robots, **null-space optimization** is one of the most important concepts.

Suppose the desired TCP velocity is:

$$\dot{x} = J(q)\dot{q}$$

$$\dot{x} = J(q)\dot{q}$$

Because J is not square for a redundant robot, we can use the pseudoinverse:

$$\dot{q} = J^+ \dot{x}$$

$$\dot{q} = J^+ \dot{x}$$

But this only gives one particular solution. The general solution is:

$$\dot{q} = J^+ \dot{x} + (I - J^+ J) \dot{q}_0$$

$$\dot{q} = J^+ \dot{x} + (I - J^+ J) \dot{q}_0$$

The first term,

$$J^+ \dot{x}$$

$$J^+ \dot{x}$$

makes the TCP perform the required motion.

The second term,

$$(I - J^+ J) \dot{q}_0$$

$$(I - J^+ J) \dot{q}_0$$

moves the joints **without affecting the TCP task**. This is the **null-space motion**.

You can therefore use the redundant DOF to optimize something useful, such as:

- avoiding joint limits,
- avoiding singularities,
- avoiding obstacles/self-collision,
- keeping the elbow away from the workpiece,
- minimizing joint motion or energy,
- maximizing manipulability,
- maintaining a preferred posture.

For example, if $H(q)$ is a cost representing distance from joint limits, you can choose:

$$\dot{q}^0 = -k \nabla H(q)$$

$$\dot{q}_0 = -k \nabla H(q)$$

and obtain:

$$\dot{q} = J^+ \dot{x} - k(I - J^+ J) \nabla H(q)$$

$$\dot{q} = J^+ \dot{x} - k(I - J^+ J) \nabla H(q)$$

So the robot **maintains the required TCP trajectory while simultaneously moving toward a better joint configuration.**

For path planning, there is another important approach

If you have an entire Cartesian tool path

$$T_1, T_2, T_3, \dots, T_N$$

$$T_1, T_2, T_3, \dots, T_N$$

you generally should **not independently choose an arbitrary IK solution at each waypoint.**

You might get:

$$q_1^{(3)} \rightarrow q_2^{(8)} \rightarrow q_3^{(1)}$$

$$q_1^{(3)} \rightarrow q_2^{(8)} \rightarrow q_3^{(1)}$$

which can cause large joint jumps even though the TCP path is smooth.

Instead, generate multiple IK candidates:

$$T_i \rightarrow \{q_i^1, q_i^2, \dots, q_i^m\}$$

$$T_i \rightarrow \{q_i^1, q_i^2, \dots, q_i^m\}$$

and then select a **continuous sequence**:

$$q_1^a \rightarrow q_2^b \rightarrow q_3^c \rightarrow \dots$$

$$q_1^a \rightarrow q_2^b \rightarrow q_3^c \rightarrow \dots$$

while minimizing something like

$C = w_1 C_{\text{joint motion}} + w_2 C_{\text{joint limits}} + w_3 C_{\text{collision}} + w_4 C_{\text{singularity}} + w_5 C_{\text{posture}}$

$$C = w_1 C_{\text{joint motion}} + w_2 C_{\text{joint limits}} + w_3 C_{\text{collision}} + w_4 C_{\text{singularity}} + w_5 C_{\text{posture}}$$

This is particularly relevant for **industrial process paths such as NDT inspection, dispensing, welding, AFP, painting, etc.**

So there are really **two related problems**:

IK:

"What joint configurations can achieve this TCP pose?"

Redundancy resolution / path IK:

"Among all those configurations, which sequence should I use along the complete TCP path?"

For a redundant industrial robot following a long surface path, the **second problem is often more important than simply finding an IK solution.**

What is the method or approach in finding inverse kinematics solutions for a redundant robotic arm they use in tesseract planning framework

In the **Tesseract Planning framework**, there isn't one special "Tesseract IK algorithm" for redundant manipulators. Tesseract separates **kinematics** from **motion planning** and can use different IK solver plugins.

For redundant robots, the important approach is typically **redundant-solution search/sampling combined with numerical or analytical IK**, followed by planning/optimization to choose useful solutions.

A simplified flow is:

TCP pose → IK solver → $\{q_1, q_2, \dots, q_n\}$ → collision/constraint checks → trajectory planner/optimizer

TCP pose → IK solver → $\{q_1, q_2, \dots, q_n\}$ → collision/constraint checks → trajectory planner/optimizer

For example, if the robot has **7 DOF** and the TCP constraint is 6 DOF, Tesseract can exploit the remaining redundancy by searching different values of the redundant joint(s). For each sampled redundant configuration, the IK solver finds configurations satisfying the TCP pose. The resulting candidates can then be filtered for joint limits and collisions and evaluated by the planner.

The actual mechanism depends strongly on the **kinematics plugin** being used. Tesseract commonly supports solvers such as **KDL**, **OPW**, and **IKFast**, among others. A numerical solver such as KDL generally starts from a seed configuration and iteratively solves

$$q_{k+1} = q_k + J^\dagger e$$

$$q_{k+1} = q_k + J^\dagger e$$

where e is the Cartesian pose error and $J^\dagger$ is a pseudoinverse or related numerical solution. Different seeds can converge to different configurations, which is useful for redundant robots.

For an industrial Cartesian process path, however, Tesseract's planning architecture becomes more interesting. Suppose you have:

$P_1, P_2, \dots, P_N$

$$P_1, P_2, \dots, P_N$$

At each Cartesian waypoint there may be multiple valid configurations:

$P_1 \rightarrow \{q_{11}, q_{12}, q_{13}, \dots\}$ $P_2 \rightarrow \{q_{21}, q_{22}, q_{23}, \dots\} :$

$$\begin{aligned} P_1 &\rightarrow \{q_{11}, q_{12}, q_{13}, \dots\} \\ P_2 &\rightarrow \{q_{21}, q_{22}, q_{23}, \dots\} \\ &\vdots \end{aligned}$$

The planning problem then becomes selecting a good **continuous sequence through these IK candidates**, rather than simply solving each waypoint independently. This is important for redundant arms because otherwise the solver can jump between very different elbow/posture configurations.

Tesseract's planners can then consider objectives and constraints such as **joint motion, collision avoidance, joint limits, smoothness, and Cartesian path requirements**. Trajectory-optimization planners such as **TrajOpt/TrajOpt-IFOPT** can optimize the trajectory globally rather than treating every waypoint as an isolated IK problem.

So I would describe the Tesseract approach as:

Cartesian/process waypoints $\rightarrow$ generate feasible IK configurations using a kinematics plugin $\rightarrow$ exploit redundancy through seeds/search $\rightarrow$ filter/constraint solutions $\rightarrow$ select or optimize a continuous joint-space trajectory.

One distinction is important relative to what we discussed about null-space IK: **Tesseract does not inherently resolve every redundant IK problem by using the classic**

$$\dot{q} = J^\dagger \dot{x} + (I - J^\dagger J) \dot{q}_0$$

$$\dot{q} = J^\dagger \dot{x} + (I - J^\dagger J) \dot{q}_0$$

null-space controller formulation. That is one method for redundancy resolution, but Tesseract's planning framework is broader: redundancy can be handled through the selected IK plugin, sampling/search, and trajectory optimization.

For your **coverage/process-path use cases**, the most relevant Tesseract component is actually its approach to **Cartesian process planning with many possible IK solutions along a tool path**. I can explain how Tesseract's **Descartes/Ladder-Graph style planning** handles this, because that is particularly relevant to redundant industrial robots.

What is the descartes in tesseract planning framework

Yes. **Descartes** in the Tesseract ecosystem is particularly relevant to the question you asked about **IK for redundant robots**.

In simple terms, Descartes is a **Cartesian/process-path planner**. Instead of asking only:

“Can the robot reach this TCP pose?”

it asks:

“Among all valid IK configurations at every TCP waypoint, what is a good continuous joint-space path through the complete Cartesian tool path?”

How Descartes works

Suppose your process path contains Cartesian TCP waypoints:

$P_1, P_2, P_3, \dots, P_N$

$P_1, P_2, P_3, \dots, P_N$

For each waypoint, the IK solver can produce multiple solutions:

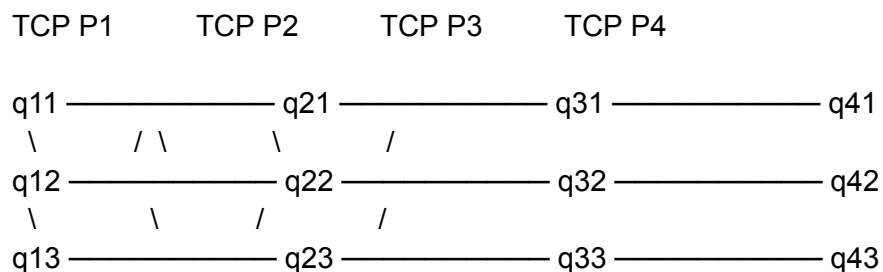
$P_1 \rightarrow \{q_{11}, q_{12}, q_{13}, \dots\}$ $P_2 \rightarrow \{q_{21}, q_{22}, q_{23}, \dots\}$ $P_3 \rightarrow \{q_{31}, q_{32}, q_{33}, \dots\}$

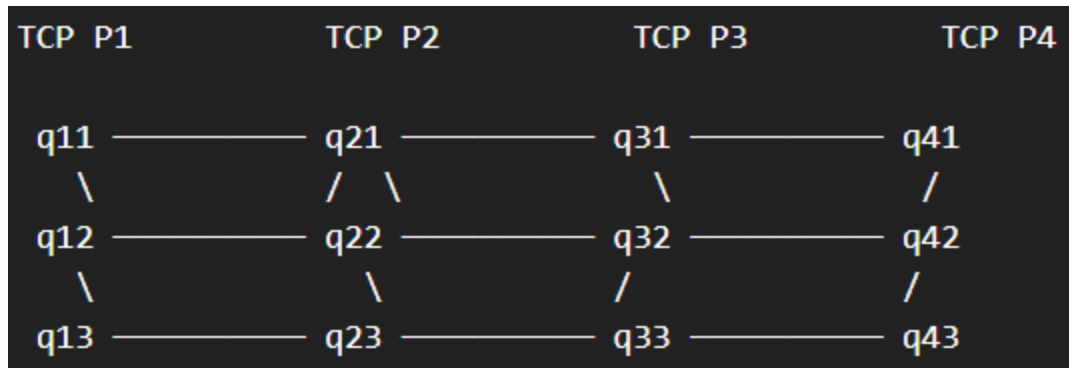
$P_1 \rightarrow \{q_{11}, q_{12}, q_{13}, \dots\}$

$P_2 \rightarrow \{q_{21}, q_{22}, q_{23}, \dots\}$

$P_3 \rightarrow \{q_{31}, q_{32}, q_{33}, \dots\}$

Descartes organizes these candidates conceptually as a **layered graph**:





Each node represents a valid robot configuration for a Cartesian waypoint.

Edges connect configurations that the robot can reasonably transition between.

An edge can have a cost such as:

$$C_{ij} = \|q_j - q_i\|$$

$$C_{ij} = \|q_j - q_i\|$$

or a weighted joint-space cost:

$$C_{ij} = \sum_k w_k |q_{j,k} - q_{i,k}|$$

$$C_{ij} = \sum_k w_k |q_{j,k} - q_{i,k}|$$

Then graph search finds a low-cost sequence:

$$q_{12} \rightarrow q_{22} \rightarrow q_{32} \rightarrow q_{42}$$

$$q_{12} \rightarrow q_{22} \rightarrow q_{32} \rightarrow q_{42}$$

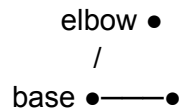
rather than simply taking the first IK solution at every waypoint.

Why this matters for redundant robots

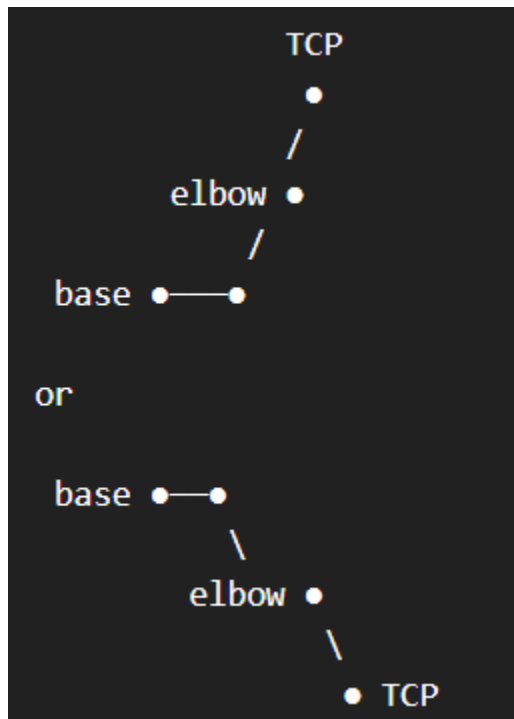
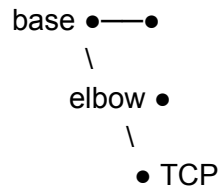
Imagine a 7-DOF robot following a surface.

At one TCP pose there might be many configurations with different elbow positions:





or



Both can give essentially the **same TCP pose**.

If you independently call IK for every waypoint, you could get:

$q_1^A \rightarrow q_2^B \rightarrow q_3^A$

$$q_1^A \rightarrow q_2^B \rightarrow q_3^A$$

and the elbow could make large unnecessary movements.

Descartes instead considers the **whole sequence of candidate solutions** and searches for continuity.

Another important feature: Cartesian tolerances

For many manufacturing processes, the TCP pose isn't necessarily one exact pose.

For example, during inspection you might require the TCP position and surface normal to remain fixed, but allow rotation around the tool axis:

$$\Theta_{\text{tool}} \in [-180^\circ, 180^\circ]$$

$$\theta_{\text{tool}} \in [-180^\circ, 180^\circ]$$

Descartes can discretize that allowable freedom:

$$\theta = -180^\circ, -175^\circ, \dots, 175^\circ, 180^\circ$$

$$\theta = -180^\circ, -175^\circ, \dots, 175^\circ, 180^\circ$$

and calculate IK solutions for those sampled poses.

This dramatically increases the number of possible configurations available to the planner.

So conceptually:

Cartesian tolerances → pose samples → IK solutions → graph → best joint path

Cartesian tolerances → pose samples → IK solutions → graph → best joint path

This is why Descartes-style planning is particularly attractive for **welding, NDT inspection, painting, dispensing, grinding and other surface-following processes**.

One important terminology point: in modern Tesseract you'll encounter the **Descartes planner and ladder-graph concepts** rather than thinking of Descartes as an IK solver itself. **Descartes does not replace IK**. It uses IK solutions to solve the larger problem of finding a continuous joint trajectory through a Cartesian process path.

For a coverage path such as your zig-zag inspection trajectory, this distinction is especially useful: **coverage planning generates the TCP path; IK generates candidate robot configurations; Descartes chooses a good continuous sequence through those candidates; trajectory optimization/time parameterization can then refine it further.**

We are working with IRT Joule verne and LAAS

IRT uses moveit2 and tesseract planner

since moveit2 is not industrial, to make it industrialize they started using Tesseract

LAAS uses HPP (humanoid path planner)

no i need to do a comparison between IRT JV approach and LAAS approach

i would like to do by component wise

like Tool path generation for use case like inspection (swathe pattern)

then robot path and trajectory planner with constraints imposed

trajectory optimization

route optimization techniques like visiting 100+ defects in a sequence (HPP uses TSP and GTSP and IRT doesn't have anything place)

first probably focus of robot path and trajectory planner

Yes. I would compare **IRT Jules Verne's Moveit 2 + Tesseract approach** against **LAAS's HPP approach component-by-component**, rather than comparing "Tesseract vs HPP" as two monolithic frameworks.

And I agree that the best place to start is **Robot Path & Trajectory Planning**, because that is where the architectural difference becomes clear.

One small correction to the framing: I would avoid saying simply "**Moveit 2 is not industrial.**" Moveit 2 is used in industrial applications, but Tesseract explicitly positions itself as an optimization-centric framework engineered for industrial/process-driven robotics.

1. First comparison: Robot Path & Trajectory Planning

At a high level, I would represent the two approaches like this:

IRT Jules Verne

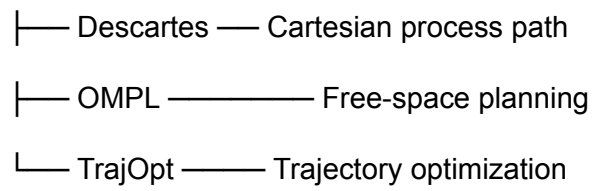
Process / Tool Path



Cartesian TCP Waypoints



Tesseract



Time Parameterization



Executable Robot Trajectory

LAAS

Process / Tool Path



Cartesian TCP Waypoints / Constraints



HPP



Configuration-space planning

+ Explicit / Implicit constraints

+ Constraint projection



RRT / Bi-RRT / PRM / etc.



Path Optimization



Time Parameterization



Executable Robot Trajectory

IRT Jules Verne

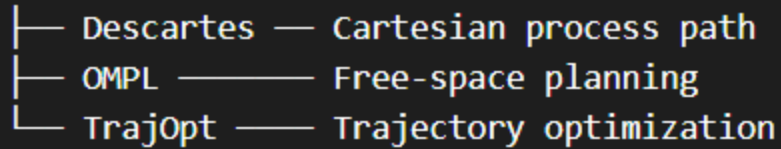
Process / Tool Path



Cartesian TCP Waypoints



Tesseract



Time Parameterization



Executable Robot Trajectory

LAAS

Process / Tool Path



Cartesian TCP Waypoints / Constraints



HPP



Configuration-space planning

- + Explicit / Implicit constraints
- + Constraint projection



RRT / Bi-RRT / PRM / etc.

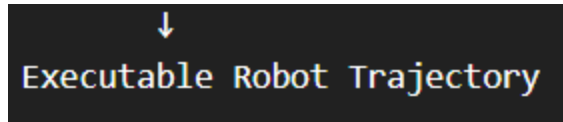


Path Optimization



Time Parameterization





Tesseract's current planning documentation explicitly exposes OMPL, TrajOpt/TrajOpt-IFOPT and Descartes, with Descartes targeted at dense Cartesian toolpaths and TrajOpt at optimization/smoothing. HPP, in contrast, structures the planning problem around configuration-space planning, steering methods, roadmaps and constraints.

Component-level comparison

Aspect	IRT — MoveIt 2 + Tesseract	LAAS — HPP
Primary philosophy	Industrial/process-oriented motion planning	General constrained configuration-space motion planning
Cartesian process path	Descartes is particularly suited	Represent TCP/path requirements as constraints / EE trajectory
Free-space planner	OMPL — RRTConnect, RRT*, PRM, etc.	HPP planners — RRT/roadmap-based approaches etc.
IK handling	Kinematics plugins + Descartes candidate solutions	Numerical constraint solving/projection
Redundant robot	Generate/evaluate multiple IK possibilities along Cartesian path	Redundancy naturally exists in constraint manifold; numerical constraint solver projects configurations
Path constraints	Planner-specific constraints + TrajOpt costs/constraints	Core architectural concept in HPP

Collision avoidance	Tesseract environment/contact managers + planner	HPP-FCL + path validation
Trajectory optimization	TrajOpt / TrajOpt-IFOPT	HPP PathOptimizer family
Pipeline composition	Task Composer	ProblemSolver + planner/steering method/constraints/optimizers
Industrial process focus	Very strong	More general/research-oriented, but applicable to industrial constrained motion
ROS 2 integration	Strong	Requires more application-specific integration

The distinction in the **constraint architecture** is particularly important. HPP explicitly represents nonlinear constraints through [ConstraintSet/ConfigProjector](#); its constraint solver solves nonlinear equations on the robot configuration using a Newton-Raphson-like iterative method with Jacobian pseudoinverses.

2. Most important difference: Cartesian trajectory handling

For your inspection example, suppose the tool-path generator gives:

$P_1, P_2, \dots, P_{500}$

$P_1, P_2, \dots, P_{500}$

where every P_i contains the required inspection TCP pose.

IRT / Tesseract

The natural Tesseract approach is approximately:

500 Cartesian TCP poses



Descartes



Generate IK possibilities

for Cartesian waypoints



Graph search for a good

continuous joint path



TrajOpt

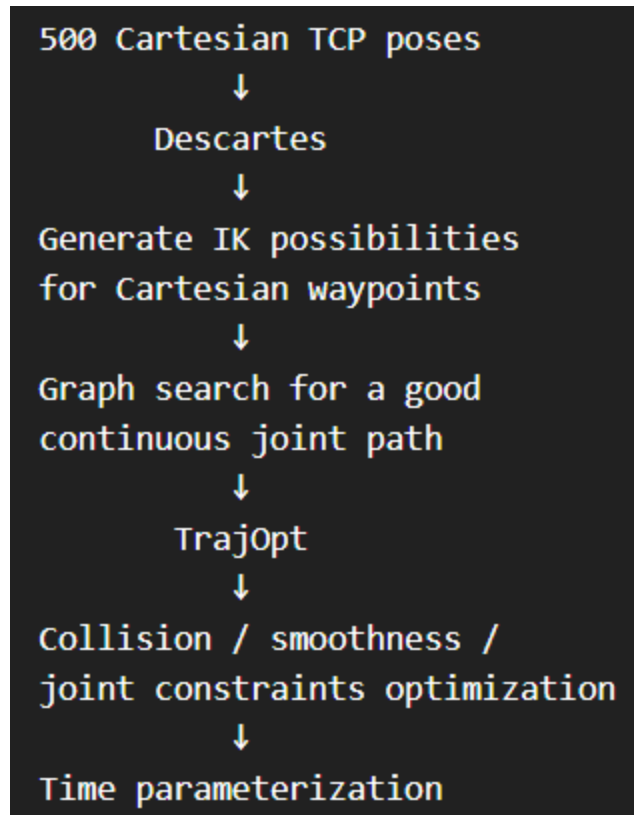


Collision / smoothness /

joint constraints optimization



Time parameterization



The current Tesseract Cartesian pipeline uses **Descartes graph search followed by TrajOpt smoothing**, specifically targeting precise/dense Cartesian paths.

This is very natural for:

“Here is the manufacturing TCP path. Find a good robot trajectory that follows it.”

3. LAAS / HPP approaches the problem differently

HPP is fundamentally very strong at:

“Find a path through configuration space while satisfying these constraints.”

You can formulate your inspection condition as something such as:

$$f(q) = T_{TCP}(q) - T_{desired} = 0$$

$$f(q) = T_{TCP}(q) - T_{desired} = 0$$

or only constrain selected dimensions.

For example:

$x=x_d$ $y=y_d$ $z=z_d$

and

$Z_{tool} \parallel N_{surface}$

$$\begin{aligned} x &= x_d \\ y &= y_d \\ z &= z_d \\ Z_{tool} &\parallel N_{surface} \end{aligned}$$

while perhaps allowing rotation around the tool normal.

HPP's constraint machinery supports nonlinear constraints, equality/inequality formulations and locked joints, with numerical projection used to obtain configurations satisfying the constraints.

Conceptually:

TCP requirement

↓

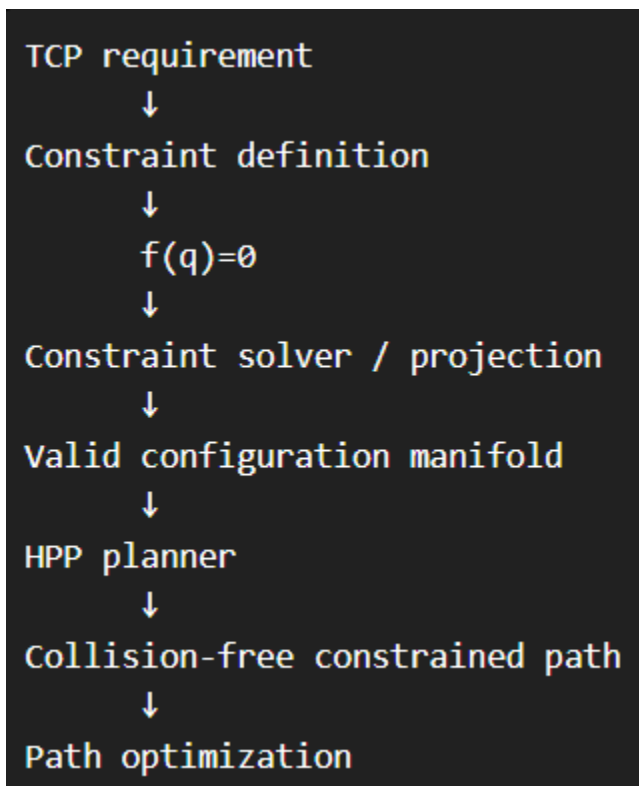
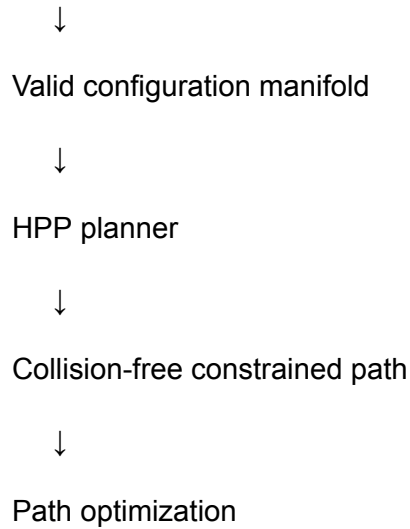
Constraint definition

↓

$f(q)=0$

↓

Constraint solver / projection



This is a **fundamental architectural difference** from the Descartes-style candidate-IK graph.

4. Redundant robot is where the comparison gets interesting

Suppose you have a 7-DOF robot.

For a particular inspection TCP pose:

$$T_{TCP} = T_d$$

$$T_{TCP} = T_d$$

there are many configurations:

$$q_1, q_2, q_3, \dots$$

$$q_1, q_2, q_3, \dots$$

Tesseract / Descartes

Think approximately:

$$P_i \rightarrow \{q_{i1}, q_{i2}, \dots, q_{iN}\}$$

$$P_i \rightarrow \{q_i^1, q_i^2, \dots, q_i^N\}$$

and then:

$$P_1 \quad P_2 \quad P_3$$

$$q_{11} \text{ ————— } q_{21} \text{ ————— } q_{31}$$

$$\backslash \quad / \quad \backslash \quad /$$

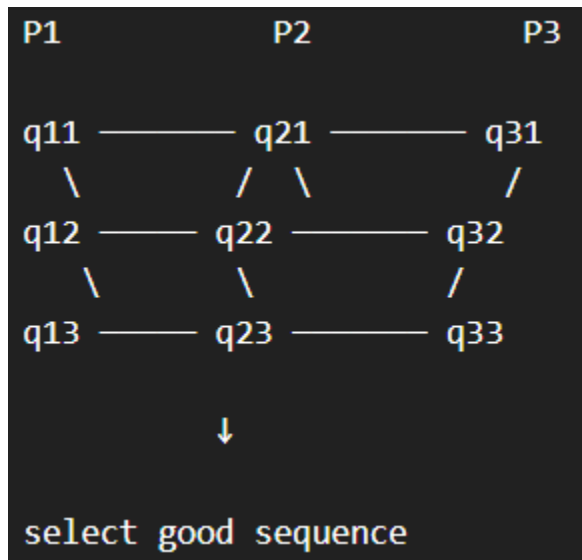
$$q_{12} \text{ ————— } q_{22} \text{ ————— } q_{32}$$

$$\backslash \quad \backslash \quad /$$

$$q_{13} \text{ ————— } q_{23} \text{ ————— } q_{33}$$

↓

select good sequence



So the problem is heavily about **finding a good sequence among Cartesian waypoint solutions**.

HPP

HPP can instead view the valid configurations as a **constraint manifold**:

$$M = \{q \mid f(q) = 0\}$$

$$\mathcal{M} = \{q \mid f(q) = 0\}$$

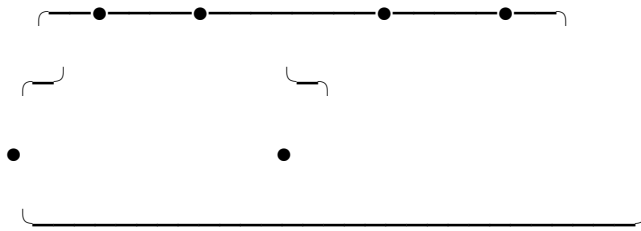
and the planner searches while configurations/local paths are constrained/projected onto the appropriate manifold.

So conceptually:

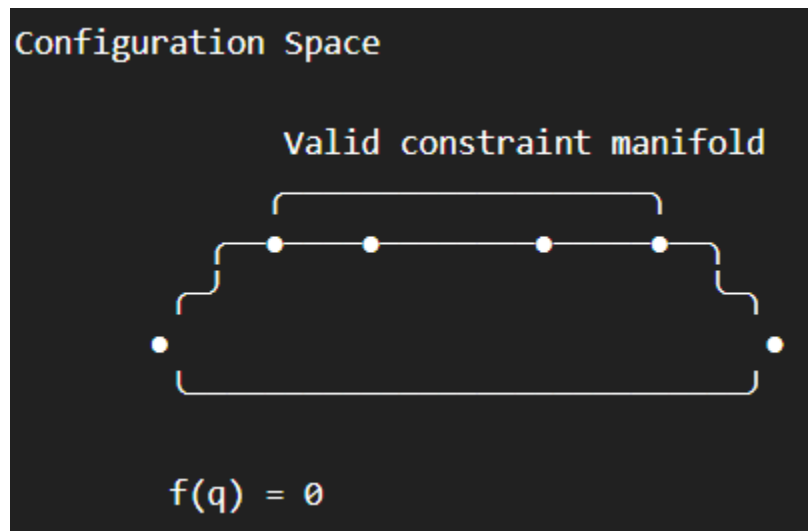
Configuration Space

Valid constraint manifold





$$f(q) = 0$$



This is one reason HPP is powerful for complex constrained systems and redundant robots. HPP was explicitly developed around geometric planning with kinematic/dynamic constraints, originally motivated by complex humanoid systems.

5. Constraint handling deserves its own row in your comparison

This may actually be one of the strongest differentiators.

Imagine your inspection requirements are:

Hard requirements

$d_{TCP-surface}=100 \text{ mm}$ $Z_{tool} // N_{surface}$ $q_{min} < q < q_{max}$

$$d_{TCP-surface} = 100 \text{ mm}$$

$$Z_{tool} \parallel N_{surface}$$

$$q_{min} < q < q_{max}$$

No collision.

And perhaps:

Rail and robot arm must not move simultaneously.

HPP

These kinds of requirements fit naturally into the HPP philosophy:

Configuration

↓

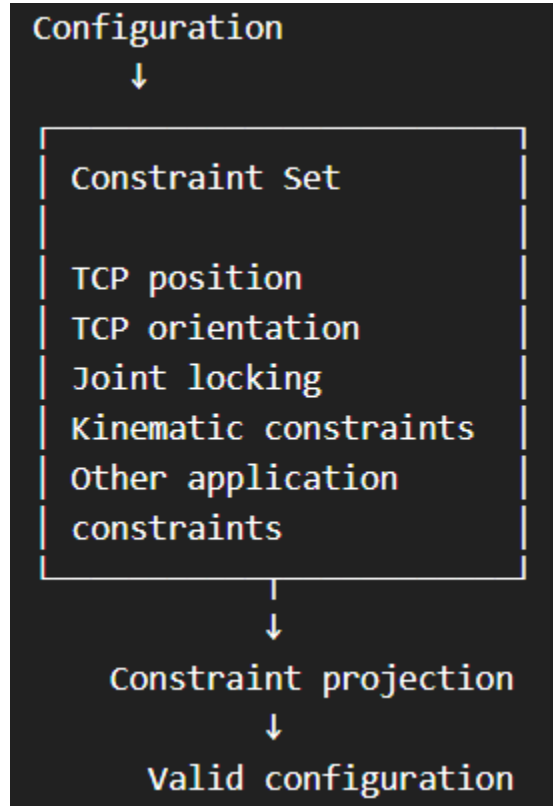
Constraint Set	
TCP position	
TCP orientation	
Joint locking	
Kinematic constraints	
Other application	
constraints	

↓

Constraint projection

↓

Valid configuration



HPP explicitly supports locked joints and implicit nonlinear constraints through its constraint architecture.

That's highly relevant to your NDT setup where you have previously imposed the rule that **rail and arm should not move simultaneously**.

Tesseract

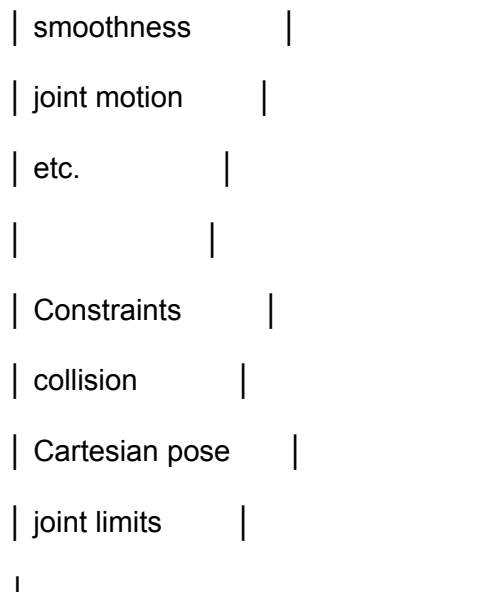
Tesseract is more optimization/pipeline oriented:

Initial trajectory

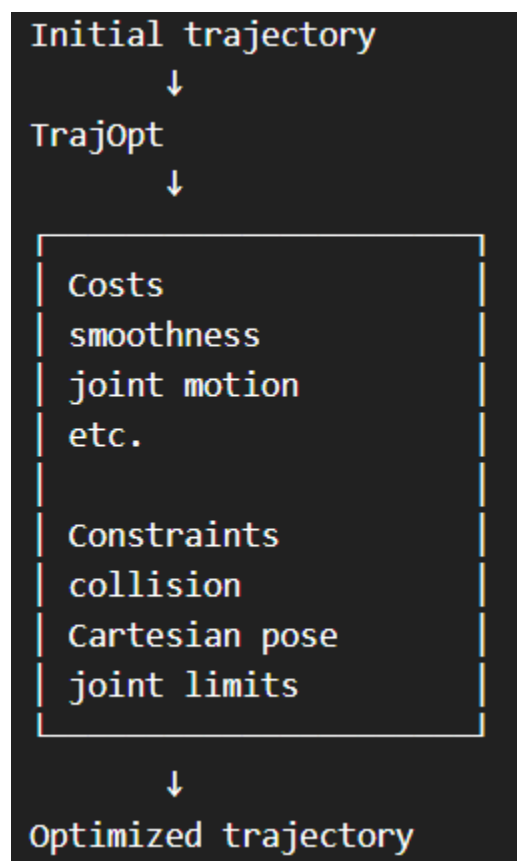


TrajOpt





Optimized trajectory



TrajOpt is explicitly one of Tesseract's central optimization-based planners, while TrajOpt-IFOPT provides an SQP-based formulation.

6. Trajectory optimization

This should be a separate comparison component rather than mixing it with initial path generation.

IRT

Tesseract has a very clear architecture:

Initial feasible path → TrajOpt → optimized trajectory

Initial feasible path → TrajOpt → optimized trajectory

For example, the standard free-space pipeline can effectively be:

OMPL → TrajOpt → TimeParameterization

OMPL → TrajOpt → TimeParameterization

while Cartesian planning can be:

Descartes → TrajOpt → TimeParameterization

Descartes → TrajOpt → TimeParameterization

This pipeline structure is documented directly in Tesseract's Task Composer.

LAAS

HPP separates the planner and path optimizer:

Planner → Initial Path → PathOptimizer → Optimized Path

Planner → Initial Path → PathOptimizer → Optimized Path

HPP provides path optimization mechanisms including shortcutting and spline/gradient-based optimization; its optimization machinery can retain problem constraints and incorporate collision constraints during optimization.

So I would characterize this difference as:

Tesseract: trajectory optimization is a central part of the industrial planning pipeline.

HPP: constrained path generation is central, followed by configurable path optimization.

That's a much more meaningful comparison than saying one "has optimization" and the other doesn't.

7. Therefore I'd structure your complete study like this

Don't start with software packages. Start with **functional components**:

#	Component	IRT Jules Verne	LAAS
1	Tool-path generation	CAD/surface → swathe/coverage path	CAD/surface → swathe/coverage path
2	IK / redundancy resolution	Tesseract kinematics + Descartes	HPP constraints / numerical projection
3	Cartesian process-path planning	Descartes	HPP constrained planning / EE trajectory
4	Free-space motion planning	OMPL	HPP planner / roadmap

5	Constraint handling	Tesseract planner/TrajOpt constraints	hpp-constraints / ConfigProjector
6	Collision checking	Tesseract environment/contact managers	hpp-fcl
7	Path/trajectory optimization	TrajOpt / TrajOpt-IFOPT	HPP PathOptimizer
8	Time parameterization	Tesseract time parameterization	HPP/application trajectory parameterization
9	Route optimization — 100+ defects	Gap / application-level solution needed	Your LAAS implementation: TSP/GTSP
10	Task/planning pipeline	Task Composer	HPP ProblemSolver / application orchestration
11	ROS/industrial integration	MovelIt 2 + Tesseract + ROS 2	HPP + application/ROS integration

For the **TSP/GTSP** row, I would be careful to phrase this as an **implementation comparison**—e.g. “not present in the current IRT JV stack” rather than “Tesseract has no route optimization.” Tesseract's documented core planning stack is focused on robot motion/process planning, not sequencing hundreds of inspection targets.

The first deep-dive should therefore be exactly this

Component 1 — Robot Path & Trajectory Planning

Break it into five subproblems:

A. Cartesian process motion

IRT:

TCP Path → Descartes

TCP Path → Descartes

LAAS:

TCP constraints → HPP constrained planning

TCP constraints → HPP constrained planning

B. Free-space motion

IRT:

OMPL

LAAS:

HPP sampling/roadmap planner

C. Redundancy resolution

IRT:

IK candidates → Descartes graph

IK candidates → Descartes graph

LAAS:

$f(q)=0$ → constraint projection

$f(q) = 0 \rightarrow \text{constraint projection}$

D. Constraint handling

IRT:

TrajOpt/Planner constraints

LAAS:

hpp-constraints+ConfigProjector

E. Optimization

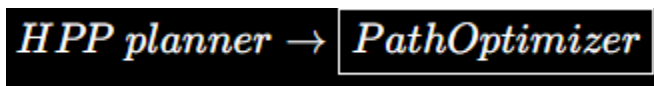
IRT:

Descartes/OMPL→TrajOpt



LAAS:

HPP planner→PathOptimizer



That gives you a **technical architecture comparison**, rather than a feature checklist. It will also make it much easier later to add the tool-path generation and TSP/GTSP layers without mixing different planning problems together.

i need a detailed comparison which is better and why

like collision check, singularity check, IK handling, Constraints, path generation and validation, trajectory optimization etc

i need a detailed comparison which is better and why like collision check, singularity check, IK handling, Constraints, path generation and validation, trajectory optimization etc

For the comparison you want, I would **not declare one framework universally better**. For an industrial inspection/coverage use case, the two stacks have different strengths:

- **IRT JV – MoveIt 2 + Tesseract** is stronger as an **industrial process-planning and trajectory-optimization pipeline**, especially for dense Cartesian tool paths, continuous collision checking, and TrajOpt-based optimization. Tesseract was explicitly created for complex industrial motion-planning applications and supports OMPL, Descartes, TrajOpt and TrajOpt-IFOPT in a common planning stack.
- **LAAS – HPP** is stronger as a **constraint-centric motion-planning framework**, particularly when the robot has redundancy, coupled mechanisms, nonlinear kinematic constraints, locked joints, or unusual configuration-space restrictions. HPP treats constraints as a first-class part of the planning problem and provides explicit/implicit constraints plus numerical solvers.

For your inspection/NDT type of application, I would compare them as follows.

Detailed comparison — Robot Path & Trajectory Planning

Capability	IRT JV — MoveIt 2 + Tesseract	LAAS — HPP	Better for your use case
Basic motion planning	OMPL + Tesseract planner ecosystem	Native HPP planners/roadmaps	≈ Both
Cartesian process path	Very strong: Descartes + process planning	Possible through constrained planning / EE constraints	Tesseract

Redundant IK handling	IK plugins + multiple candidate solutions; Descartes can search combinations	Constraint solver directly works in redundant configuration space	HPP for complex redundancy
Collision checking	Very strong; discrete + continuous collision checking	Strong collision/config/path validation with security margins	Tesseract slight advantage
Collision avoidance during optimization	Native strength of TrajOpt	Planner/path optimizer + collision validation	Tesseract
Singularity avoidance	Can be incorporated through planner/optimization objectives; MoveIt Servo explicitly checks singularity proximity	Can be modeled through constraints/costs but not simply a universal built-in “singularity checker”	≈ Depends on implementation
Joint limits	Native robot limits + planner constraints	Native configuration validation + constraints	≈
Complex nonlinear constraints	Supported, especially through optimization formulations	Core design strength	HPP
Constraint hierarchy	Less central to architecture	Strong hierarchical numerical constraint machinery	HPP

Custom coupled constraints	Possible but often requires defining optimization/planner constraints	Very natural through explicit/implicit constraints	HPP
Rail + arm coupling restrictions	Possible but requires custom constraints/task composition	Very natural to formulate in HPP	HPP
Path validation	Collision + limits + custom checks throughout planning	Explicit distinction between configuration validation, path validation, constraint projection	HPP architecturally cleaner
Trajectory optimization	TrajOpt / TrajOpt-IFOPT — major strength	PathOptimizer family	Tesseract
Smooth process trajectory	Very strong	Strong but less process-optimization-centric	Tesseract
Dense 500–5000 TCP waypoint path	Designed for process paths	Can handle constrained paths but not its main industrial workflow	Tesseract
Time parameterization	Dedicated package	Typically post-processing/application layer	Tesseract
Process pipeline orchestration	Task Composer	ProblemSolver + application-level orchestration	Tesseract

Highly constrained mechanisms	Good	Excellent	HPP
Research flexibility	Good	Very high	HPP
Industrial process-planning readiness	Very high	Requires more application engineering	Tesseract

The important thing is understanding **why** these differences exist.

1. Inverse Kinematics and redundant robots

This is probably the first major architectural difference.

Suppose you have a 7-DOF robot and a required inspection pose:

$T_{TCP} = T_d$

$$T_{TCP} = T_d$$

Because:

$n_q = 7 > n_{task} = 6$

$$n_q = 7 > n_{task} = 6$$

you can have many solutions:

$q(1), q(2), q(3), \dots$

$$q^{(1)}, q^{(2)}, q^{(3)}, \dots$$

Tesseract / Descartes approach

Tesseract's kinematics layer provides common FK/IK interfaces with implementations such as KDL and OPW, while its planner layer includes Descartes.

For a process path:

$P_1, P_2, \dots, P_N$

$$P_1, P_2, \dots, P_N$$

you can think of the problem as:

$P_1 \rightarrow \{q_{11}, q_{12}, \dots\}$ $P_2 \rightarrow \{q_{21}, q_{22}, \dots\} :$

$$\begin{aligned} P_1 &\rightarrow \{q_1^1, q_1^2, \dots\} \\ P_2 &\rightarrow \{q_2^1, q_2^2, \dots\} \\ &\vdots \end{aligned}$$

Then the Cartesian planner searches for a good sequence:

$Q_1^a \rightarrow Q_2^b \rightarrow Q_3^c \rightarrow \dots$

$$q_1^a \rightarrow q_2^b \rightarrow q_3^c \rightarrow \dots$$

This is excellent when your input is already:

“Here are 1,000 TCP points making up a swathe.”

HPP approach

HPP takes a more general mathematical view:

$f(q)=0$

where $f(q)$ could represent your TCP constraint.

Instead of thinking primarily about enumerating IK solutions, HPP's constraint machinery numerically finds configurations satisfying the constraint.

HPP supports both **implicit constraints** and **explicit constraints**, and its **HierarchicalIterative** solver solves implicit constraints through a Newton-like iterative method.

So:

$q_{\text{candidate}} \rightarrow \text{constraint projection} \rightarrow q_{\text{valid}}$

$q_{\text{candidate}} \rightarrow \text{constraint projection} \rightarrow q_{\text{valid}}$

Which is better?

For:

dense predefined industrial TCP path

I give **Tesseract/Descartes** an advantage.

For:

highly redundant robot where the valid robot states form a complicated constrained configuration manifold

I give **HPP** an advantage.

2. Constraint handling

Here I would give **HPP** a clear advantage.

This is one of HPP's strongest architectural features.

An HPP planning problem explicitly contains:

Robot+Constraints+Steering Method+Path Validation+Configuration Validation+Planner

The HPP **Problem** API explicitly carries the constraint set, path projector, path validation, configuration validation, obstacles, steering method and distance function. Constraints are passed to the steering method when applicable.

That separation is powerful.

Imagine your inspection robot has these rules:

$d_{TCP,surface} = 100 \text{ mm}$ $Z_{tool} \parallel N_{surface}$ $q_{min} < q < q_{max}$

$$\begin{aligned} d_{TCP,surface} &= 100 \text{ mm} \\ Z_{tool} &\parallel N_{surface} \\ q_{min} &< q < q_{max} \end{aligned}$$

and perhaps:

$\dot{q}_{rail} = 0 \Rightarrow \dot{q}_{arm} = 0$

$$\dot{q}_{rail} \neq 0 \Rightarrow \dot{q}_{arm} = 0$$

or equivalent phase constraints.

HPP is fundamentally designed for expressing unusual kinematic/configuration constraints.

Tesseract

Tesseract also handles constraints, particularly through optimization planners such as TrajOpt.

A trajectory optimization can be conceptually formulated as:

$q_{min} J(q)$

$$\min_q J(q)$$

subject to

$g(q) = 0$

$$g(q) = 0$$

and

$$h(q) \leq 0$$

$$h(q) \leq 0$$

with objectives such as:

$$J = w_1 J_{\text{smooth}} + w_2 J_{\text{collision}} + w_3 J_{\text{joint}} + w_4 J_{\text{Cartesian}}$$

$$J = w_1 J_{\text{smooth}} + w_2 J_{\text{collision}} + w_3 J_{\text{joint}} + w_4 J_{\text{Cartesian}}$$

TrajOpt uses sequential convex optimization with penalties and constraints.

That is powerful, but philosophically different.

Winner

HPP for complex **hard constraints** and **unusual robot mechanisms**.

Tesseract when constraints are naturally part of an **optimization problem**.

3. Collision checking

Here Tesseract is particularly strong.

Tesseract's collision infrastructure supports both:

Discrete collision checking

$$q_i \rightarrow \text{CollisionCheck}(q_i)$$

$$q_i \rightarrow \text{CollisionCheck}(q_i)$$

and **continuous collision checking**

$Q_i \rightarrow q_{i+1}$

$q_i \rightarrow q_{i+1}$

where the motion between configurations is checked rather than only checking sampled configurations. Tesseract's documented collision system supports both discrete and continuous contact checking.

This matters for an industrial trajectory.

Imagine:

q1 ----- q2

q1 = collision free

q2 = collision free

q1 ----- q2

q1 = collision free

q2 = collision free

but midway:

q1 ---- X ---- q2

↑

Collision

q1 ---- X ---- q2
↑
collision

A purely discrete check could theoretically miss it depending on sampling.

Continuous collision checking evaluates the swept motion.

HPP

HPP has strong collision integration too.

Its **Problem** provides:

- configuration validation,
- path validation,
- collision obstacles,
- collision-pair filtering,
- per-pair security margins.

The default configuration validation includes collision and joint-bound checking.

For example:

Qsteeringqj


$$q_i \xrightarrow{\text{steering}} q_j$$

is not simply accepted.

The local path goes through:

PathValidation(path)


$$\text{PathValidation}(\text{path})$$

before becoming a valid roadmap edge.

My assessment

For general geometric planning:

HPP = very strong.

For industrial trajectory optimization, especially where distance/contact information feeds the optimizer:

Tesseract = stronger.

Tesseract was actually created partly because TrajOpt needed richer contact/distance information and industrial collision capabilities than the MoveIt infrastructure available at the time.

So I'd score:

Tesseract: 5/5

HPP: 4.5/5

4. Collision avoidance vs collision checking

These are not the same.

Collision checking

asks:

$\text{Collision}(q) = \{ \text{true} \text{ false} \}$

$$\text{Collision}(q) = \begin{cases} \text{true} \\ \text{false} \end{cases}$$

Collision avoidance optimization

asks:

How should I move the trajectory away from the obstacle?

For example:

$d(q, O) > d_{\text{safe}}$

$$d(q, O) > d_{\text{safe}}$$

and possibly:

$J_{\text{collision}} = \sum_i \phi(d(q_i, O))$

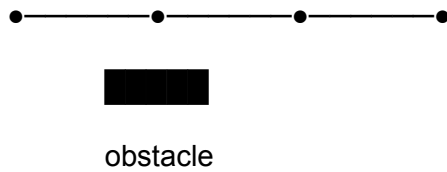
$$J_{collision} = \sum_i \phi(d(q_i, O))$$

This is where **TrajOpt** is particularly strong.

It uses contact-distance information to modify the trajectory iteratively.

Conceptually:

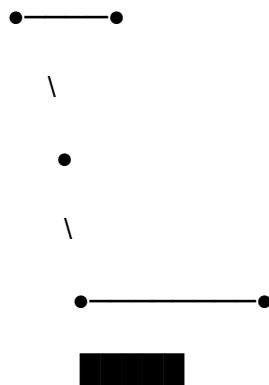
Initial path

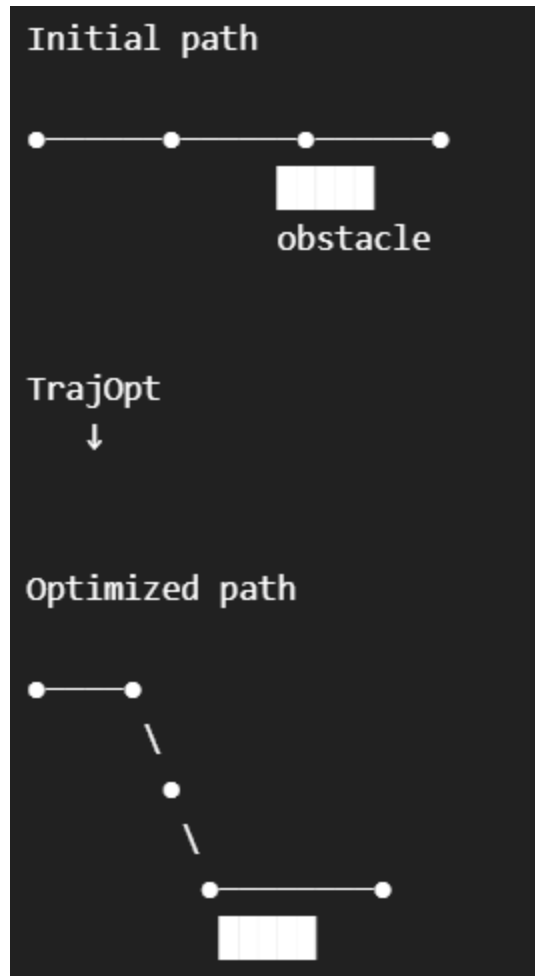


TrajOpt



Optimized path





This is a major advantage for Tesseract in industrial environments.

5. Singularity handling

This needs careful wording because I would **not claim either HPP or Tesseract simply has a magical universal singularity checker in its standard planner.**

For a Jacobian:

$J(q)$

$$J(q)$$

singularity occurs when its effective rank decreases.

You could use:

$\sigma_{\min}(J)$

$$\sigma_{\min}(J)$$

or Yoshikawa manipulability:

$w(q) = \det(JJ^T)$

$$w(q) = \sqrt{\det(JJ^T)}$$

and impose something such as:

$w(q) > w_{\min}$

$$w(q) > w_{\min}$$

or penalize:

$J_{\text{singularity}} = w(q) + \epsilon$

$$J_{\text{singularity}} = \frac{1}{w(q) + \epsilon}$$

Movelt

Movelt Servo explicitly includes singularity checking and scales motion as singularities are approached.

But that is **Servo**, not automatically every offline Movelt/Tesseract trajectory planner.

HPP

HPP's constraint architecture gives you the machinery to introduce avoidance conditions, numerical task constraints or custom validity criteria. But again, singularity avoidance depends on how the application formulates it.

Therefore

For your comparison, write:

Neither stack should be credited with automatic universal singularity avoidance at the global-planner level. Both require the desired singularity metric/avoidance policy to be incorporated into the planning or validation formulation.

That is much more technically accurate.

6. Path generation

There are two completely different paths to distinguish.

A. Free-space path

For example:

Home

↓

Approach point

IRT can use OMPL.

Movelt 2 supports configurable planning pipelines, including sampling-based planners and user-defined kinematic constraints. Collision checking is part of the planning pipeline.

Tesseract also directly wraps OMPL in its planner ecosystem.

HPP uses its own planner/roadmap architecture:

$q_{start} \rightarrow \text{Roadmap} \rightarrow q_{goal}$



$q_{start} \rightarrow \text{Roadmap} \rightarrow q_{goal}$

The HPP **PathPlanner** computes paths between an initial configuration and goal configurations and maintains a roadmap.

For this:

Winner

Tie, broadly.

Both are capable.

7. Process-path generation

Now consider:

Inspection swathe

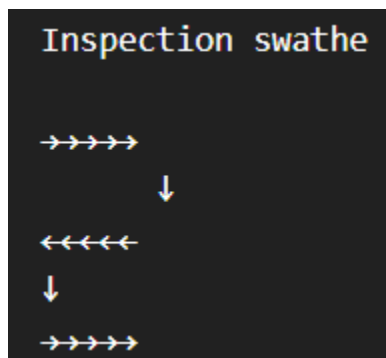
→→→→→

↓

←←←←←

↓

→→→→→



with hundreds or thousands of TCP poses.

This is where Tesseract becomes particularly attractive.

Its industrial training material explicitly describes:

- raster motions,
- transitions between rasters,
- entry/exit motions,

- complete process toolpaths,
- robot trajectories carrying joint position, velocity, acceleration and timestamps.

So:

Toolpath=Raster1+Transition+Raster2+...

$$\textit{Toolpath} = \textit{Raster}_1 + \textit{Transition} + \textit{Raster}_2 + \dots$$

maps very naturally to Tesseract's process-planning pipeline.

HPP

HPP can absolutely solve the robot motion once you encode the TCP requirements.

But **toolpath/process semantics are not as central to HPP's architecture.**

Therefore:

Winner: Tesseract

for:

- inspection,
- welding,
- painting,
- grinding,
- dispensing,
- AFP-like surface processing.

8. Path validation

This is an area where I particularly like HPP's architecture.

HPP separates:

ConfigurationValidation

from

PathValidation

and from

PathProjection

and

Constraints

explicitly in the planning problem.

That gives you a very clear pipeline:

Candidate configuration



Configuration validation



Constraint satisfaction



Steering / local path



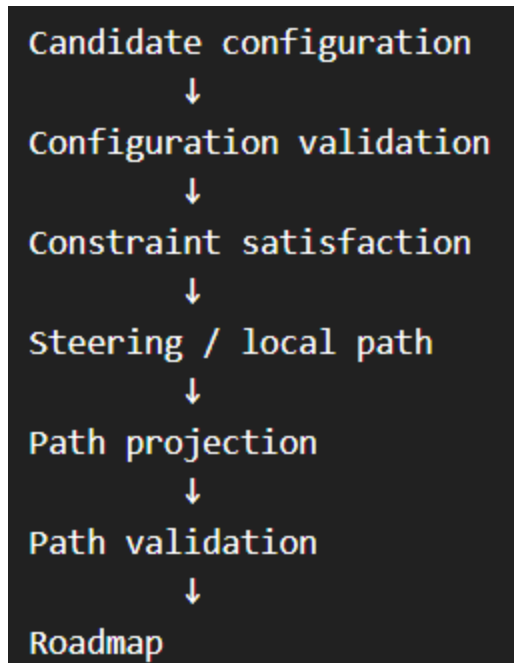
Path projection



Path validation



Roadmap



For research and verification, this decomposition is excellent.

Tesseract

Validation is more tightly integrated into its process/planning/optimization pipeline:

Initial program



planner



collision checking



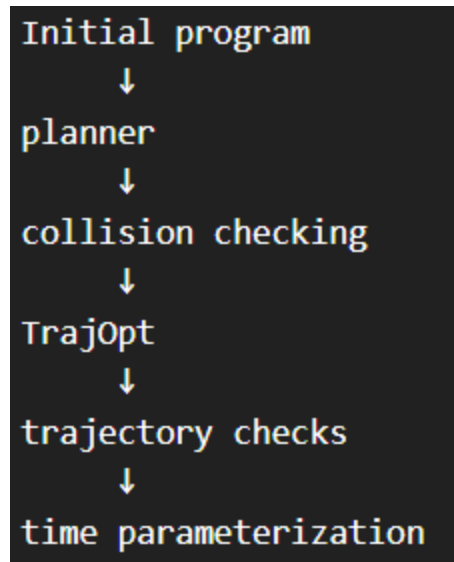
TrajOpt



trajectory checks



time parameterization



Winner

For clean explicit planning semantics:

HPP

For industrial processing workflow:

Tesseract

9. Trajectory optimization

This is one of the clearest victories for **Tesseract**.

TrajOpt solves:

$Q_{\min J(Q)}$

$$\min_Q J(Q)$$

for the complete trajectory

$Q=[q_1, q_2, \dots, q_N]$

$$Q = [q_1, q_2, \dots, q_N]$$

instead of finding only a collision-free geometric path.

Typical objectives can include:

$$J = w_v J_{\text{velocity}} + w_a J_{\text{acceleration}} + w_c J_{\text{collision}} + w_p J_{\text{Cartesian}} + \dots$$

$$J = w_v J_{\text{velocity}} + w_a J_{\text{acceleration}} + w_c J_{\text{collision}} + w_p J_{\text{Cartesian}} + \dots$$

Tesseract's TrajOpt implementation uses sequential convex optimization.

The practical industrial pipeline becomes:

Descartes / OMPL

↓

Seed

↓

TrajOpt

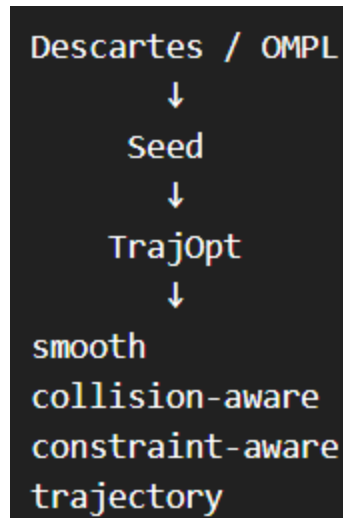
↓

smooth

collision-aware

constraint-aware

Trajectory



This is an excellent architecture.

HPP

HPP also separates planning and optimization:

Planner



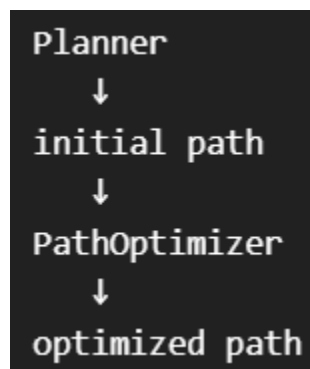
initial path



PathOptimizer



optimized path



But trajectory optimization is not as central to the HPP architecture as it is in Tesseract.

Winner: Tesseract

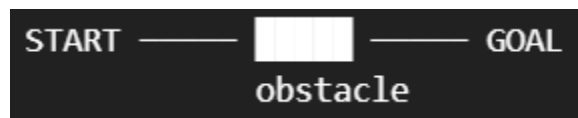
10. Initial guess / local minima

However, there is an important weakness of TrajOpt.

Trajectory optimization is generally **local optimization**.

If the initial trajectory is:

START ————  ———— GOAL
Obstacle



the optimizer may have difficulty finding a completely different homotopy class.

Therefore Tesseract often benefits from:

OMPL→TrajOpt

OMPL → TrajOpt

or:

Descartes→TrajOpt

Descartes → TrajOpt

where the first planner gives a reasonable seed.

This is one reason Tesseract provides multiple planners rather than using TrajOpt alone. Its planning stack explicitly includes OMPL, TrajOpt and Descartes.

HPP's sampling/roadmap planning is naturally aimed at global configuration-space connectivity.

So for highly cluttered environments:

Global feasibility search

HPP / OMPL-style sampling planners

may outperform pure trajectory optimization.

Final smooth trajectory

TrajOpt

has the advantage.

11. Redundant manipulator + constraints

Here I would strongly favor **HPP** when the problem becomes complex.

Imagine:

$q = [q_{rail}, q_1, q_2, \dots, q_7]$

$$q = [q_{rail}, q_1, q_2, \dots, q_7]$$

giving 8 DOF.

Your TCP constraint uses only six dimensions:

$f(q) = 0$

$$f(q) = 0$$

leaving redundancy.

Now also impose:

$q_{rail} \in \text{bounds}$ $TCP_z = \text{surface}_z + d$ $Z_{TCP} \parallel N$ $q_4 = \text{singularity-region}$ $\text{Robot} \cap \text{Environment} = \emptyset$

$$\begin{aligned}
 q_{rail} &\in \text{bounds} \\
 TCP_z &= \text{surface}_z + d \\
 Z_{TCP} &\parallel N \\
 q_4 &\neq \text{singularity} - \text{region} \\
 \text{Robot} \cap \text{Environment} &= \emptyset
 \end{aligned}$$

HPP is architecturally very comfortable with this kind of configuration-space problem because its steering methods, constraints and path validation are explicitly interconnected.

Therefore:

Winner: HPP

for highly constrained redundant systems.

12. But for your NDT inspection swathe...

Suppose the application is:

CAD surface

↓

Generate swathe

↓

5000 TCP points

↓

Robot must follow exact surface normal

↓

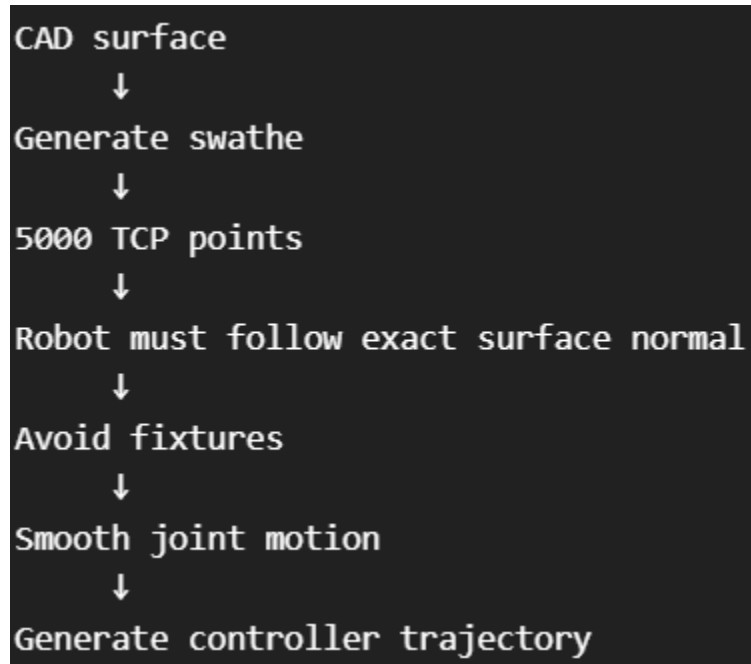
Avoid fixtures



Smooth joint motion



Generate controller trajectory



I would give the advantage to:

Tesseract

because the problem is fundamentally:

Convert a dense industrial Cartesian process toolpath into a high-quality executable robot trajectory.

That is precisely where the combination

Descartes→TrajOpt→TimeParameterization

Descartes → TrajOpt → TimeParameterization

is extremely compelling.

Tesseract's industrial material explicitly frames the planning goal around toolpaths made from raster, transition and entry/exit motions.

13. But consider your rail + robot restriction

Now introduce:

Rail and robot arm must not move simultaneously.

Your configuration is:

$q = [q_{rail}, q_{arm1}, \dots, q_{arm6}]$

$$q = [q_{rail}, q_{arm1}, \dots, q_{arm6}]$$

and you want something conceptually equivalent to:

$\dot{q}_{rail} = 0 \Rightarrow \dot{q}_{arm} = 0$

$$\dot{q}_{rail} \neq 0 \Rightarrow \dot{q}_{arm} = 0$$

and

$\dot{q}_{arm} = 0 \Rightarrow \dot{q}_{rail} = 0$

$$\dot{q}_{arm} \neq 0 \Rightarrow \dot{q}_{rail} = 0$$

Now I would shift toward:

HPP

because this is no longer merely trajectory optimization.

It is a **hybrid/coupled configuration-space constraint**.

HPP's explicit constraint, steering-method, path-projector and path-validation architecture gives you much more natural machinery for such unusual constraints.

That kind of constraint-heavy behavior is exactly why the HPP approach can remain attractive even if Tesseract has the stronger industrial process pipeline.

14. My scoring specifically for industrial inspection

I would use this scoring in your study.

Capability	Tesseract / IRT	HPP / LAAS
Cartesian swathe execution	5/5	4/5
Free-space planning	4.5/5	4.5/5
Redundant IK	4.5/5	5/5
Complex kinematic constraints	4/5	5/5
Collision checking	5/5	4.5/5
Continuous collision support	5/5	4/5
Collision-aware optimization	5/5	4/5

Singularity handling	4/5*	4/5*
Joint limit handling	5/5	5/5
Path validation architecture	4.5/5	5/5
Trajectory optimization	5/5	4/5
Dense industrial process path	5/5	4/5
Complex mechanism planning	4/5	5/5
Industrial workflow integration	5/5	3.5–4/5
ROS 2 ecosystem	5/5	3.5–4/5
Research/custom planning flexibility	4.5/5	5/5

*Singularity handling depends strongly on how the metric/avoidance constraint is actually configured.

15. Therefore, “which one is better?”

For a **standard industrial inspection robot following generated swathes**, my conclusion would be:

Tesseract approach is stronger

because you typically have:

CAD→Toolpath→Cartesian process planning→Trajectory optimization→Time parameterization

and Tesseract is architecturally designed around that industrial workflow.

But for a **highly redundant or mechanically unusual robot with strong coupled constraints**, my conclusion changes:

HPP approach is stronger

because:

constraint manifold+projection+constrained steering+path validation

are first-class elements of the planner architecture.

So the real engineering conclusion is:

Tesseract is more process-centric and optimization-centric. HPP is more configuration-space and constraint-centric.

For your comparison between **IRT Jules Verne and LAAS**, that should probably be the headline finding rather than “one is industrial and one is research.” It explains *why* each team made its architectural choices.

And there is an interesting possibility for your own architecture: **HPP's constraint/redundancy strengths and Tesseract's industrial trajectory-optimization strengths are complementary rather than mutually exclusive.** The next useful comparison would therefore be to trace **one identical NDT swathe through both pipelines step-by-step — TCP input → IK → redundancy resolution → singularity → collision → constraints → path validation → optimization → time parameterization — and identify exactly what algorithm/module is invoked at every step.** That will give you a very defensible technical comparison for your discussion with IRT JV and LAAS

now can you compare tool path generation between HPP and IRT?

IRT uses combination of open3D and open cascade

HPP uses customised GMM approach

Yes. For the **tool-path generation layer**, I would compare them as:

- **IRT JV:** geometry/CAD driven, using **Open3D + Open CASCADE**
- **LAAS:** customized **GMM-based surface representation / path-generation approach**

That difference is actually quite fundamental: **IRT starts from explicit geometry**, whereas **LAAS appears to start from a learned/fitted mathematical representation of the surface**.

I could verify the Open3D/Open CASCADE capabilities publicly, and IRT publicly describes work on “3D perception of complex surfaces and definition of robotic trajectories.” I could not find a public LAAS source documenting the exact customized GMM implementation you are using, so the GMM-side comparison below is based on the architecture you described rather than claiming it is a standard HPP capability.

Tool-path generation comparison

Capability	IRT JV – Open3D + Open CASCADE	LAAS – customized GMM	Advantage
Primary representation	CAD/BRep + point cloud/mesh	Probabilistic surface model	Depends
Exact CAD geometry	Excellent	Approximation	IRT
Measured point-cloud geometry	Excellent with Open3D	Very good if GMM fitted to scan	≈

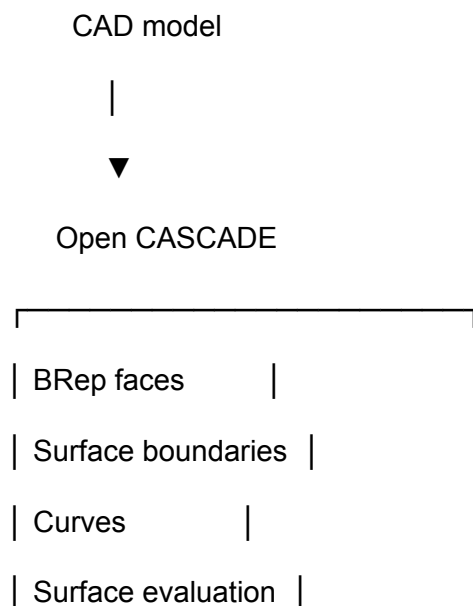
Noisy point clouds	Requires filtering/reconstruction	Naturally smooth representation	LAAS/GMM
Surface normal generation	Open3D normals / exact CAD normals from OCC	Derived analytically from fitted GMM/model	Depends
Curvature representation	Exact/near-exact from CAD surfaces	Smooth approximation	IRT for nominal CAD
Complex topology / holes / boundaries	Strong because OCC knows faces/edges/wires	Harder; requires explicit boundary handling	IRT
Swathe/raster generation	Geometry intersections, iso-curves, offsets, projection	Generate path directly on fitted surface/manifold	Depends
Path continuity	Must manage continuity across CAD faces	Potentially very smooth if GMM surface is continuous	GMM slight advantage
Sensor-based adaptation	Open3D works well, but CAD fusion needed	Strong if GMM is updated/re-fitted from measured surface	LAAS
Determinism	High	Depends on fitting/model parameters	IRT
Explainability	Very high	More mathematical	IRT

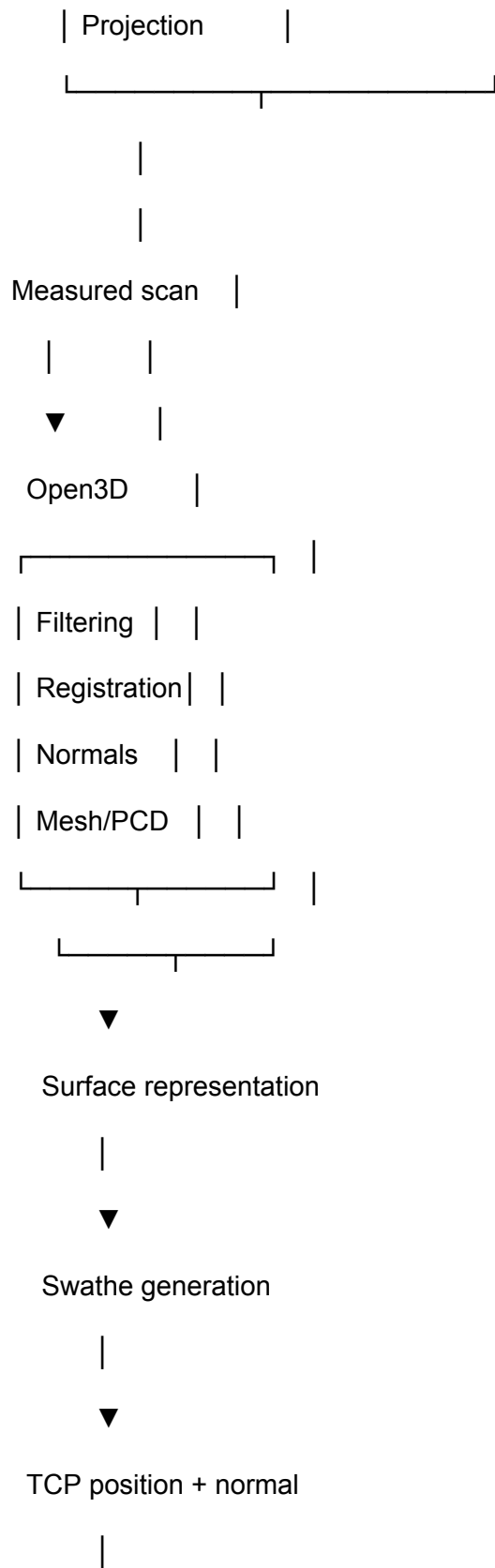
Industrial CAD interoperability	Excellent	Limited	IRT
Irregular/noisy real surfaces	Good	Potentially excellent	LAAS
Implementation complexity	Mature libraries but pipeline integration needed	Custom algorithm, significant ownership/maintenance	IRT
Generic industrial applicability	High	More specialized	IRT

The distinction becomes clearer if we trace the two pipelines.

1. IRT: Open CASCADE + Open3D

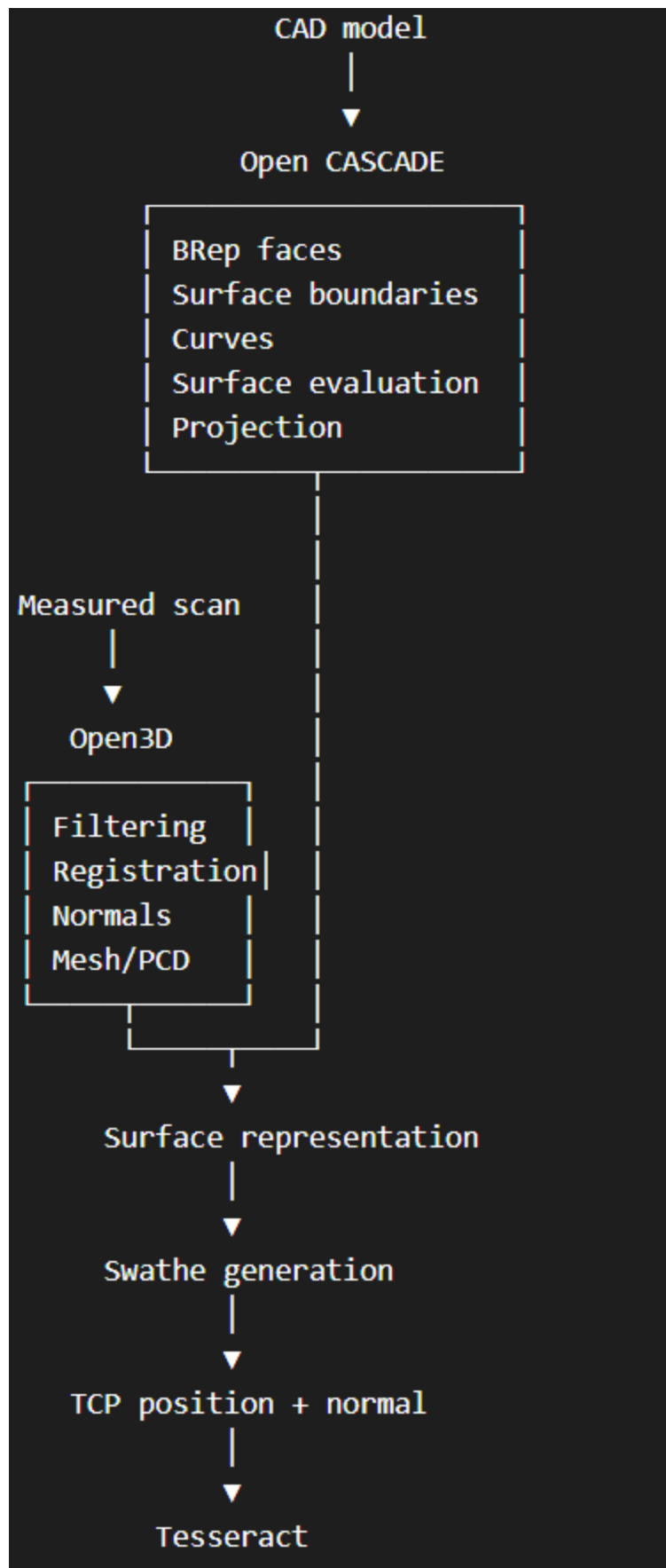
I would represent IRT's tool-path layer approximately as:







Tesseract



This combination makes architectural sense because **Open CASCADE** and **Open3D** solve different problems.

Open CASCADE

Open CASCADE represents CAD using BRep topology: vertices, edges, wires, faces, shells and solids. Its geometry layer represents curves and surfaces and supports interpolation, approximation, B-splines, coordinate evaluation and geometric extrema.

So given a CAD face:

$S(u,v)$

$$S(u, v)$$

you can generate a raster in parameter space:

$u=u_0, u_1, \dots, u_n$

$$u = u_0, u_1, \dots, u_n$$

and evaluate:

$P=S(u,v)$

$$P = S(u, v)$$

The surface derivatives:

$S_u = \partial u \partial S$ $S_v = \partial v \partial S$

$$S_u = \frac{\partial S}{\partial u}$$
$$S_v = \frac{\partial S}{\partial v}$$

give the surface normal:

$n = \parallel S_u \times S_v \parallel / \parallel S_u \times S_v \parallel$

$$n = \frac{S_u \times S_v}{\|S_u \times S_v\|}$$

That gives you directly:

$TCP_i = (P_i, n_i)$

$$TCP_i = (P_i, n_i)$$

for the inspection tool.

This is a major strength.

2. Why Open3D is added

Open CASCADE primarily tells you:

What the nominal CAD surface is.

But the manufactured part may actually be:

$S_{actual} = S_{CAD}$

$$S_{actual} \neq S_{CAD}$$

because of:

- manufacturing variation,
- assembly deformation,
- part positioning,
- fixture error,
- scan/reference error.

This is where Open3D becomes useful.

Open3D supports point-cloud processing, normal estimation, registration, reconstruction and related 3D-processing operations.

For example:

Sensor scan



Point cloud



Downsampling



Outlier filtering



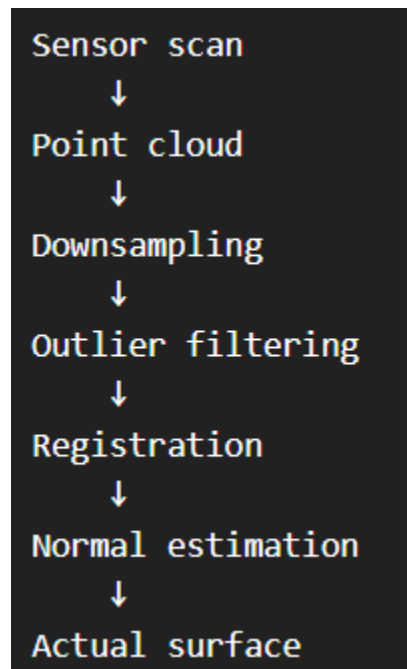
Registration



Normal estimation



Actual surface



Open3D estimates normals locally by fitting planes to neighborhoods of point-cloud samples.

So the IRT architecture can conceptually become:

CADOpenCascadeNominal surface

$$CAD \xrightarrow{\text{OpenCascade}} \text{Nominal surface}$$

plus

ScanOpen3DMeasured surface

$$Scan \xrightarrow{\text{Open3D}} \text{Measured surface}$$

followed by:

Nominal+Measured→Corrected toolpath

$$\text{Nominal} + \text{Measured} \rightarrow \text{Corrected toolpath}$$

That's an attractive **Scan & Plan** architecture, which aligns with IRT's stated work on 3D perception and automatic robot trajectory generation.

3. LAAS: GMM surface representation

Your LAAS architecture is conceptually quite different.

Instead of directly retaining all discrete measured points or working primarily from the CAD BRep, the scanned surface can be approximated by a Gaussian mixture:

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

$$p(x) = \sum_{k=1}^K \pi_k \mathcal{N}(x | \mu_k, \Sigma_k)$$

Each Gaussian has:

$$\mu_k = [x_k, y_k, z_k]$$

$$\mu_k = \begin{bmatrix} x_k \\ y_k \\ z_k \end{bmatrix}$$

and covariance:

$$\Sigma_k$$

$$\Sigma_k$$

which describes the local spatial distribution.

Conceptually:

Raw point cloud

• • • • •
• • • • • • • • • •
• • • • • • • • • •
• • • • •



GMM fitting



○ ○
○
○ ○

Gaussian components



Smooth surface/manifold representation



Generate swathe



So rather than retaining potentially millions of points:

$P = \{p_1, p_2, \dots, p_N\}$

$$P = \{p_1, p_2, \dots, p_N\}$$

you reduce the representation to:

$$G = \{(\pi_k, \mu_k, \Sigma_k)\}_{k=1}^K$$

$$G = \{(\pi_k, \mu_k, \Sigma_k)\}_{k=1}^K$$

with:

$$K \ll N$$

$$K \ll N$$

depending on the implementation.

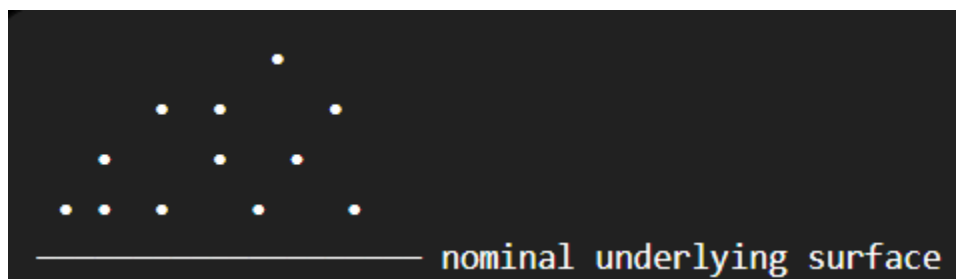
4. Why GMM can be attractive for inspection

This approach can have an important advantage with **noisy real geometry**.

Imagine the scanner gives:

•
• • •
• • •
• • • •

———— nominal underlying surface



Open3D can filter and estimate normals, but the representation remains fundamentally point/mesh based.

A fitted GMM can produce something closer to:

Measured samples

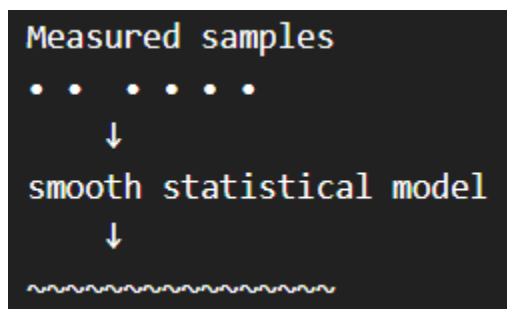
• • • • •



smooth statistical model



~~~~~



So the toolpath can follow the **underlying surface distribution** rather than every local measurement fluctuation.

That means the GMM naturally acts somewhat like a **surface regularizer**.

This can be very valuable in NDT because you don't want:

Noisescanner→TCP oscillation

*Noise<sub>scanner</sub> → TCP oscillation*

You want:

Noisescanner→Surface model→Smooth TCP

*Noise<sub>scanner</sub> → Surface model → Smooth TCP*

---

## 5. Surface normals are an interesting comparison

Inspection frequently requires:

$Z_{TCP} \parallel n_{surface}$

$$Z_{TCP} \parallel n_{surface}$$

so normal quality is critical.

### IRT – Open CASCADE

If working from CAD:

$n(u,v) = \parallel S_u \times S_v \parallel / \parallel S_u \times S_v \parallel$

$$n(u, v) = \frac{S_u \times S_v}{\parallel S_u \times S_v \parallel}$$

This is usually excellent because you have the analytical CAD surface.

**Advantage: IRT**

---

### IRT – Open3D

For measured point clouds, normal estimation is based on neighboring samples. Open3D provides this explicitly.

Conceptually:

$N(p_i) = \{p_j : \parallel p_j - p_i \parallel < r\}$

$$\mathcal{N}(p_i) = \{p_j : \parallel p_j - p_i \parallel < r\}$$

PCA is then applied to the neighborhood covariance.

The eigenvector corresponding to the smallest eigenvalue approximates the surface normal.

But it depends on:

- neighborhood radius,
- point density,
- scan noise,
- edge effects.

---

## LAAS – GMM

A fitted smooth representation can potentially produce a continuous tangent/normal estimate.

Therefore:

$p_i \rightarrow \text{GMM} \rightarrow n(x)$

$$p_i \rightarrow GMM \rightarrow n(x)$$

may give smoother orientations than raw point-cloud PCA.

**For actual noisy surfaces:**

I would potentially give **GMM the advantage**.

**For nominal CAD:**

**Open CASCADE clearly wins.**

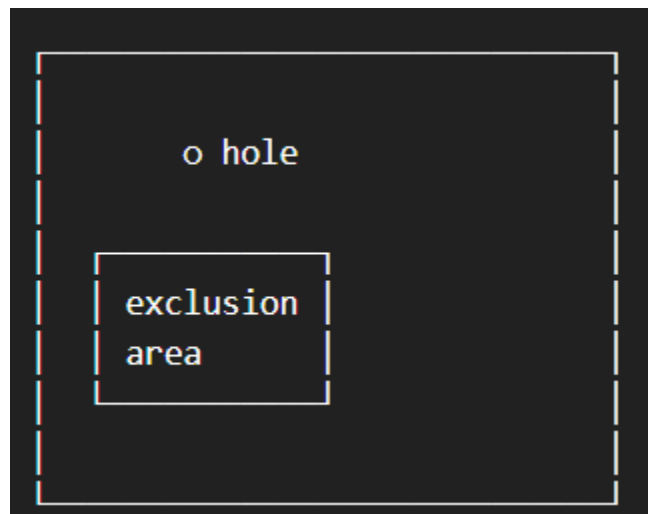
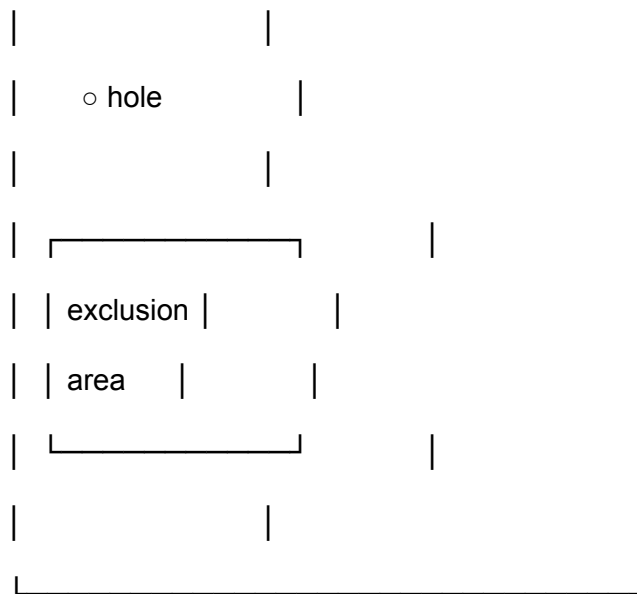
---

## 6. Boundary handling

This is where Open CASCADE is substantially stronger.

Suppose the aircraft panel contains:

\_\_\_\_\_



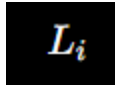
Open CASCADE knows:

- outer wire,
- inner wires,
- face boundary,
- holes,
- trims,
- CAD topology.

Because BRep explicitly maintains topology.

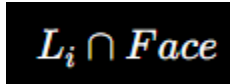
So a raster line:

Li



can be intersected with the face:

$L_i \cap \text{Face}$

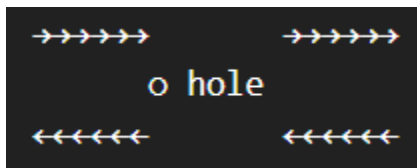


and trimmed correctly.

You obtain:



○ hole



rather than driving straight through invalid geometry.

## GMM

A GMM primarily tells you about the **distribution/shape**.

It does not automatically give strong topological semantics such as:

this is an outer boundary,

this is a hole,

this region belongs to another CAD face.

Those would need additional logic.

**Clear winner: Open CASCADE**

---



# 7. Curvature

This is another subtle distinction.

With CAD:

$S(u,v)$

$$S(u,v)$$

you have analytical derivatives.

Therefore you can obtain:

$\kappa_1, \kappa_2$

$$\kappa_1, \kappa_2$$

principal curvatures or equivalent geometric properties with high fidelity.

That allows adaptive spacing such as:

$d_{swathe} = f(\kappa)$

$$d_{swathe} = f(\kappa)$$

For flatter areas:

---

---

---

For strongly curved areas:

---

---

---

---

if your sensor/process requires increased density.

## **GMM**

Curvature can also be derived from an appropriate smooth fitted surface, but it is dependent on:

$K, \Sigma_k$ , fitting parameters

**$K, \Sigma_k$ , fitting parameters**

So GMM curvature represents:

**estimated physical surface**

rather than exact CAD geometry.

Again:

**Nominal part → Open CASCADE**

**Measured/noisy part → GMM can become attractive**

---

## **8. Generating the actual swathe**

Suppose inspection width is:

$W=50\text{mm}$

and required overlap is:

$o=20\%$

**$o = 20\%$**

Then line spacing is:

$d=W(1-o)$   $d=50(0.8)=40\text{mm}$

$$d = W(1 - o)$$

$$d = 50(0.8) = 40 \text{ mm}$$

The toolpath generator should produce something like:

START →→→→→→→→

↓ 40 mm

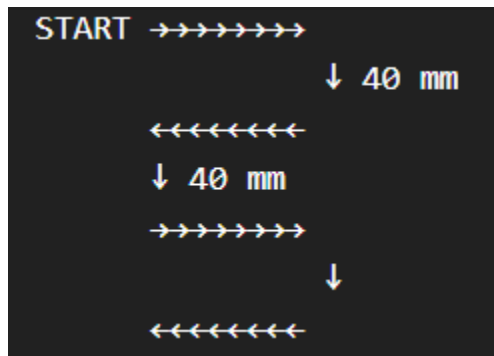
←←←←←←←←

↓ 40 mm

→→→→→→→→

↓

←←←←←←←←



## IRT approach

Conceptually:

CAD face → parameterization/geometric operations → parallel curves/intersections  
 → trim against boundaries → sample → (P<sub>i</sub>, n<sub>i</sub>)

*CAD face* → *parameterization/geometric operations*  
→ *parallel curves/intersections*  
→ *trim against boundaries*  
→ *sample*  
→  $(P_i, n_i)$

This is very deterministic.

---

## LAAS GMM approach

Conceptually:

Point cloud → GMM → surface/model → select scan direction → propagate path over surface  
→ compute tangent + normal →  $(P_i, n_i)$

*Point cloud* → *GMM*  
→ *surface/model*  
→ *select scan direction*  
→ *propagate path over surface*  
→ *compute tangent + normal*  
→  $(P_i, n_i)$

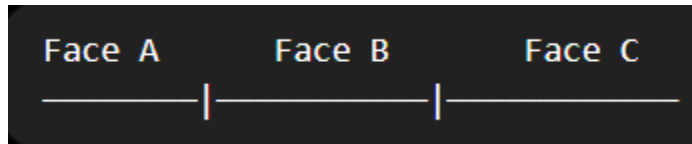
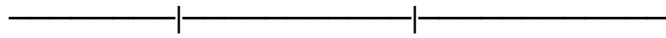
This can be much more flexible when the surface is only known from sensing.

---

## 9. Path continuity

Imagine the CAD is composed of several trimmed faces:

Face A    Face B    Face C



Even if the physical surface is smooth, BRep may contain separate topological faces.

You can therefore have:

$$n_A = n_B$$

$$n_A \neq n_B$$

numerically around the boundary, depending on representation/tolerances.

The Open CASCADE implementation must ensure:

C0/C1

$$C^0/C^1$$

continuity as required when traversing face boundaries.

## GMM

If one GMM representation covers the complete region:

~~~~~

rather than:

~~~~~|~~~~~|~~~~~

you may get smoother geometric transitions.

## Advantage:

**GMM for globally smooth measured surfaces**, assuming the fitted model is good.

---

## 10. Local defects / local surface changes

Suppose the nominal aircraft panel is:



but actual measurement shows:



### Open CASCADE

CAD won't know about it.

You therefore need:

CAD+Open3D measurement

and some correction logic.

### GMM

If GMM is fitted directly from measured geometry:

Measured surface → GMM

the deviation naturally becomes part of the model.

### Advantage:

**GMM**, for direct sensor-derived geometry.

---

# 11. But there is an industrial robustness issue

This is important.

With Open CASCADE, if you specify:

$u=0.1, v=0.5$

$$u = 0.1, \quad v = 0.5$$

you deterministically obtain:

$S(u, v)$

$$S(u, v)$$

same CAD  $\rightarrow$  same result.

With GMM:

PointCloudfitGMM

$$PointCloud \xrightarrow{\text{fit}} GMM$$

result depends on:

- number of components  $K$ ,
- initialization,
- covariance constraints,
- stopping criteria,
- point density,
- sensor noise,
- preprocessing.

If your GMM is trained/fitted using EM:

$$\theta_{t+1} = \arg \theta \max E[\log p(X, Z | \theta)]$$

$$\theta^{t+1} = \arg \max_{\theta} E[\log p(X, Z|\theta)]$$

model selection becomes an engineering issue.

This means **verification and certification are generally easier with the geometry-driven CAD approach.**

That is a major industrial advantage for IRT.

---

## 12. Computational scalability

Assume:

N=5,000,000

scan points.

Open3D can downsample:

5M→500k→100k

$$5M \rightarrow 500k \rightarrow 100k$$

and provides optimized point-cloud processing.

But subsequent operations still interact with point/mesh data.

With GMM:

5M points→K=100–1000 components

$$5M \text{ points} \rightarrow K = 100 - 1000 \text{ components}$$

potentially gives a very compact model.

After fitting:

$K \ll N$



can make subsequent geometry reasoning cheaper.

However, **fitting the GMM itself can be expensive**, especially if the model is frequently regenerated.

Therefore:

### **Static offline toolpath**

No huge GMM advantage.

### **Repeated model queries after fitting**

GMM can be efficient.

---

## **13. CAD interoperability**

For aerospace, this is a big deal.

You frequently start from:

- STEP,
- IGES,
- exact CAD surfaces,
- face IDs,
- manufacturing zones.

Open CASCADE is specifically a CAD geometry/topology kernel, so this is one of its strongest areas. Its BRep representation preserves faces, wires, edges and solids.

GMM doesn't inherently understand:

Aircraft CAD

└ Skin face 203

└ Stringer

└ Fastener zone

└ Cut-out

└ Inspection region

```
Aircraft CAD
├─ Skin face 203
├─ Stringer
├─ Fastener zone
├─ Cut-out
└─ Inspection region
```

You need additional semantic mapping.

**Strong winner: IRT/Open CASCADE**

---

## 14. So which approach is better for inspection?

It depends on where the surface comes from.

### Scenario A — exact CAD available and trusted

For example aircraft panel inspection.

I would choose:

Open CASCADE + Open3D

because you get:

- exact boundaries,
- exact surface topology,
- deterministic swathes,
- accurate normals,
- STEP/CAD interoperability,
- easier industrial validation.

**IRT wins.**

---

## Scenario B — no reliable CAD, surface obtained from scanning

For example:

Unknown part→3D scan→automatic inspection

Then I would strongly consider:

GMM-based approach

because:

- smooth surface fitting,
- robustness to measurement noise,
- compact representation,
- continuous normals,
- potentially better adaptation to actual geometry.

**LAAS approach can win.**

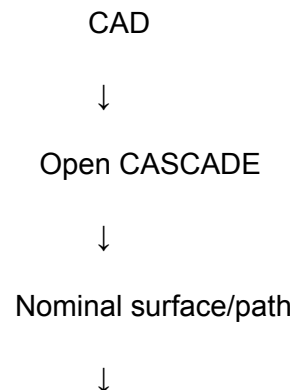
---

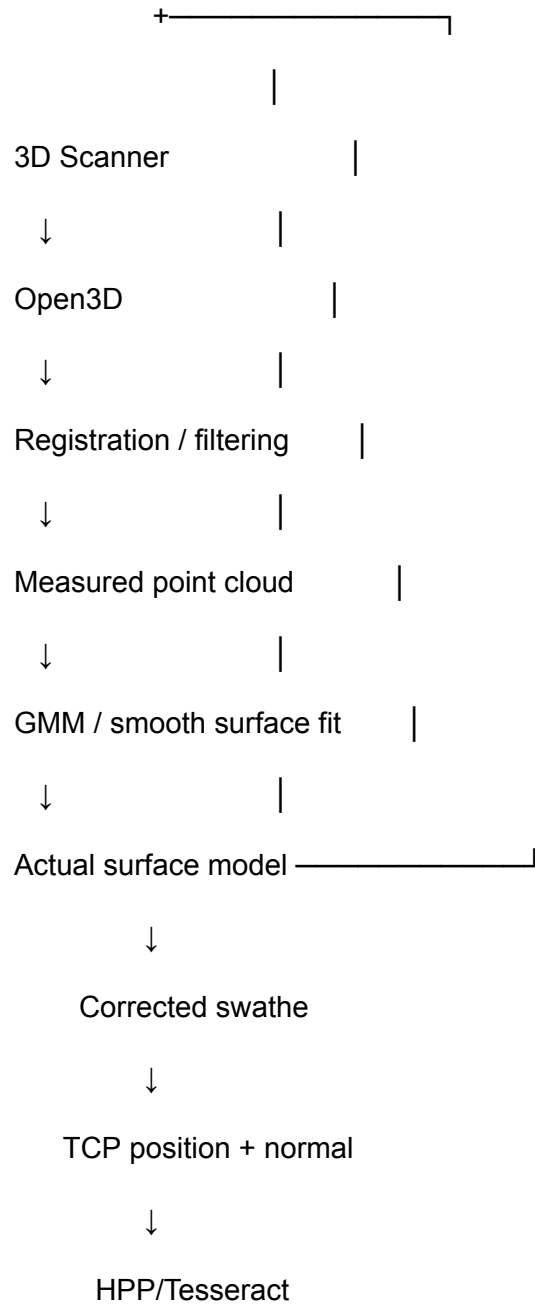
## 15. Scenario C — CAD exists but actual component deviates

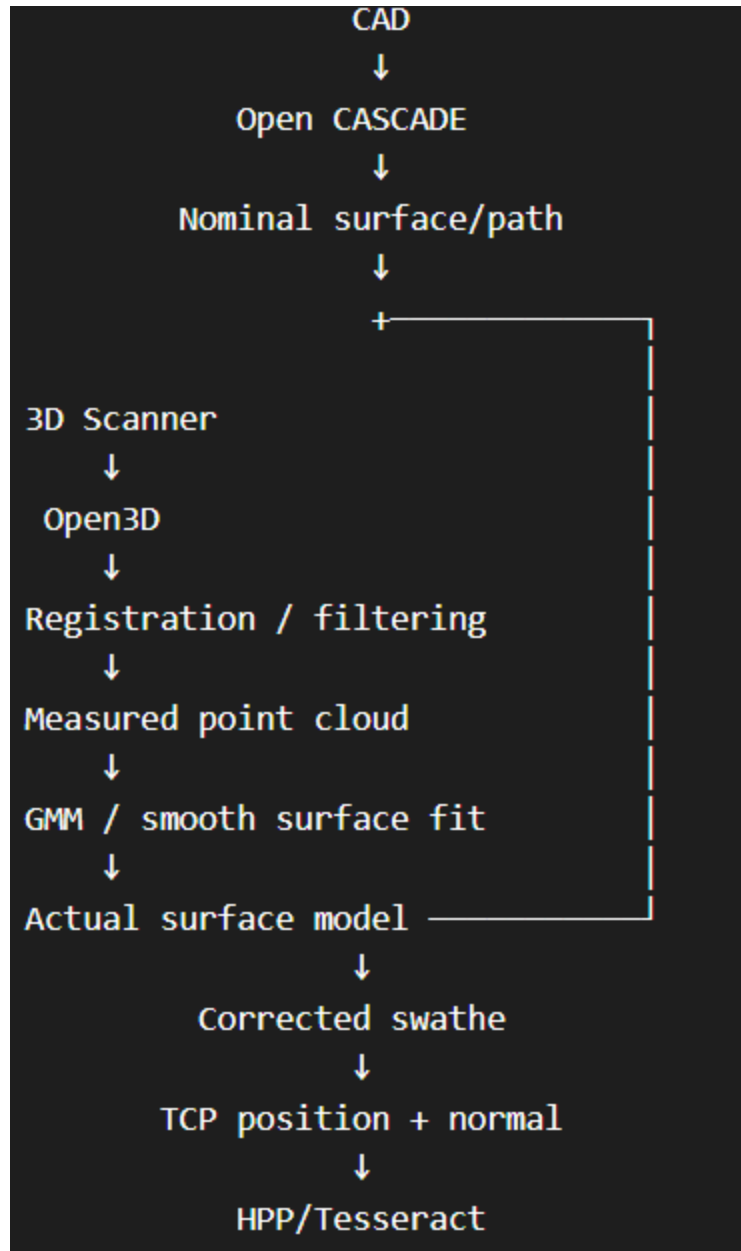
This is probably the **most interesting industrial case**.

And here I don't think either approach alone is ideal.

The strongest architecture would be:







That combines the best aspects of both:

CAD topology+Measured geometry+Smooth surface model

---

## 16. My scores for Tool Path Generation

For your IRT vs LAAS comparison, I would score them approximately like this:

| Tool-path criterion                      | IRT Open3D + OCC | LAAS GMM       |
|------------------------------------------|------------------|----------------|
| Exact CAD geometry                       | <b>5/5</b>       | 3/5            |
| CAD topology/boundaries                  | <b>5/5</b>       | 2.5/5          |
| STEP/IGES support                        | <b>5/5</b>       | 2/5            |
| Point-cloud handling                     | <b>5/5</b>       | 4.5/5          |
| Noise robustness                         | 4/5              | <b>5/5</b>     |
| Smooth surface reconstruction            | 4.5/5            | <b>5/5</b>     |
| Surface normal stability from noisy data | 4/5              | <b>4.5–5/5</b> |
| Exact nominal surface normals            | <b>5/5</b>       | 4/5            |
| Complex trims/holes                      | <b>5/5</b>       | 3/5            |
| Swathe/raster generation                 | <b>5/5</b>       | 4.5/5          |
| Cross-surface smoothness                 | 4/5              | <b>4.5/5</b>   |
| Adaptation to actual component           | 4.5/5            | <b>5/5</b>     |

|                           |            |            |
|---------------------------|------------|------------|
| Determinism               | <b>5/5</b> | 4/5        |
| Model compactness         | 3.5/5      | <b>5/5</b> |
| Industrial verification   | <b>5/5</b> | 3.5–4/5    |
| Generic applicability     | <b>5/5</b> | 4/5        |
| Online/adaptive potential | 4.5/5      | <b>5/5</b> |

## My conclusion for your comparison

For **industrial inspection tool-path generation from CAD**, I would rank:

IRT JV Open CASCADE + Open3D approach higher

because **the CAD kernel gives exact topology and geometry, while Open3D brings measured 3D data processing**. That's a very practical industrial combination. Open3D explicitly provides point-cloud registration, normal estimation and reconstruction, while Open CASCADE gives exact BRep surfaces, curves and topology.

But the **LAAS GMM approach has a genuine advantage in representing noisy, measured surfaces as a smooth mathematical model**. So I would not describe it as inferior; I would describe it as **better suited to sensor-derived/adaptive surface planning**, whereas the IRT solution is **better suited to deterministic CAD-driven industrial programming**.

That gives us a useful emerging comparison:

IRT JVCAD/geometry drivenOpenCascade+Open3DDescartes/TesseractTrajOptIndustrial  
process pipelineLAASConstraint/model drivenCustomized GMMHPPHPP PathOptimizerFlexible  
constrained planning

| IRT JV                             | LAAS                                 |
|------------------------------------|--------------------------------------|
| <i>CAD/geometry driven</i>         | <i>Constraint/model driven</i>       |
| <i>OpenCascade + Open3D</i>        | <i>Customized GMM</i>                |
| <i>Descartes/Tesseract</i>         | <i>HPP</i>                           |
| <i>TrajOpt</i>                     | <i>HPP PathOptimizer</i>             |
| <i>Industrial process pipeline</i> | <i>Flexible constrained planning</i> |

The next component I would compare is route/sequence optimization—especially your 100+ defect case—because that is where LAAS's TSP/GTSP layer appears to give a much clearer functional advantage over the current IRT JV architecture



**Now about the comparison of execution of the trajectories**

**in case of HPP it is customisable , we can use URTDE, ROS2, Fanuc socket communication etc**

**we can configure the sample rate of the trajectory**

**both TCP trajectory and joint trajectory is supported in HPP**

**in case of IRT they mentioned that we cannot configure the sample rate of a trajectory**

**currently only ROS2 is supported for sending trajectories to robot**

**only joint values are supported**

Yes. Based on what IRT JV told you, the **trajectory-execution layer is currently much more flexible on the LAAS/HPP side**. I would frame this as a comparison of the **execution interface and trajectory representation**, not as a limitation of Tesseract itself unless IRT confirms that the restriction is intrinsic to their Tesseract integration.

| <b>Capability</b>                    | <b>LAAS / HPP approach</b>                                              | <b>IRT JV current approach</b>                                                  | <b>Advantage</b> |
|--------------------------------------|-------------------------------------------------------------------------|---------------------------------------------------------------------------------|------------------|
| <b>Robot communication interface</b> | Customizable: UR RTDE, ROS 2, FANUC socket, proprietary interface, etc. | Currently ROS 2-based execution                                                 | <b>HPP/LAAS</b>  |
| <b>Controller independence</b>       | High; application can implement different robot-specific adapters       | Lower in current implementation because execution is coupled to ROS 2 interface | <b>HPP/LAAS</b>  |
| <b>Joint trajectory execution</b>    | Supported                                                               | Supported                                                                       | Tie              |

|                                             |                                                         |                                                                        |                 |
|---------------------------------------------|---------------------------------------------------------|------------------------------------------------------------------------|-----------------|
| <b>Cartesian/TCP trajectory execution</b>   | Supported through custom execution layer                | Currently joint values only, according to IRT                          | <b>HPP/LAAS</b> |
| <b>Trajectory sampling rate</b>             | Configurable by application/execution backend           | Not configurable in current IRT implementation                         | <b>HPP/LAAS</b> |
| <b>Trajectory resampling</b>                | Can interpolate/resample path before robot transmission | Currently constrained by generated/execution trajectory representation | <b>HPP/LAAS</b> |
| <b>Robot-specific optimization</b>          | High                                                    | Lower unless implemented in ROS 2 adapter                              | <b>HPP/LAAS</b> |
| <b>Standardized ROS integration</b>         | Requires integration work                               | Stronger standardized ROS 2 pipeline                                   | <b>IRT</b>      |
| <b>Ease of switching robots/controllers</b> | High if adapter exists                                  | Depends on ROS 2 driver availability                                   | <b>HPP/LAAS</b> |
| <b>Custom low-level execution</b>           | Strong                                                  | More restricted in current architecture                                | <b>HPP/LAAS</b> |

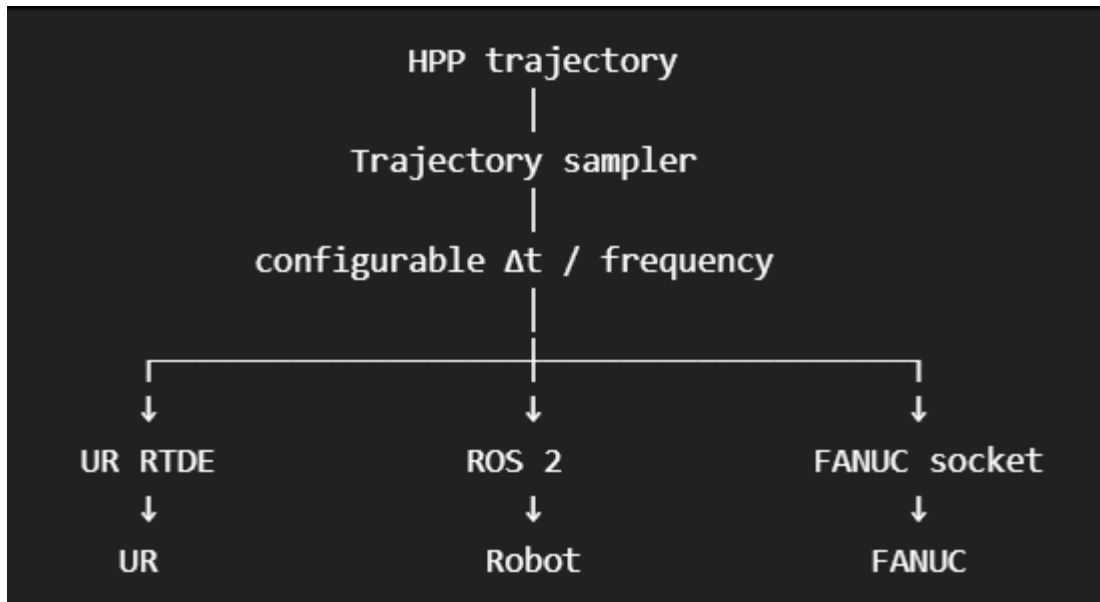
The biggest difference is therefore **execution architecture**.

## **LAAS / HPP**

HPP essentially produces the motion description, while the execution layer can be adapted to the robot:

HPP planned path→trajectory generation→sampling/resampling→robot-specific interface

For example:



This gives LAAS a very useful separation between:

### **Planning**

and

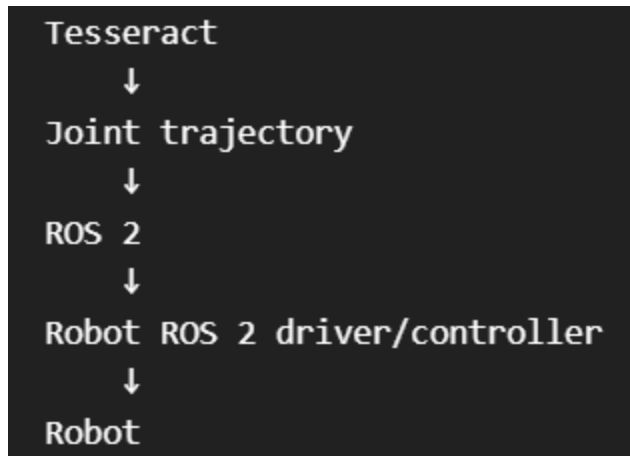
### **Execution**

The planner does not have to know whether the final robot is a UR, FANUC, or another platform.

That is a strong architectural advantage for R&T work where the same planning stack may be reused across several industrial robots.

### **IRT JV current approach**

From what they described to you, their pipeline is more like:



This is simpler and much more standardized.

That has benefits too: ROS 2 gives a common interface, and using joint trajectories aligns naturally with common ROS control interfaces such as [FollowJointTrajectory](#).

But it creates some limitations if their current implementation exposes only that route:

Tesseract→JointTrajectory→ROS2

rather than allowing arbitrary execution adapters.

## 1. Sampling-rate control is an important difference

Suppose the planner generates a continuous trajectory:

$q(t)$

The execution system eventually needs discrete commands:

$$q(t_0), q(t_1), q(t_2), \dots, q(t_N)$$

with:

$$t_{i+1} - t_i = \Delta t$$

and:

$$f_s = \frac{1}{\Delta t}$$

For example:

$$\Delta t = 0.002s \Rightarrow f_s = 500Hz$$

or:

$$\Delta t = 0.008s \Rightarrow 125Hz$$

or:

$$\Delta t = 0.01s \Rightarrow 100Hz$$

In the **LAAS architecture**, if the trajectory evaluation/sampling layer is under your control, you can do:

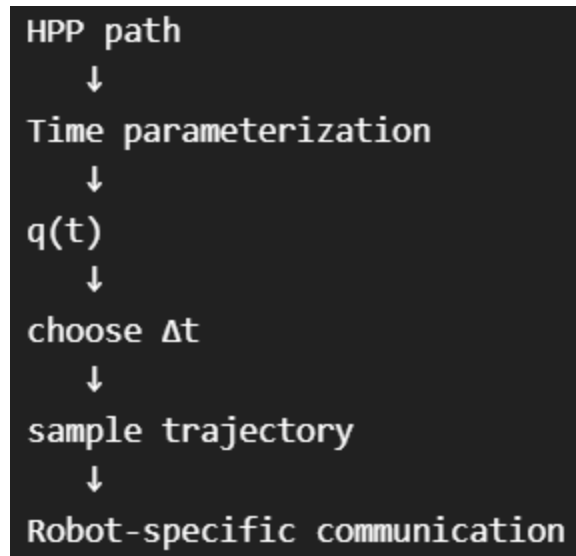
$$q_i = q(t_i)$$

for whatever appropriate:

$$\Delta t$$

the robot interface requires.

So:



That is useful because different robots/controllers expect very different communication/update patterns.

## Why it matters

A coarse trajectory sampling interval can produce larger interpolation steps:

$$\Delta q_i = q(t_{i+1}) - q(t_i)$$

whereas a smaller interval gives:

$$|\Delta q_i| \downarrow$$

and a denser commanded representation.

However, an important clarification for your report is:

**Command sampling rate should not be confused with the robot controller's internal servo frequency.**

For example, even if you send waypoints at 100 Hz, the robot's internal controller may interpolate those points at a much higher servo rate.

So you should compare **trajectory discretization/output frequency**, not claim that HPP controls the underlying robot servo loop.

---

## 2. TCP trajectory vs joint trajectory

This is probably the second strongest difference.

A Cartesian trajectory is:

$$T_{TCP}(t) \in SE(3)$$

such as:

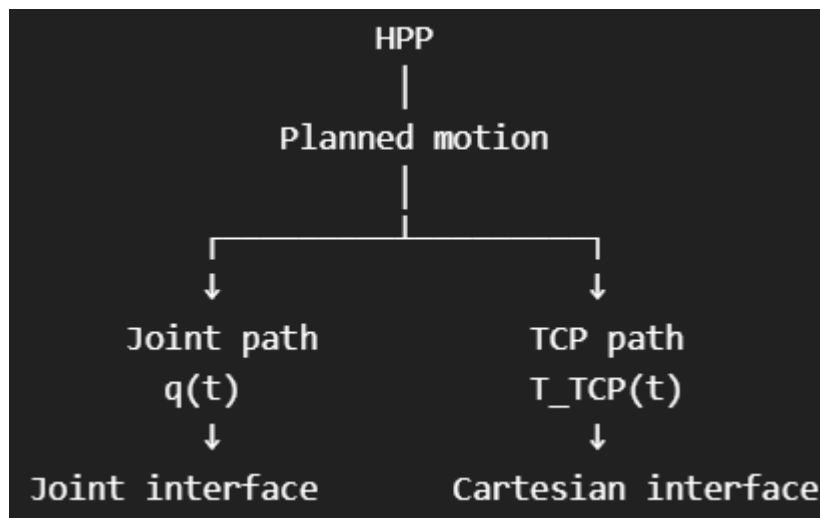
$$[x(t), y(t), z(t), R(t)]$$

A joint trajectory is:

$$q(t) = [q_1(t), q_2(t), \dots, q_n(t)]$$

### LAAS

Your execution layer can preserve either representation:



This is particularly useful for process applications.

For example, in NDT the actual process requirement is:

$$T_{TCP}(t)$$

because you care about:

- sensor position,
- sensor orientation,
- surface normal,
- standoff distance.

Similarly for dispensing:

$$TCP(t)$$

is often much more meaningful than the individual joints.

### IRT current approach

If they currently export only joint positions:

$$q_0, q_1, \dots, q_N$$

then the robot execution layer receives the already-resolved joint motion.

That is completely valid and often desirable for deterministic execution.

But you lose some flexibility at the execution interface.

You cannot simply tell another Cartesian-capable controller:

Follow this TCP trajectory.

Instead, the planning side has already committed to:

$$T_{TCP} \xrightarrow{IK} q$$

before the trajectory reaches the robot interface.

---

## 3. Why Cartesian execution can be useful

Imagine your NDT scanner must maintain:



$$d_{TCP-surface} = d_{ref}$$

During execution, your sensor measures:

$$d_{actual}$$

and you calculate:

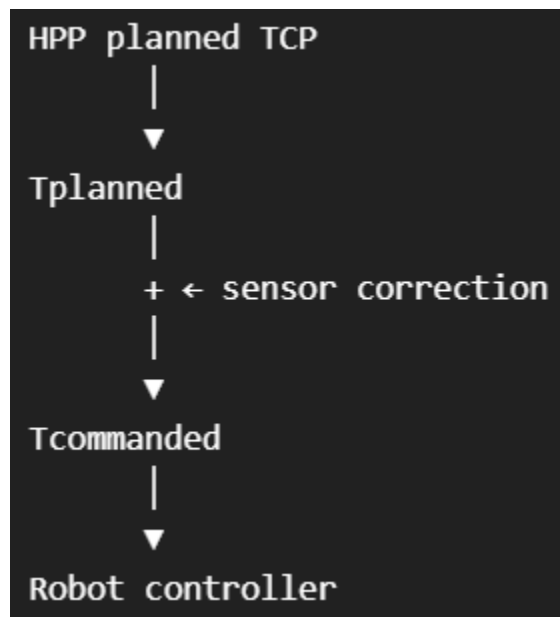
$$e = d_{ref} - d_{actual}$$

If your robot interface supports TCP trajectory/offset commands, you could modify:

$$T_{TCP,new} = T_{TCP,planned} \cdot T_{offset}$$

without necessarily replanning the complete joint trajectory.

Conceptually:



This kind of architecture is highly useful for **sensor-based path correction**.

With only joint-trajectory execution, you may instead need:

TCP correction → IK → new q

or use a separate controller capability.

So for adaptive robotics, direct Cartesian execution support can become important.

---

## 4. But joint-space execution has a major advantage too

I would not describe IRT's joint-only approach simply as worse.

There are good reasons to use it.

Once Tesseract has selected:

$q(t)$

the trajectory is deterministic.

The execution controller does not need to solve IK online.

That avoids:

- branch switching,
- unexpected elbow configurations,
- runtime IK failure,
- redundancy ambiguity.

Suppose the Cartesian command is:

$T_i$

A Cartesian controller might theoretically produce:

A black square containing the mathematical expression  $q_i^A$  in a stylized, bold, yellow font.

or:

A black square containing the mathematical expression  $q_i^B$  in a stylized, bold, yellow font.

depending on its IK/posture logic.

If Tesseract already sends:



that ambiguity is removed.

For highly optimized industrial trajectories, this is often desirable.

So:

### **Joint execution**

provides:

***deterministic robot configuration***

### **Cartesian execution**

provides:

***greater process/control flexibility***

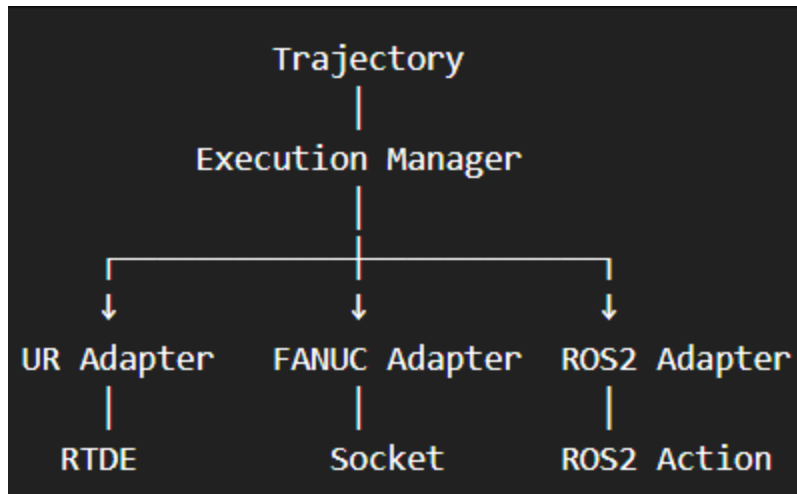
Neither is universally superior.

---

## **5. Robot communication adaptability**

This is where HPP/LAAS appears significantly better in your current implementation.

You can build an abstraction such as:



Then planning remains unchanged.

For a UR:

Trajectory→RTDE/URScript

For FANUC:

Trajectory→Socket/customcontroller

For ROS-controlled robot:

Trajectory→JointTrajectory

That is good **software architecture**, because:

*Planner ≠ RobotCommunication*

---

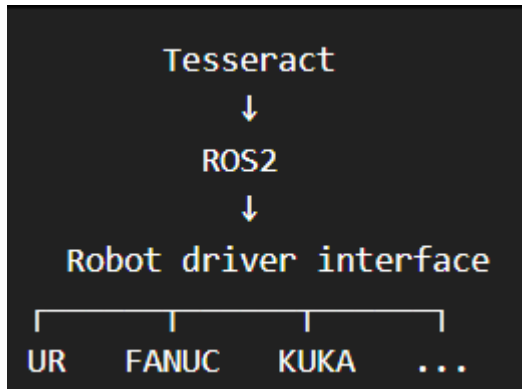
## 6. IRT's ROS 2-only approach has another advantage

It provides standardization.

Instead of maintaining:

```
UR communication
FANUC communication
KUKA communication
ABB communication
...
```

IRT can aim for:



So responsibility shifts to the robot driver.

This can reduce application-level maintenance.

Therefore I would characterize it as:

**IRT philosophy**

***standardized execution interface***

versus

**LAAS philosophy**

***customizable execution interface***

That is a more balanced comparison.

---

## 7. Trajectory resampling

This deserves its own criterion.

Assume HPP returns:

$q(t)$

and you need:

**UR interface**

500Hz

**another robot**

125Hz

**simulation**

100Hz

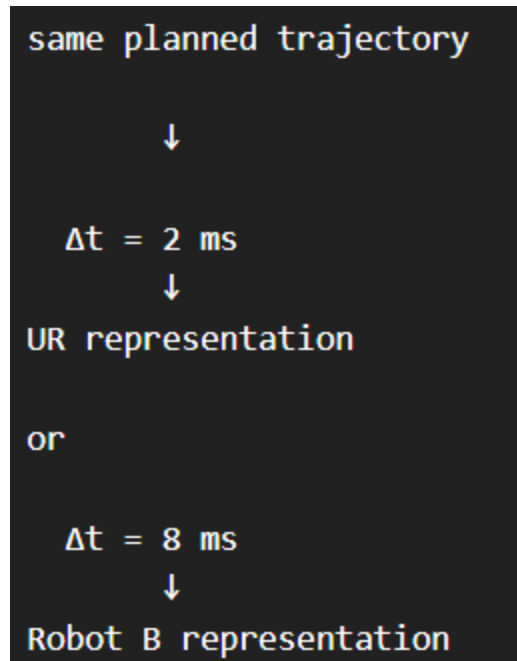
If the trajectory is represented continuously, the application can evaluate:

$$q(k\Delta t)$$

and potentially also:

$$\dot{q}(k\Delta t)$$
$$\ddot{q}(k\Delta t)$$

giving:



That's a strong advantage in terms of portability.

If IRT's current implementation fixes/disallows output resampling, then the output representation is less configurable.

**Advantage: LAAS**

---

## 8. But trajectory timing should not be arbitrarily changed

One caveat I would explicitly put into your comparison.

You cannot simply change:

$\Delta t$

without respecting:

$$|\dot{q}| \leq \dot{q}_{max}$$
$$|\ddot{q}| \leq \ddot{q}_{max}$$

and potentially:

$$|\ddot{q}| \leq \ddot{q}_{max}$$

Resampling should preserve the **time-parameterized trajectory**.

So the desirable capability is:

configurable **trajectory sampling of a valid time-parameterized motion**

rather than:

arbitrarily changing trajectory frequency.

That distinction will make your comparison technically stronger.

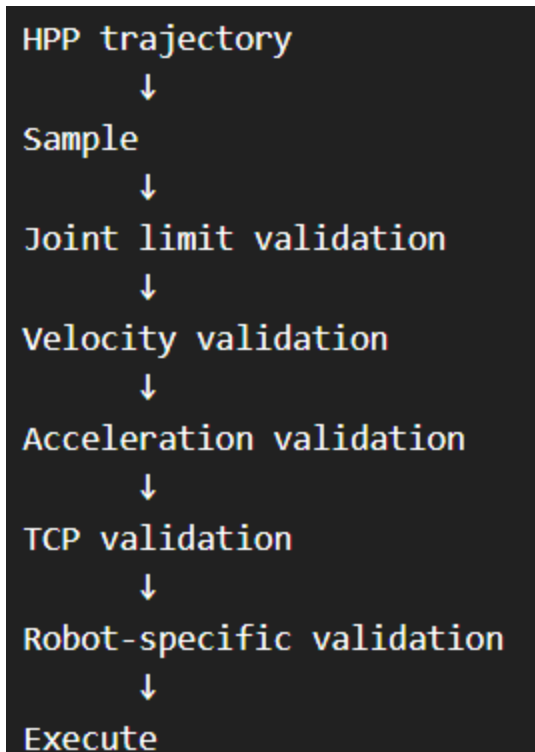
---

## 9. Execution validation

This should also become a component.

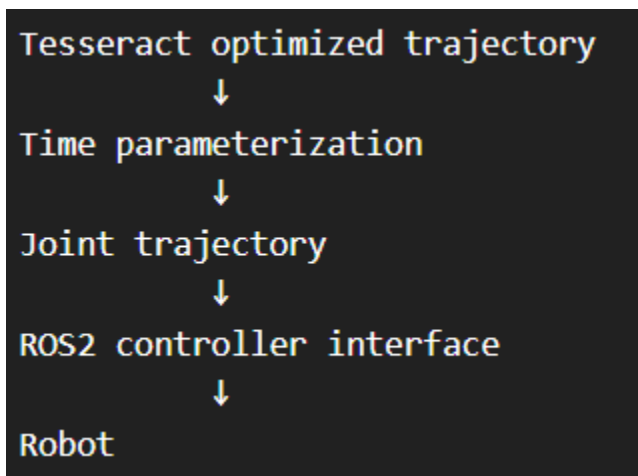
Before sending the trajectory, LAAS can potentially implement:





Because the execution layer is customized.

IRT's pipeline is more naturally:



Here validation responsibilities are distributed between:

- Tesseract,
- trajectory parameterization,
- ROS 2 controller,
- robot driver/controller.

So HPP gives **more application-level control**, whereas IRT's approach potentially gives **more standardization and clearer ROS ecosystem integration**.

---

## 10. Important distinction: HPP vs the LAAS execution framework

For your final comparison document, I would write this carefully.

Do **not** say:

“HPP supports UR RTDE, FANUC sockets and ROS2.”

That makes it sound as if those communication backends are built into HPP.

Instead say:

**“The LAAS architecture around HPP exposes the planned trajectory independently of the robot execution interface, enabling custom adapters such as UR RTDE, ROS 2 or FANUC socket communication.”**

Similarly:

**“Trajectory sampling can be configured in the LAAS execution layer.”**

That's technically much safer.

Likewise for IRT:

**“In the current IRT JV implementation, trajectory execution is exposed through ROS 2 and joint-space trajectory commands; configurable output sampling and Cartesian trajectory commands are currently not exposed.”**

rather than:

“Tesseract cannot change sample rate or execute TCP trajectories.”

Those may be **implementation choices**, not fundamental Tesseract limitations.

---

## 11. Updated comparison scoring

For the **Trajectory Execution** component specifically, I'd score:

| Execution criterion               | IRT JV                       | LAAS                         |
|-----------------------------------|------------------------------|------------------------------|
| Joint trajectory                  | <b>5/5</b>                   | <b>5/5</b>                   |
| Cartesian/TCP trajectory          | 2.5/5 current implementation | <b>5/5</b>                   |
| Configurable sampling             | 2.5/5 current implementation | <b>5/5</b>                   |
| UR native interface               | Depends on ROS driver        | <b>5/5 with RTDE adapter</b> |
| FANUC custom socket               | Not current architecture     | <b>5/5 if implemented</b>    |
| ROS 2 execution                   | <b>5/5</b>                   | 4.5/5                        |
| Robot independence                | 4/5 via ROS ecosystem        | <b>5/5</b>                   |
| Execution customization           | 3/5                          | <b>5/5</b>                   |
| Standardization                   | <b>5/5</b>                   | 3.5/5                        |
| Maintenance complexity            | <b>4.5/5</b>                 | 3.5/5                        |
| Adaptive TCP correction potential | 3/5                          | <b>5/5</b>                   |

Deterministic joint execution

5/5

5/5

Controller abstraction

ROS-centric

**Custom abstraction**

So for this particular layer:

***LAAS/HPP architecture has the stronger capability***

if your goal is **flexibility and robot independence**.

IRT has the advantage if your priority is:

***standardized ROS 2 industrial integration***

rather than maintaining multiple robot-specific communication stacks.

The larger comparison is becoming quite clear now: **IRT is stronger in the upstream industrial tool-path/trajectory optimization pipeline, while LAAS appears stronger in constraint customization and downstream execution flexibility**. That is a much more nuanced and useful conclusion than simply selecting one framework overall.

in case of patha planning can you do comparison study for robot sequencing problem as well

as i mentioned there is use case where we need to visit dfect ;ocations one by one and perform swathe scanning

currently HPP uses TSP + GTSP and can do scanning to that particular area

IRT doesn't suupport these kind of robot task sequence optimization and also IRT is not acapable of sacnning to a particular area

Usually there will be 100+defects

es. For the **robot sequencing / defect-visiting problem**, the LAAS/HPP approach is clearly stronger **in the current implementations you described**.

The key point is that this is not just a motion-planning problem. It is a **two-level optimization problem**:

Level 1: Which defect should I visit next? Level 2: How should the robot move and scan that defect?

For 100+ defects, the first level becomes very important because a poor visiting order can create a lot of unnecessary robot travel.

## Robot sequencing comparison

| Capability           | LAAS / HPP approach | IRT JV current approach                         | Advantage   |
|----------------------|---------------------|-------------------------------------------------|-------------|
| Visit many defects   | Supported           | Not currently supported as global optimization  | <b>LAAS</b> |
| Optimize visit order | <b>TSP / GTSP</b>   | No equivalent sequencing layer in current stack | <b>LAAS</b> |

|                                             |                                            |                                                                                                           |                              |
|---------------------------------------------|--------------------------------------------|-----------------------------------------------------------------------------------------------------------|------------------------------|
| 100+ target locations                       | Designed for this type of problem          | Would need application-level implementation                                                               | <b>LAAS</b>                  |
| Multiple candidate robot configs per defect | Can be considered through GTSP formulation | Not available as task-sequencing optimization                                                             | <b>LAAS</b>                  |
| Move to selected defect                     | HPP motion planner                         | Movelt/Tesseract can plan point-to-point motion                                                           | Tie                          |
| Scan only selected defect region            | Supported in current LAAS workflow         | According to IRT, not currently supported                                                                 | <b>LAAS</b>                  |
| Local swathe after arrival                  | Supported                                  | Existing process planning focuses on predefined process paths rather than automatic defect-local scanning | <b>LAAS currently</b>        |
| Global travel minimization                  | <b>Explicit optimization objective</b>     | Not provided by current IRT stack                                                                         | <b>LAAS</b>                  |
| Collision-aware transition between defects  | HPP planner after sequence selection       | Tesseract/Movelt can plan each transition                                                                 | Both                         |
| Task sequencing framework                   | Custom TSP/GTSP + HPP pipeline             | Task Composer / Movelt Task Constructor can sequence predefined stages, but not inherently solve TSP      | <b>LAAS for optimization</b> |

Extensibility

Custom  
optimization easily  
inserted

Would require additional  
sequencing module

LAAS currently

The distinction between **task sequencing** and **task composition** is particularly important.

Movelt Task Constructor can arrange complex robot tasks into sequential or parallel stages, and Tesseract has process/task composition capabilities. But these frameworks primarily execute or plan a **task structure that has already been defined**; they do not automatically solve the combinatorial question “what is the optimal order of these 100 defect regions?”

---

# 1. What exactly is the defect-sequencing problem?

Suppose inspection identifies:

$D = \{D1, D2, \dots, D120\}$

Each defect has a location:

$p_i = (x_i, y_i, z_i)$

and around every defect you need to perform a local swathe scan.

A naïve sequence might simply use the detection order:

$D1 \rightarrow D2 \rightarrow D3 \rightarrow \dots \rightarrow D120$

But imagine the defects are spatially distributed like this:

D1

D2

D7

D3

D5          D6

D4

Detection order may cause:

$D1 \rightarrow D2 \rightarrow D3 \rightarrow D4 \rightarrow D5 \rightarrow D6 \rightarrow D7$

with the robot repeatedly crossing the workpiece.

Instead you want something more like:

$D1 \rightarrow D7 \rightarrow D6 \rightarrow D5 \rightarrow D4 \rightarrow D3 \rightarrow D2$

if that minimizes the total robot travel.

Mathematically:

$$\pi_{\min} = \min_{\pi} \sum_{i=1}^{N-1} C(D_{\pi_i}, D_{\pi_{i+1}})$$

where  $\pi$  is the visiting order.

That is essentially a **Traveling Salesman Problem**.

---

## 2. Basic TSP in the LAAS approach

The simplest cost matrix is based on Euclidean defect distance:

$$C_{ij} = \|p_i - p_j\|$$

Then the TSP finds:

$$\pi^* = \arg \min_{\pi} \sum C_{\pi_i, \pi_{i+1}}$$

So:

Defect detector

↓



120 defect positions



Distance matrix



TSP



Optimized defect order



D17 → D32 → D4 → D91 → ...

This is already useful.

But for robotics, plain Euclidean distance is often not enough.

---

### 3. Robot-aware TSP is better than geometric TSP

Suppose two defects are physically close:

$\|p_A - p_B\| = 200\text{mm}$

but because of:

- obstacles,
- joint limits,
- rail constraints,
- robot posture,
- collision,
- accessibility,

the robot path is actually:

$L_{\text{robot}}(A,B) = 1.7\text{m}$

Meanwhile another defect may be farther in Cartesian space but much easier to reach.

Therefore a better cost is:

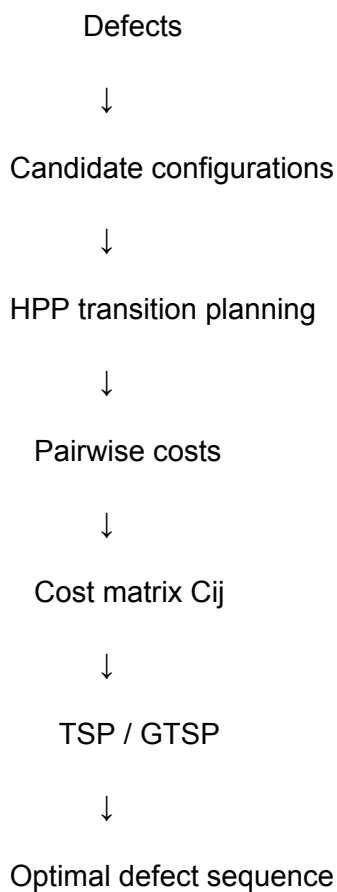
$$C_{ij} = \text{CostHPP}(q_i, q_j)$$

rather than simply:

$$C_{ij} = \| p_i - p_j \|$$

This is where the LAAS architecture becomes quite powerful.

Conceptually:



So HPP can provide the **robotically meaningful transition cost**, while TSP provides the **global visiting sequence**.

That separation is excellent architecture.

---

## 4. Why GTSP becomes even more interesting

For each defect, you may not have only one robot configuration.

For defect  $D_i$ :

$$D_i \rightarrow \{q_{i1}, q_{i2}, q_{i3}, \dots\}$$

For example, defect 25 may be scanned from several feasible:

- approach directions,
- elbow configurations,
- rail positions,
- swathe start orientations.

This creates clusters:

$$G_1 = \{q_{11}, q_{12}, q_{13}\} \quad G_2 = \{q_{21}, q_{22}, q_{23}\}$$

etc.

Now the planner needs to answer **two questions simultaneously**:

1. Which defect should I visit next?
2. Which configuration should I use for that defect?

That becomes a **Generalized Traveling Salesman Problem**.

Mathematically:

$$\min \sum C(q_{ia}, q_{jb})$$

subject to:

choose exactly one candidate from every defect group

This is much more powerful than ordinary TSP.

---

## 5. Why GTSP is particularly useful for redundant robots

Suppose defect D1 can be inspected using:

q1A,q1B,q1C

and defect D2:

q2A,q2B,q2C

A pure TCP-distance TSP sees only:

D1→D2

But GTSP can discover:

q1B→q2C

has much lower robot motion than:

q1A→q2A

So you are optimizing at the **robot configuration level**, not merely the XYZ level.

For a redundant robot, this is a major advantage.

---

## 6. Then HPP plans the transition

Once GTSP gives:

D17→D32→D4→D91

HPP solves each transition:

qexit17→qentry32

then:

qexit32→qentry4

while respecting:

- collision constraints,
- joint limits,
- rail constraints,
- robot kinematics,
- other application constraints.

So the global pipeline becomes:

Defect detection



100+ defect regions



Candidate inspection poses



Candidate IK / configurations



Pairwise robot-motion costs



TSP / GTSP



Optimized defect sequence



HPP



Free-space transition



Local defect swathe



Free-space transition



Next defect swathe

That is a very strong architecture for your inspection use case.

---

## 7. “Go to defect” and “scan defect” are separate planning problems

This distinction should appear clearly in your comparison.

For every defect:

$D_i$

you have:

### Transit motion

$q_{\text{previous}} \rightarrow q_{\text{approach}, i}$

This is a **free-space planning problem**.

Then:

### Inspection motion

$P_{i1} \rightarrow P_{i2} \rightarrow \dots \rightarrow P_{im}$

This is a **local process/coverage path problem**.

Then:

### Exit motion

$q_{\text{scan}, \text{end}} \rightarrow q_{\text{exit}, i}$

So one defect operation looks like:

Transit

↓

Approach

↓

Local swathe

↓

Retreat

↓

Transit to next defect

The complete mission is:

HOME

↓

Defect 37

↓

Scan region 37

↓

Defect 81

↓

Scan region 81

↓

Defect 15

↓

Scan region 15

↓

...

↓

HOME

This is much more than a simple motion-plan request.

---

## 8. Why IRT currently has a gap here

Tesseract itself is strong in **motion and process planning**. Its documented planners include OMPL, TrajOpt and Descartes, while its process-planning components cover industrial tasks such as painting, grinding and welding.

That means it can solve:

$A \rightarrow B$

and:

Given process toolpath  $\rightarrow$  robot trajectory

very well.

But that does not inherently solve:

Among 120 inspection regions, determine the globally optimal visiting order

This is a combinatorial **task/routing optimization** problem.

Similarly, MoveIt Task Constructor can build a sequence of interdependent stages and connect them with motion plans.

For example:

Move  $\rightarrow$  Scan  $\rightarrow$  Move  $\rightarrow$  Scan  $\rightarrow$  Move  $\rightarrow$  Scan

But the key distinction is that this does **not inherently determine the globally optimal permutation**:

$[1, 2, 3, \dots, 120]$

out of:

120!



possible orders.

So:

**Task sequencing is supported. Task-sequence optimization is not automatically the same thing.**

That is precisely the gap you are describing in IRT's current implementation.

---

## 9. Important terminology for your comparison

I would use these three terms separately:

### **Task composition**

Defines:

Move→Scan→Move→Scan

Tesseract Task Composer / MoveIt Task Constructor are relevant here. MoveIt Task Constructor explicitly decomposes complex tasks into interdependent stages.

### **Task sequencing**

Defines:

D4→D18→D7→...

### **Task-sequence optimization**

Finds:

$\pi^* = \arg \pi_{\min \text{Cost}}(\pi)$

This is where:

TSP/GTSP

belongs.

That terminology will make your technical comparison much stronger.

---

## 10. Scanning only around the defect

The second LAAS advantage you mentioned is equally important.

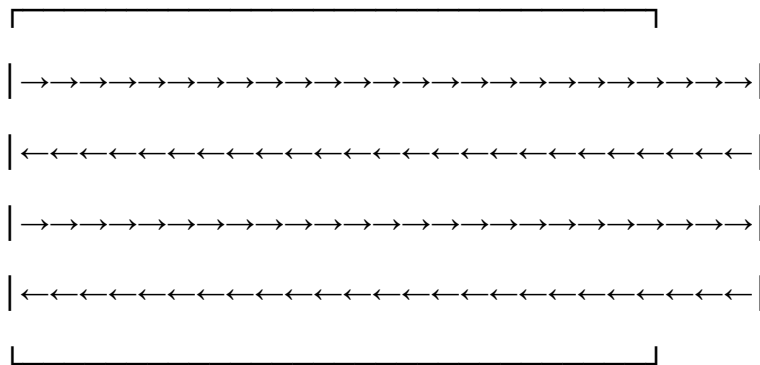
Suppose the defect center is:

$pd=(x_d,y_d,z_d)$

and inspection requires a local area:

$Ad=[x_d-\Delta x,x_d+\Delta x]\times[y_d-\Delta y,y_d+\Delta y]$

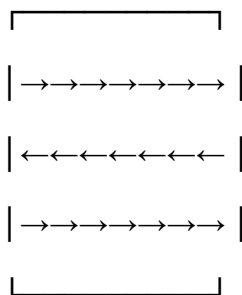
Instead of scanning the entire panel:



you generate:

defect

X



That is:

DefectLocation→ROI→LocalSurface→LocalSwathe

This is exactly the behavior needed for **automatic defect re-inspection or detailed NDT scanning**.

If the current IRT implementation only accepts a predefined global process path and does not dynamically generate a local scan around an arbitrary detected defect, then this is indeed an important functional gap.

But I would write:

**“Local defect-centered swathe generation is not available in the current IRT JV implementation.”**

rather than:

“Tesseract cannot scan a particular area.”

Because Tesseract is the downstream motion/process planner; custom application code could certainly generate a local toolpath and give it to Tesseract. Tesseract even has published Scan-and-Plan examples, so this is more accurately an **IRT application-stack gap** than a fundamental Tesseract limitation.

---

## 11. The LAAS architecture therefore has three optimization levels

This is actually worth emphasizing.

### Level 1 — Mission optimization

TSP/GTSP

Answers:

Which defect should be inspected in what order?

---

## Level 2 — Robot motion planning

HPP

Answers:

How do I safely move from defect A to defect B?

Considering:

- collisions,
  - constraints,
  - kinematics,
  - redundant configurations.
- 

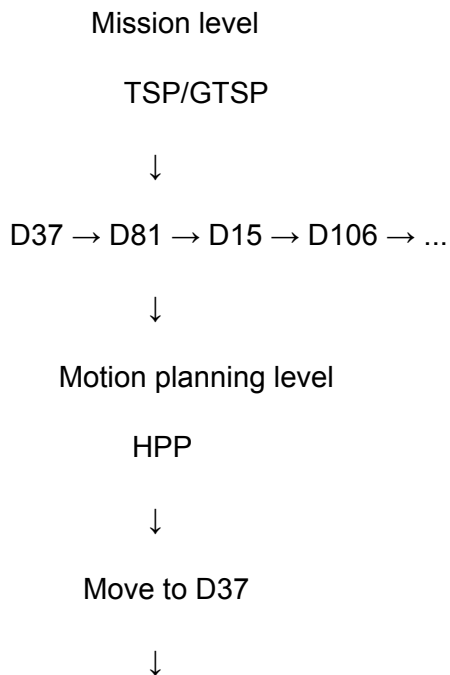
## Level 3 — Local inspection planning

Local swathe

Answers:

Once I arrive at defect B, how do I scan the required surface area?

That gives:



Inspection level

Local swathe



Move to D81



Local swathe



...

This is a powerful architecture.

---

## 12. The IRT architecture currently has mostly Levels 2 and 3

Based on what they told you, I would represent the current IRT stack approximately as:

Predefined process area/path



OpenCascade/Open3D



Process toolpath



Tesseract

└─ Descartes

└─ OMPL

└─ TrajOpt



Robot trajectory

What is missing is:

100 defects



??? GLOBAL ROUTE OPTIMIZER ???



Optimal defect order

and, from your discussion with them:

Selected arbitrary defect



Automatically create

defect-centered inspection ROI



Generate local swathe

is also currently missing.

So the IRT gap exists **above Tesseract**, not necessarily inside Tesseract itself.

---

## 13. Computational issue with 100+ defects

This deserves consideration.

For:

$N=100$

a full pairwise matrix has:

$N(N-1)$

directed transitions:

$100 \times 99 = 9,900$

For  $N=150$ :

$150 \times 149 = 22,350$

If every cost requires a full HPP path-planning query, preprocessing can become expensive.

Therefore the architecture can use multiple cost levels.

### **Stage 1 — cheap approximation**

$C_{ij} \text{approx} = w_p // p_i - p_j // + w_o \Delta \theta_{ij} + w_r | \Delta q_{rail} |$

Use this to solve the first TSP.

### **Stage 2 — robot-aware refinement**

For promising edges:

$C_{ij} \text{robot} = w_{LLHPP} + w_{TTTHPP} + w_{EE} \text{emotion}$

Then optimize again.

This avoids solving thousands of expensive motion plans unnecessarily.

---

## **14. GTSP becomes even more computationally demanding**

Suppose each of 100 defects has 10 feasible configurations:

$100 \times 10 = 1000$

candidate nodes.

Now possible inter-group edges become very large.

So practical GTSP requires:

- candidate pruning,
- infeasible-edge rejection,
- approximate costs,
- hierarchical optimization.

This is something I'd add as a **LAAS limitation**.

The approach is more capable, but also substantially more computationally complex.

---

## 15. Complete comparison table for sequencing

| Criterion                          | IRT JV current approach               | LAAS/HPP + TSP/GTSP                  | Better      |
|------------------------------------|---------------------------------------|--------------------------------------|-------------|
| Basic point-to-point planning      | Excellent                             | Excellent                            | Tie         |
| Sequence predefined tasks          | Task Composer / MTC possible          | Custom workflow possible             | Tie         |
| Determine optimal target order     | Not current functionality             | <b>TSP</b>                           | <b>LAAS</b> |
| Multiple configurations per target | Planner can find alternatives         | <b>GTSP explicitly exploits them</b> | <b>LAAS</b> |
| 100+ defect sequencing             | External/custom implementation needed | <b>Current capability</b>            | <b>LAAS</b> |



|                                |                                        |                                       |                       |
|--------------------------------|----------------------------------------|---------------------------------------|-----------------------|
| Robot-aware sequence cost      | Not currently integrated               | <b>Can use HPP transition costs</b>   | <b>LAAS</b>           |
| Collision-aware travel         | Tesseract/MoveIt                       | HPP                                   | Tie                   |
| Joint-aware route optimization | Requires custom layer                  | <b>Can integrate with GTSP</b>        | <b>LAAS</b>           |
| Redundancy-aware sequence      | Requires custom implementation         | <b>Strong with GTSP</b>               | <b>LAAS</b>           |
| Local defect-centered scan     | Not current IRT capability             | <b>Supported</b>                      | <b>LAAS</b>           |
| Local swathe generation        | Process toolpath supported if supplied | <b>Integrated in current workflow</b> | <b>LAAS currently</b> |
| Full-area raster scanning      | <b>Strong</b>                          | Strong                                | IRT/Tie               |
| Task orchestration             | <b>Task Composer / MTC strong</b>      | Custom                                | <b>IRT</b>            |
| Global optimality              | No route optimizer currently           | TSP/GTSP optimization                 | <b>LAAS</b>           |
| Computational complexity       | Lower                                  | Higher                                | <b>IRT</b>            |

|                                      |                    |                                           |             |
|--------------------------------------|--------------------|-------------------------------------------|-------------|
| Implementation complexity            | Lower              | Higher                                    | <b>IRT</b>  |
| Adaptability to dynamic defect lists | Requires extension | <b>Architecture naturally supports it</b> | <b>LAAS</b> |

## 16. How I would score it

For your **100+ defect inspection scenario**:

| Capability                  | IRT JV            | LAAS       |
|-----------------------------|-------------------|------------|
| Free-space planning         | 5/5               | 5/5        |
| Multi-target sequencing     | 2/5               | <b>5/5</b> |
| Global route optimization   | 1.5–2/5           | <b>5/5</b> |
| Redundancy-aware sequencing | 2/5               | <b>5/5</b> |
| Local defect inspection     | 2/5 current stack | <b>5/5</b> |
| Swathe around arbitrary ROI | 2/5 current stack | <b>5/5</b> |
| Collision-aware transitions | <b>5/5</b>        | <b>5/5</b> |

|                               |                          |              |
|-------------------------------|--------------------------|--------------|
| Task orchestration            | <b>5/5</b>               | 4/5          |
| Optimization for >100 defects | Requires new development | <b>4.5/5</b> |
| Computational efficiency      | <b>4.5/5</b>             | 3.5–4/5      |
| Mission-level optimization    | 2/5                      | <b>5/5</b>   |

For this component specifically:

LAAS/HPP + TSP/GTSP is clearly stronger

The reason is not that HPP itself is a TSP solver. The strength comes from the **architecture around HPP**, where:

Global sequencing→Robot planning→Local scanning

have been combined.

That gives LAAS the ability to solve the **whole inspection mission**, rather than only individual robot motions.

A concise statement for your overall study would be:

**IRT JV provides strong motion and process-trajectory planning once the task and process path are defined. The current LAAS architecture additionally addresses mission-level optimization: TSP/GTSP determines an efficient sequence through 100+ defects, HPP plans feasible robot transitions, and localized swathe generation enables inspection only around the selected defect regions. This gives LAAS a significant advantage for sparse, defect-driven inspection missions, whereas IRT's current architecture is better aligned with predefined continuous process paths.**

That distinction — **predefined continuous process vs dynamically selected sparse inspection regions** — is probably one of the most important differentiators in the entire IRT JV vs LAAS study.

also a detailed comparison (between option available in IRT planner and HPP) on implicit , explicit constraints

equality and in equality constraints with example

Keep in mind that HPP have cartesian path planning, it is called end effector trajectory algorithm, which supports different types of paths such as linear, curvved, spline (with different polynomials etc)

Yes. This is an important comparison because **HPP and the IRT stack do not treat constraints in exactly the same way**. Also, one correction to our earlier comparison is important: **HPP should not be characterized as lacking Cartesian path planning**. In your LAAS implementation, `EndEffectorTrajectory` is specifically used for Cartesian path generation/following and supports straight, curved and spline/polynomial trajectories.

The most important conceptual point is:

Implicit/Explicit=Equality/Inequality

They describe **two different classifications of constraints**.

HPP has a particularly formal treatment of these concepts. Its constraint package explicitly defines both **implicit constraints** and **explicit constraints**, and its implicit constraints support component-wise `Equality`, `EqualToZero`, `Inferior`, and `Superior` comparison types.

## 1. High-level comparison

| Capability             | IRT JV — MoveIt2 + Tesseract                              | LAAS — HPP                                       | Advantage  |
|------------------------|-----------------------------------------------------------|--------------------------------------------------|------------|
| Equality constraints   | Yes, especially through optimization/waypoint constraints | <b>Native and formal</b>                         | <b>HPP</b> |
| Inequality constraints | Yes, particularly optimization, bounds, collision margins | <b>Native Superior/Inferior comparison types</b> | HPP slight |

|                                             |                                                          |                                                                    |                                   |
|---------------------------------------------|----------------------------------------------------------|--------------------------------------------------------------------|-----------------------------------|
| Implicit nonlinear constraints              | Possible through optimizer/custom constraint formulation | <b>First-class concept</b>                                         | <b>HPP</b>                        |
| Explicit constraints / variable elimination | Not a central planning abstraction                       | <b>First-class <b>Explicit</b> constraint mechanism</b>            | <b>HPP</b>                        |
| Hierarchical constraints                    | Can emulate through weights/stages                       | <b>Hierarchical iterative solver with priorities</b>               | <b>HPP</b>                        |
| Constraint projection                       | Planner-dependent                                        | <b>Core capability</b>                                             | <b>HPP</b>                        |
| Constraint manifold planning                | Less central                                             | <b>Core philosophy</b>                                             | <b>HPP</b>                        |
| Costs vs hard constraints                   | Major TrajOpt strength                                   | Available but constraint satisfaction more central                 | <b>Tesseract for optimization</b> |
| Cartesian waypoint constraints              | <b>Very strong</b>                                       | <b>Very strong through EndEffectorTrajectory + HPP constraints</b> | Tie                               |
| Cartesian tolerances                        | Natural in process planning / TrajOpt                    | Can be represented with masks/inequalities/custom constraints      | Tesseract convenient              |

|                                      |                                             |                                                                 |                            |
|--------------------------------------|---------------------------------------------|-----------------------------------------------------------------|----------------------------|
| Joint constraints                    | Yes                                         | Yes                                                             | Tie                        |
| Collision constraints                | <b>Very strong optimization integration</b> | Strong path/configuration validation                            | Tesseract slight           |
| Custom mechanism constraints         | Possible                                    | <b>Very strong</b>                                              | <b>HPP</b>                 |
| Cartesian curve/path formulation     | Descartes/process waypoints                 | <b>EndEffectorTrajectory: linear, curved, spline/polynomial</b> | Both, different philosophy |
| Optimization of constraint violation | <b>TrajOpt strength</b>                     | Primarily project/satisfy                                       | <b>Tesseract</b>           |

So my overall conclusion for this component would be:

**HPP provides the richer mathematical architecture for defining and solving complex kinematic constraints, while Tesseract provides the stronger optimization-oriented mechanism for combining Cartesian requirements, collisions, smoothness and other objectives into trajectory optimization.**

Tesseract itself describes its planning philosophy as optimization-first, where collisions, smoothness, joint limits and process constraints are solved together.

---

## 2. First understand equality vs inequality

Consider a robot TCP:

$$T(q) = [x(q), y(q), z(q), R(q)]$$

An **equality constraint** says:

$$f(q) = b$$

For example:

$$z_{TCP}(q) = 500 \text{ mm}$$

The TCP must be exactly at the required height, within numerical tolerance.

An **inequality constraint** says:

$$f(q) \leq b$$

or

$$f(q) \geq b$$

For example:

$$d_{robot, obstacle}(q) \geq 50 \text{ mm}$$

The robot is free to be anywhere as long as it maintains at least 50 mm clearance.

---

### 3. Equality constraint example — inspection surface

Suppose your NDT sensor must remain exactly:

$d=100 \text{ mm}$

away from a surface.

You could define:

$$f(q) = d_{TCP, surface}(q)$$

and impose:

$$f(q) = 0.1$$

This is an equality constraint.

Another example is tool orientation.

If the inspection sensor must remain normal to the surface:

$$z_{tool}(q) = n_{surface}$$

or equivalently an orientation error:

$$e_R(q) = 0$$

Again:

$$e_R(q) = 0$$

is an equality constraint.

---

## 4. Inequality constraint example

Suppose the standoff doesn't need to be exactly 100 mm, but can vary:

$$95 \text{ mm} \leq d \leq 105 \text{ mm}$$

This becomes two inequalities:



$$d(q) \geq 0.095$$

and

$$d(q) \leq 0.105$$

Similarly, collision clearance:

$$d_{collision}(q) \geq 30 \text{ mm}$$

joint limits:

$$q_i^{min} \leq q_i \leq q_i^{max}$$

or perhaps inspection incidence:

$$|\theta_{TCP-normal}| \leq 5^\circ$$

These are inequality constraints.

---

## 5. HPP has explicit comparison types

This is one place where HPP is particularly clean.

An HPP `Implicit` constraint contains:

$h(q)$

a right-hand side, and a **comparison vector**.

The HPP API supports:

$$\{Equality, EqualToZero, Inferior, Superior\}$$

for individual components of the constraint.

So you can have one constraint vector:

$$h(q) = \begin{bmatrix} h_1(q) \\ h_2(q) \\ h_3(q) \\ h_4(q) \end{bmatrix}$$

with comparisons:

$$c = \begin{bmatrix} \textit{Equality} \\ \textit{Equality} \\ \textit{Inferior} \\ \textit{Superior} \end{bmatrix}$$

This means different dimensions of the **same differentiable function** can have different mathematical requirements. HPP also supports masking rows so selected dimensions of a constraint can be ignored.

That is very powerful for Cartesian tasks.

---

## 6. Example: inspection orientation with one free DOF

Suppose your scanner must:

- maintain XYZ position,
- keep its Z-axis aligned to the surface normal,
- but rotation around the scanner Z-axis is free.

You don't want to constrain all 6 Cartesian DOFs.

You effectively want:

$$x = x_d$$

$$y = y_d$$

$$z = z_d$$

and two orientation DOFs constrained:

$$R_x = R_{x,d}$$

$$R_y = R_{y,d}$$

but:

$$R_z = \text{free}$$

Conceptually:

Cartesian DOF

Cartesian DOF

X = constrained

Y = constrained

Z = constrained

Roll = constrained

Pitch = constrained

Yaw = FREE

HPP's masking mechanism fits this formulation naturally: components excluded by the mask are not used to determine constraint satisfaction.

This is particularly useful for redundant industrial robots because the unconstrained orientation can be exploited to improve:

- joint posture,
  - manipulability,
  - collision clearance,
  - rail location,
  - singularity avoidance.
- 

## 7. Now: what is an implicit constraint?

This is where terminology often causes confusion.

An implicit constraint describes valid configurations through an equation or inequality:

$$f(q)=0$$

or:

$$f(q) \leq 0$$

You don't directly tell the planner what the corresponding joint values are.

For example:

“Keep the sensor normal to the inspection surface.”

Mathematically:

$$f(q) = z_{TCP}(q) \times n_{surface} = 0$$

The planner has to determine:

$$q_1, q_2, \dots, q_n$$

that satisfy that relationship.

In HPP, an implicit constraint is explicitly represented as a differentiable function, comparison type, and RHS.

---

## 8. Example of an HPP implicit constraint

For TCP position:

$$f(q) = p_{TCP}(q) - p_{desired}$$

Requirement:

$$f(q) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$$

The planner doesn't know directly:

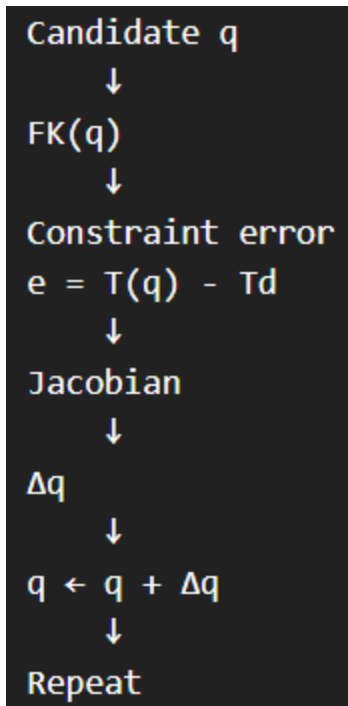
$$q = [??????]$$

It knows only:

$$FK(q) = T_{desired}$$

The HPP numerical solver iteratively adjusts  $q$ .

Conceptually:

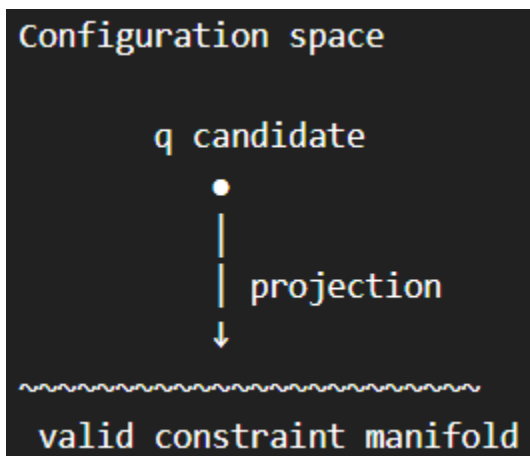


HPP's `HierarchicalIterative` solver solves sets of implicit constraints using a Newton-like iterative procedure.

---

## 9. This is essentially constraint projection

Imagine a random configuration:



The valid configurations form:

$$\mathcal{M} = \{q \mid f(q) = 0\}$$

HPP can project candidate configurations toward this manifold.

That is a fundamental HPP strength.

---

## 10. What is an explicit constraint?

An explicit constraint is different.

If some variables can be **directly computed from other variables**, you don't need to solve them numerically.

Suppose:

$$q_o = g(q_i)$$

where:

- $q_i$  = independent variables
- $q_o$  = dependent variables

Then:

$$\boxed{q_o = g(q_i)}$$

is an explicit relationship.

HPP explicitly defines such constraints and can substitute their output variables with their explicit expressions. Its **BySubstitution** solver performs this substitution and numerically solves only over the remaining variables.

This can reduce the dimension of the numerical problem.

---

# 11. HPP example of explicit constraint

The HPP documentation gives a very good manipulation example.

Imagine:

Robot hand ——— object

The object is rigidly grasped.

Once you know the robot gripper pose:

$$T_{world}^{gripper}(q)$$

and the constant grasp transform:

$$T_{gripper}^{object}$$

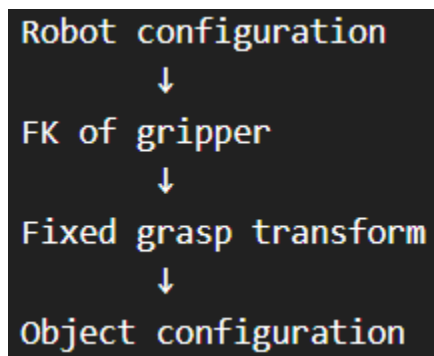
the object pose is directly:

$$T_{world}^{object} = T_{world}^{gripper}(q) T_{gripper}^{object}$$

You don't need to numerically solve the object's 6 DOF independently.

HPP specifically cites **relative pose** as a common explicit constraint: when a free-floating object is held by a gripper, the object's configuration variables can be explicitly expressed from robot variables.

So:





rather than:

```
robot variables + object variables
      ↓
solve all numerically
```

That distinction is important.

---

## 12. Explicit vs equality — they are NOT opposites

A common mistake would be writing:

Explicit constraint = equality

Implicit constraint = inequality

That is **wrong**.

Think of two axes:

| Constraint representation |          |                                                          |
|---------------------------|----------|----------------------------------------------------------|
|                           | Implicit | Explicit                                                 |
| Equality                  | ✓        | ✓                                                        |
| Inequality                | ✓        | generally not the<br>main explicit-<br>substitution case |

For HPP particularly, the `Implicit` class supports equality **and** inequalities.

The explicit classification instead asks:

Can dependent variables be solved directly as functions of other variables?

---

## 13. HPP also supports constraint priorities

Another important strength is hierarchical solving.

Imagine your application has:

### Priority 1 — mandatory

No collision.

### Priority 2

Maintain sensor standoff:

$d=100\text{mm}$

### Priority 3

Maintain orientation:

$$z_{TCP} \parallel n$$

### Priority 4

Improve robot posture.

HPP's **HierarchicalIterative** solver explicitly handles sets of implicit constraints with priority ordering.

Conceptually:

```
Priority 1
TCP / task feasibility
    ↓
Priority 2
secondary constraint
    ↓
Priority 3
posture / redundancy
```

This is a very useful architecture for redundant robots.

---

## 14. What does Tesseract do instead?

Tesseract is more **optimization-centric**.

Rather than making implicit/explicit constraint solving the central abstraction, an optimization planner such as TrajOpt asks:

$$\min_Q J(Q)$$

subject to:

$$g(Q) = 0$$

$$h(Q) \leq 0$$

where:

$$Q = [q_0, q_1, \dots, q_N]$$

is the entire trajectory.

TrajOpt uses sequential convex optimization to optimize joint trajectories while satisfying constraints.

Tesseract itself describes its planning philosophy as solving smoothness, collisions, joint limits and process constraints together.

---

## 15. The key Tesseract idea: constraint vs cost

This is one of Tesseract/TrajOpt's strongest features.

Suppose desired TCP position is:

$$p_d$$

You could treat:

$$p(q) - p_d = 0$$

as a **hard constraint**.

Or you can treat the error as a **cost**:

$$J_p = \|p(q) - p_d\|^2$$

and minimize it.

These have very different meanings.

### Hard constraint

$$\begin{array}{c} \text{Must satisfy} \\ \downarrow \\ p(q) = p_d \end{array}$$

### Cost

Prefer to satisfy  
↓  
 $\min ||p(q) - p_d||^2$

That flexibility is very useful in industrial process optimization.

For example:

**Hard**

$$d_{collision} \geq 20mm$$

**Soft**

$$d_{preferred} = 50mm$$

The optimizer can therefore prioritize feasibility while still trying to improve trajectory quality.

---

## 16. Inspection example — HPP formulation

Suppose your scanner has these requirements:

### Requirement A

TCP should follow swathe position:

$$p_{TCP}(q(s)) = p_{swathe}(s)$$

### Requirement B

Tool Z-axis aligned with surface normal:

$$z_{tool}(q(s)) = n(s)$$

### Requirement C

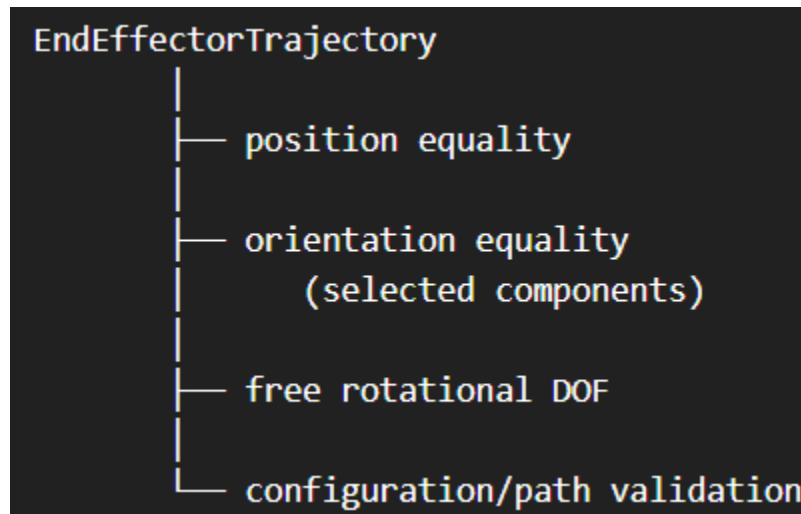
Tool rotation about Z is free.

## Requirement D

Robot remains at least 30 mm from fixture:

$$d(q) \geq 0.03$$

In HPP, this naturally becomes approximately:



With inequalities/other constraints imposed alongside it.

This is a **constraint-driven Cartesian trajectory**.

---

## 17. HPP EndEffectorTrajectory changes the comparison significantly

As you pointed out, your HPP implementation has:

***EndEffectorTrajectory***

for Cartesian planning.

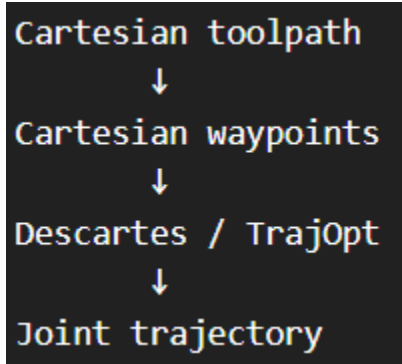
Therefore we should **not** compare:

Tesseract = Cartesian planning  
HPP = only joint-space planning.

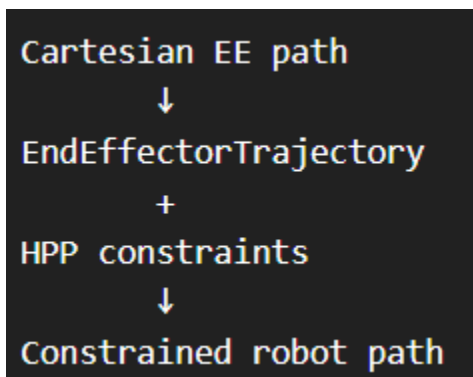
That would be incorrect.

Instead:

### IRT / Tesseract



### LAAS / HPP



The difference is **how the Cartesian path is represented and solved.**

---

## 18. Path types in your LAAS HPP implementation

As you mentioned, `EndEffectorTrajectory` supports several path representations.

## Linear

Between poses:

$$P_0 \rightarrow P_1$$

Position can be represented as:

$$p(s) = p_0 + s(p_1 - p_0)$$

for:

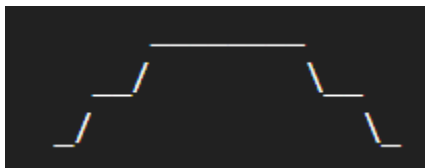
$$s \in [0, 1]$$

Useful for:

- approach,
  - retract,
  - straight inspection strip.
- 

## Curved trajectory

For curved components:



the EE trajectory follows a curved Cartesian path rather than a straight interpolation in configuration space.

This is particularly important for aircraft skin inspection.

---

## Polynomial trajectory

For example:



$$p(t) = a_0 + a_1 t + a_2 t^2 + a_3 t^3$$

cubic, or:

$$p(t) = a_0 + a_1 t + \dots + a_5 t^5$$

quintic.

These allow continuity conditions such as:

$$p(t)$$

$$\dot{p}(t)$$

$$\ddot{p}(t)$$

to be controlled.

---

## Spline trajectory

For multiple Cartesian waypoints:

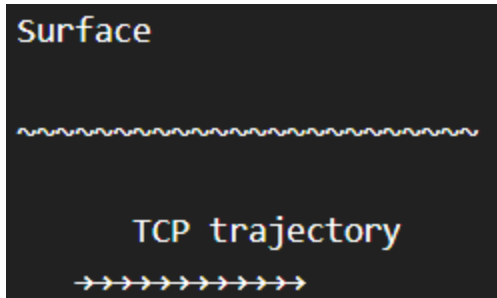
$$P_0, P_1, P_2, \dots, P_N$$

you can construct:

$$S(t)$$

giving a smooth path through/near those points.

So for an inspection swathe:



HPP can represent this as an EE trajectory rather than merely asking IK independently at each sampled TCP pose.

That is an important strength of the LAAS approach.

---

## 19. Tesseract Cartesian path handling is different

Tesseract often works naturally with a series of process waypoints:

$$T_0, T_1, T_2, \dots, T_N$$

Then:

***Descartes***

finds a feasible sequence of robot configurations through Cartesian waypoints.

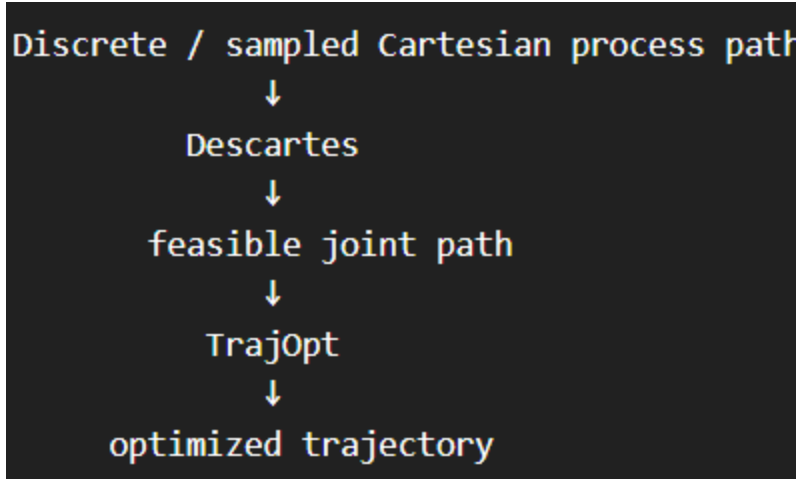
And:

***TrajOpt***

can optimize that trajectory.

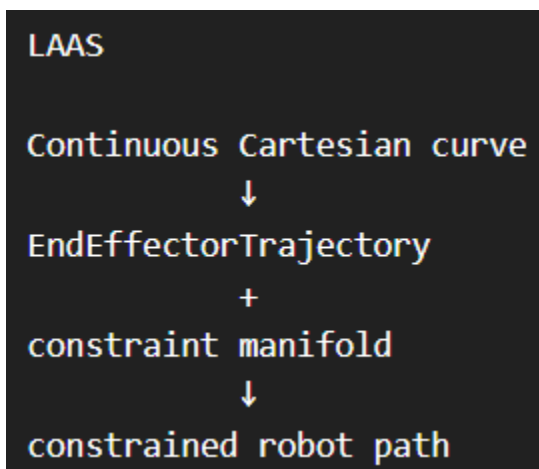
So conceptually:

IRT



Whereas HPP can more naturally express:

LAAS

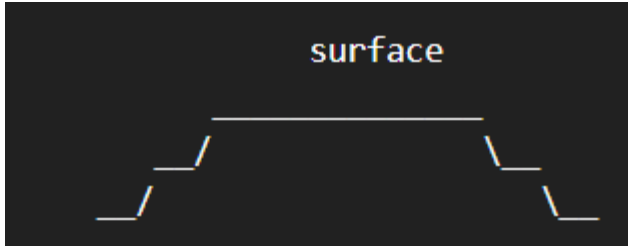


That's an important architectural difference.

---

## 20. Example: curved aircraft panel

Imagine this surface:



The sensor should maintain:

$d=100\text{mm}$

and:

$$Z_{TCP} \parallel n_{surface}$$

## IRT approach

Generate many poses:

$$T_0, T_1, T_2, \dots, T_{500}$$

such that each has the required position and orientation.

Then:

$$Descartes \rightarrow TrajOpt$$

finds and optimizes the corresponding robot joint trajectory.

## HPP approach

Represent the Cartesian curve:

$$T_d(s)$$

and impose:

$$T_{TCP}(q(s)) = T_d(s)$$

through the EE trajectory/constraint machinery.

This is conceptually:

***continuous path constraint***

rather than simply a set of discrete Cartesian targets.

For complex constrained motion, that can be a significant advantage.

---

## 21. Equality tolerance example

Suppose:

$$d_{desired} = 100mm$$

but process tolerance is:

$$\pm 2mm$$

There are two ways to model it.

### Exact equality plus numerical solver tolerance

$$d(q) = 100mm$$

with numerical tolerance:

$$|e| < \epsilon$$

### Physical inequality band

Better representation:

$$98mm \leq d(q) \leq 102mm$$

The second means:

Any solution within this band is physically acceptable.

This gives redundancy to the planner.

That is important because allowing tolerance can enable:

- better joint posture,
- collision avoidance,
- lower joint velocity,
- singularity avoidance.

---

## 22. Tesseract can exploit process tolerances very naturally

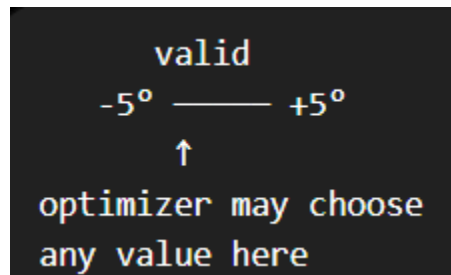
Suppose tool angle is allowed:

$$-5^\circ \leq \theta \leq 5^\circ$$

Instead of requiring:

$$\theta = 0$$

the optimizer has a feasible region:



It can choose the orientation that gives:

$$\min(J_{\text{collision}} + J_{\text{joint-motion}} + J_{\text{smoothness}})$$

This is one reason Tesseract/TrajOpt is attractive for industrial process paths: tolerances can become useful optimization freedom.

---

## 23. HPP can also exploit the same freedom

HPP can represent:

$$\theta \leq 5^\circ$$

and:

$$\theta \geq -5^\circ$$

using its **Inferior** and **Superior** comparison modes.

Or simply leave a Cartesian component unconstrained through a mask.

For example:

| Position    |               |
|-------------|---------------|
| X           | constrained   |
| Y           | constrained   |
| Z           | constrained   |
| Orientation |               |
| Rx          | constrained   |
| Ry          | constrained   |
| Rz          | unconstrained |

This gives the HPP solver an entire family of solutions.

---

## 24. A very useful inspection example

Let's formulate one complete problem.

Requirement:

Scanner follows aircraft skin at  $100 \pm 2$  mm, normal deviation less than  $5^\circ$ , while avoiding fixtures by at least 50 mm.

Mathematically:

**Standoff**

$$0.098 \leq d(q) \leq 0.102$$

Inequality

**Normal orientation**

$$|\theta(q)| \leq 5^\circ$$

Inequality

**Follow swathe longitudinal position**

$$p_{along}(q) = p_{swathe}(s)$$

Equality

**Fixture clearance**

$$d_{fixture}(q) \geq 0.05$$

Inequality

**Redundant rotation**

$$\psi = \text{free}$$

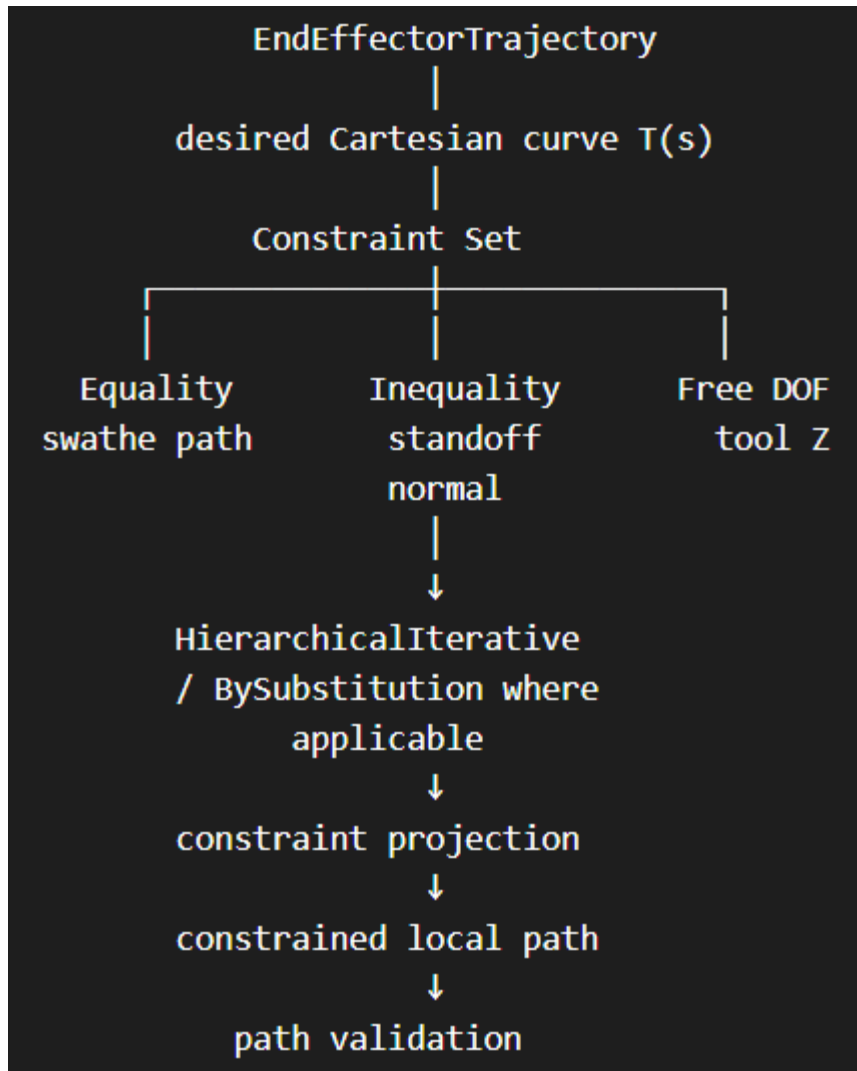
unconstrained DOF

Now compare.

---

## 25. How HPP solves that conceptually



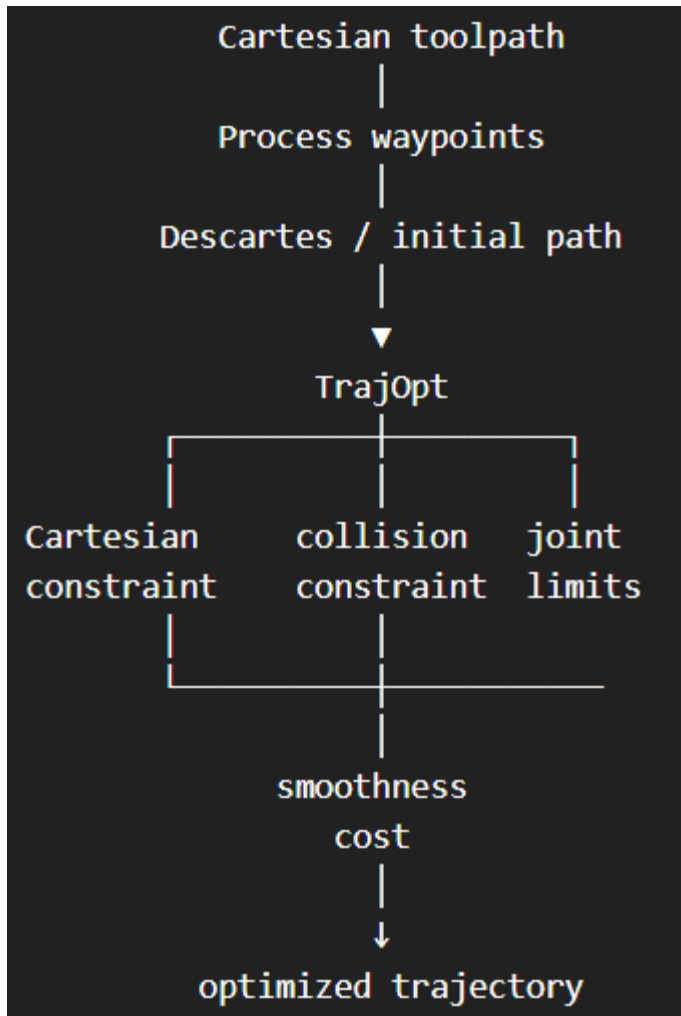


That is a very coherent constrained-planning architecture.

HPP's steering methods are explicitly defined as building paths while taking kinematic constraints into account.

---

## 26. How IRT/Tesseract solves it conceptually



Tesseract's own documentation characterizes this as solving collisions, smoothness, joint limits and process constraints jointly through its optimization-centric architecture.

---

## 27. Where Tesseract is better

For:

$$\min_Q J(Q)$$

where trajectory quality is extremely important.

For example:

$$J = w_1 J_{smoothness} + w_2 J_{collision} + w_3 J_{joint-motion} + w_4 J_{TCP-error}$$

subject to hard process constraints.

Tesseract/TrajOpt has an advantage because the optimization formulation is central to the planning framework.

Examples:

- minimize joint motion,
  - maximize clearance indirectly through costs,
  - reduce acceleration,
  - smooth Cartesian process paths,
  - exploit allowed TCP tolerance.
- 

## 28. Where HPP is better

When the primary problem is:

**The robot must remain on a complicated constraint manifold.**

Examples:

**Robot + rail**

$$q = [q_{rail}, q_1, \dots, q_6]$$

**Specific process constraint**

$$T_{TCP}(q) = T_{surface}(s)$$

**Coupled joints**

$$q_3 = f(q_2)$$

**Locked mechanism**

$$q_{rail} = q_{rail,current}$$

during a particular phase.

### Relative-pose relationship

$$T_A^{-1}T_B = T_{desired}$$

HPP's explicit/implicit constraint framework, hierarchical solver and substitution mechanism provide a strong mathematical foundation for this.

## 29. One important HPP advantage: reducing the numerical problem

Suppose your configuration has:

$$q = [q_{robot}, q_{object}]$$

with:

$$6+6=12$$

variables.

But:

$$q_{object} = g(q_{robot})$$

because the object is rigidly attached.

A pure numerical formulation might optimize 12 variables with six equations.

HPP explicit substitution can instead reduce the problem approximately to:

$q_{robot}$

with object configuration calculated explicitly.

So:

*12 variables → 6 independent variables*

conceptually.

That can improve:

- numerical stability,
- convergence,
- computational efficiency.

This explicit variable-elimination concept does not appear as an equivalent central abstraction in Tesseract's planning architecture.

---

## 30. Updated constraint comparison score

For your study I'd use something like:

| Constraint capability              | IRT /<br>Tesseract | HPP / LAAS |
|------------------------------------|--------------------|------------|
| Cartesian equality<br>constraint   | 5/5                | 5/5        |
| Cartesian inequality<br>constraint | 5/5                | 5/5        |
| Joint constraints                  | 5/5                | 5/5        |
| Collision constraints              | 5/5                | 4.5/5      |

|                                 |            |                                                           |
|---------------------------------|------------|-----------------------------------------------------------|
| Implicit nonlinear constraints  | 4/5        | <b>5/5</b>                                                |
| Explicit variable substitution  | 3/5        | <b>5/5</b>                                                |
| Constraint projection           | 3.5–4/5    | <b>5/5</b>                                                |
| Constraint manifold planning    | 4/5        | <b>5/5</b>                                                |
| Hierarchical constraint solving | 4/5        | <b>5/5</b>                                                |
| Partial Cartesian constraints   | <b>5/5</b> | <b>5/5</b>                                                |
| Exploiting process tolerance    | <b>5/5</b> | 5/5                                                       |
| Cost vs constraint flexibility  | <b>5/5</b> | 4/5                                                       |
| Whole-trajectory optimization   | <b>5/5</b> | 4/5                                                       |
| Continuous Cartesian EE curve   | 4.5/5      | <b>5/5 with your EndEffectorTrajectory implementation</b> |
| Complex/custom mechanisms       | 4/5        | <b>5/5</b>                                                |

---

## 31. Revised Cartesian planning comparison

Considering the **EndEffectorTrajectory** information you provided, I would now revise our earlier comparison.

### IRT

*Toolpath → Cartesian waypoints → Descartes → TrajOpt*

Strength:

Discrete/process waypoint planning + global trajectory optimization.

### LAAS

*Cartesian curve → EndEffectorTrajectory → HPP constraints → Constrained planning*

with:

*Linear*  
*Curved*  
*Polynomial*  
*Spline*

trajectory representations in your LAAS implementation.

Strength:

## **Continuous Cartesian trajectory representation + constraint-manifold planning.**

This means I would **not award Cartesian planning itself to Tesseract anymore**. Both are strong, but in quite different ways.

---

## **Overall conclusion for the constraints component**

For your final IRT JV vs LAAS comparison, I would state:

**LAAS/HPP provides the stronger general-purpose constraint architecture. HPP formally distinguishes implicit and explicit numerical constraints, supports equality and inequality comparison types, constraint masking, hierarchical numerical solving and explicit-variable substitution. Combined with the EndEffectorTrajectory Cartesian planner, this enables linear, curved and spline/polynomial Cartesian paths to be planned while maintaining complex kinematic constraints.**

**IRT JV's Tesseract approach is stronger when the same requirements need to be considered within whole-trajectory optimization. TrajOpt allows Cartesian requirements, collision avoidance, joint limits, smoothness and process objectives to participate in a common optimization problem, making it especially attractive where trajectory quality and process optimization are dominant objectives.**

So for this component I would summarize the distinction as:

HPP: constraint satisfaction / constraint-manifold centric

versus

Tesseract: trajectory optimization / cost-and-constraint centric

That is probably one of the **most fundamental architectural differences** between the LAAS and IRT JV approaches.



may be one more comparison about different planner or steering method option available in between IRT planner and HPP?

Yes — that changes the comparison significantly. We should **not score IRT higher simply because OMPL exposes more planners** if the LAAS/HPP stack you are evaluating already provides **RRT, RRT\*, Bi-RRT\*, PRM, Potential Field, A\*, Dijkstra, etc.**

The better comparison is by **planner family and how each family is integrated into the overall planning architecture.**

## Revised planner comparison

| Planner / capability          | IRT –<br>MoveIt2/Tesseract                             | LAAS – HPP stack                  | Comment                                                 |
|-------------------------------|--------------------------------------------------------|-----------------------------------|---------------------------------------------------------|
| RRT                           | ✓ via OMPL                                             | ✓                                 | Comparable                                              |
| RRT*                          | ✓ via OMPL                                             | ✓                                 | Comparable                                              |
| Bi-RRT /<br>bidirectional RRT | ✓ RRTConnect/OMPL<br>variants                          | ✓                                 | Comparable                                              |
| Bi-RRT*                       | Depends on configured<br>OMPL/plugin<br>implementation | ✓ in your LAAS stack              | <b>LAAS<br/>advantage if<br/>directly<br/>available</b> |
| PRM                           | ✓ OMPL                                                 | ✓                                 | Comparable                                              |
| PRM*                          | ✓ OMPL                                                 | Depends on LAAS<br>implementation | IRT if required                                         |

|                                     |                                                                                        |                                              |                         |
|-------------------------------------|----------------------------------------------------------------------------------------|----------------------------------------------|-------------------------|
| <b>A*</b>                           | Can be used in graph/planning layers, but not the primary Tesseract free-space planner | ✓                                            | <b>LAAS stack</b>       |
| <b>Dijkstra</b>                     | Graph search exists in relevant components; not the primary free-space interface       | ✓                                            | <b>LAAS stack</b>       |
| <b>Potential Field</b>              | Not a principal Tesseract global planner                                               | ✓                                            | <b>LAAS stack</b>       |
| <b>Cartesian graph search</b>       | <b>Descartes</b>                                                                       | Different approach via EndEffectorTrajectory | Different               |
| <b>Trajectory optimization</b>      | <b>TrajOpt / TrajOpt-IFOPT</b>                                                         | HPP path optimizers                          | <b>IRT</b>              |
| <b>Linear/interpolated steering</b> | Planner/interpolation dependent                                                        | ✓                                            | HPP architecture        |
| <b>Hermite steering</b>             | Not a central abstraction                                                              | ✓                                            | <b>HPP</b>              |
| <b>Kinodynamic steering</b>         | Available depending on planning backend                                                | ✓                                            | <b>HPP architecture</b> |
| <b>Cartesian EE steering/path</b>   | Descartes/process planner                                                              | <b>EndEffectorTrajectory</b>                 | Different strengths     |

|                                                      |                                                         |                              |                  |
|------------------------------------------------------|---------------------------------------------------------|------------------------------|------------------|
| <b>Constraint-aware steering</b>                     | Planner/optimization dependent                          | <b>Deep integration</b>      | <b>HPP</b>       |
| <b>Constraint projection during planning</b>         | Not central architecture                                | <b>Core HPP capability</b>   | <b>HPP</b>       |
| <b>Planner + steering independently configurable</b> | Less central                                            | <b>Yes</b>                   | <b>HPP</b>       |
| <b>Planner chaining</b>                              | <b>Very strong: OMPL → TrajOpt, Descartes → TrajOpt</b> | Planner → optimizer possible | <b>Tesseract</b> |

So I would **remove my earlier “IRT wins for sampling planner variety” conclusion**. Based on the LAAS implementation you described, that's not justified.

## More importantly, HPP actually has several different levels

The architecture is more accurately:

Global/Graph Planner+Steering Method+Constraints+Path Validation+Path Optimization

For example, you could conceptually configure:

RRT\*+HermiteSteering+CartesianConstraint+CollisionValidation

or:

BiRRT\*+EndEffectorTrajectory+TCPCConstraints+PathValidation

This composability is one of HPP's major strengths.

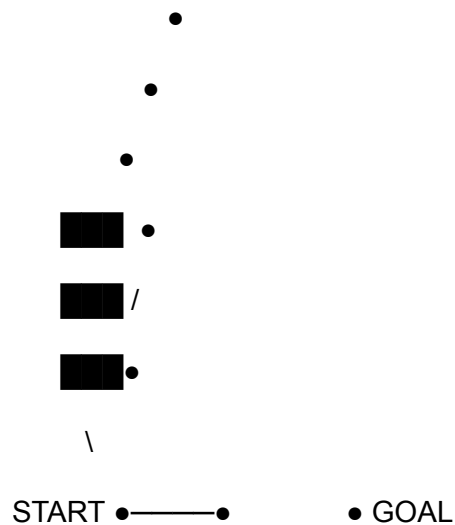
---

# 1. RRT

Both stacks support the basic concept:

$q_{start} \rightarrow q_{goal}$

through random exploration of configuration space.



RRT is particularly useful in high-dimensional configuration spaces.

## Verdict

Comparable.

---

# 2. RRT\*

RRT\* adds asymptotic optimization through rewiring.

Instead of simply:

$q_{new} \rightarrow q_{nearest}$

it considers whether another parent produces lower cost and rewires nearby nodes.

Therefore:

$Cost(path) \rightarrow Cost^*$

as sampling increases, under the usual theoretical assumptions.

Both stacks can provide this capability in the environments you described.

## Verdict

**Comparable at algorithm level.**

The more interesting comparison is how constraints and steering are integrated.

---

## 3. Bi-RRT\*

This is worth highlighting for HPP/LAAS if that's the implementation being used.

Instead of growing only:

Tstart

you grow:

Tstart

and:

Tgoal

toward one another.

Conceptually:

START ●—●—●

\

•

\

X

/

•

/

GOAL ●—●—●

For industrial problems where:

qstart

and

qgoal

are known, bidirectional search can significantly accelerate finding connectivity.

If the LAAS version also applies RRT\*-style optimization/rewiring, that gives you the combination of:

**bidirectional exploration + path-quality improvement.**

This is worth calling out as a LAAS planner option rather than lumping everything under “RRT.”

---

## 4. PRM

Both approaches also support roadmap-based planning.

Instead of solving one start-goal query, construct:

$G=(V,E)$

representing free configuration space:

●—●—●

| \ | / |

| ●—●—● |

| / \ |

●—●—●

Then:

qs→roadmap→qg

This can be particularly attractive when the robot repeatedly plans in the **same environment**.

And that's actually relevant to your defect inspection problem.

If you have:

$D_1, D_2, \dots, D_{100}$

and repeatedly plan:

$D_i \rightarrow D_j$

a reusable roadmap can potentially be valuable.

So PRM deserves more attention in your 100+ defect sequencing architecture than it would for a single motion.

---

## 5. A\* and Dijkstra

This is another area where your LAAS stack has useful options.

Once you have a graph:

$G=(V,E)$

Dijkstra minimizes accumulated edge cost:

$g(v) = \min_{e \in \text{path}} \sum c(e)$

A\* additionally uses a heuristic:

$f(n) = g(n) + h(n)$

where:

- $g(n)$  = cost already accumulated
- $h(n)$  = estimated remaining cost.

If:

$h(n)$

is informative and admissible, A\* can search much less of the graph than Dijkstra while retaining optimality under the standard conditions.

## Why this matters for your architecture

You have several graph problems:

robot roadmap defect connectivity TSP/GTSP candidate graph

So A\*/Dijkstra can have utility at levels beyond ordinary robot motion planning.

That makes the LAAS architecture more **graph-planning rich** than our previous comparison suggested.

---

## 6. Potential Field

This gives LAAS another planning paradigm.

Define an attractive potential toward the target:

$U_{att}(q)$

and repulsive potential around obstacles:

$U_{rep}(q)$

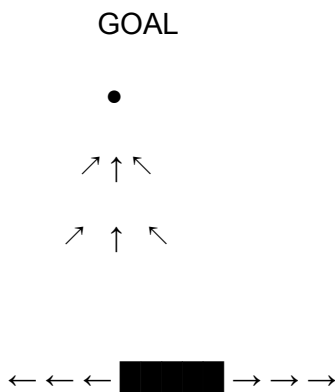
Then:

$U(q) = U_{att}(q) + U_{rep}(q)$

and motion follows:

$F(q) = -\nabla U(q)$

Conceptually:







START



Its classical weakness is:

local minima

so I would not present Potential Field as “better than RRT.”

Instead, its value is:

- fast reactive/local guidance,
- obstacle repulsion,
- useful hybrid planner component.

---

## 7. Therefore the planner-family comparison is actually very balanced

A more accurate categorization is:

| Planner family | IRT  | LAAS              |
|----------------|------|-------------------|
| Sampling-based | OMPL | HPP/LAAS planners |
| RRT            | ✓    | ✓                 |
| RRT*           | ✓    | ✓                 |

|                                     |                                  |                          |
|-------------------------------------|----------------------------------|--------------------------|
| Bidirectional RRT                   | ✓                                | ✓                        |
| Bi-RRT*                             | implementation-dependent         | ✓                        |
| PRM                                 | ✓                                | ✓                        |
| <b>Graph search</b>                 | Descartes/other graph mechanisms | A*, Dijkstra             |
| <b>Potential-based</b>              | Not central                      | Potential Field          |
| <b>Cartesian process planning</b>   | Descartes                        | EndEffectorTrajectory    |
| <b>Trajectory optimization</b>      | TrajOpt                          | PathOptimizer            |
| <b>Constraint-manifold planning</b> | Less central                     | <b>HPP core strength</b> |

This is much more representative.

---

## 8. But the biggest HPP differentiator isn't actually RRT vs RRT\*

This is what I would emphasize in your study.

The interesting thing isn't:

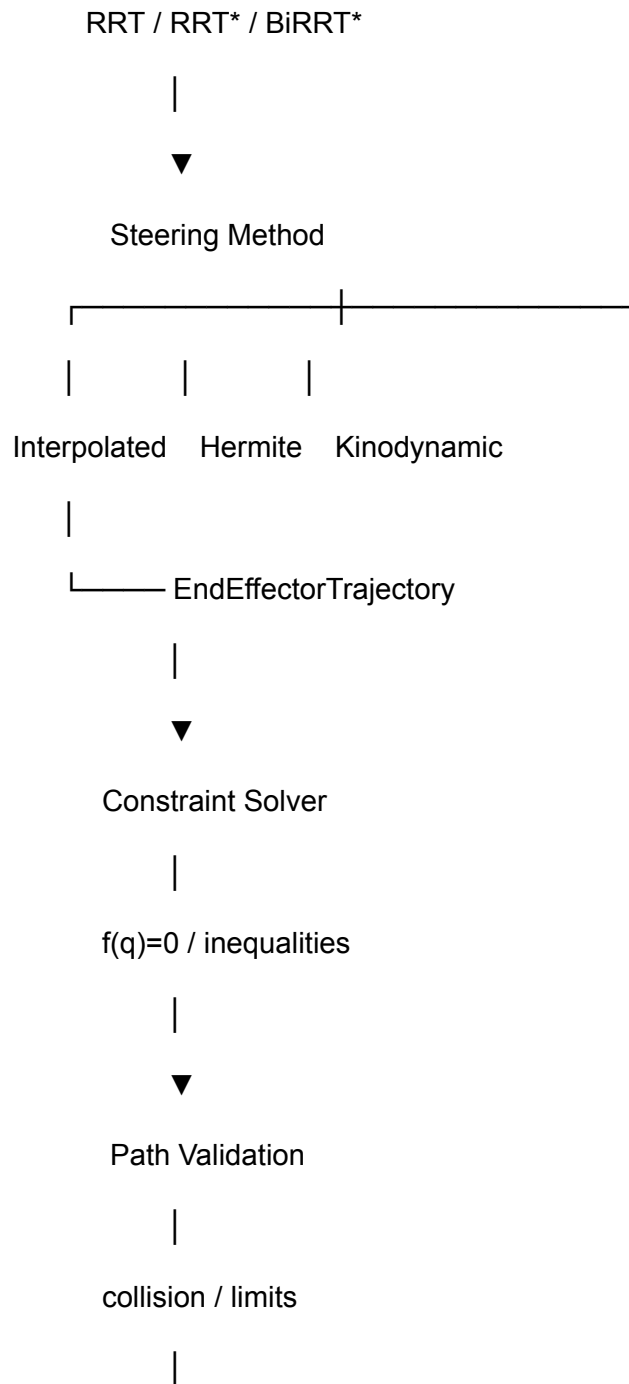
“HPP has RRT\* and IRT also has RRT\*.”

That's mostly feature parity.

The interesting thing is:

Planner+SteeringMethod+ConstraintSolver+PathValidation

For example:





Valid HPP path

That composability is more important than the raw number of algorithms.

---

## 9. Tesseract's corresponding differentiator

Tesseract's strength is different:

Planner1→Planner2→Optimizer

For example:

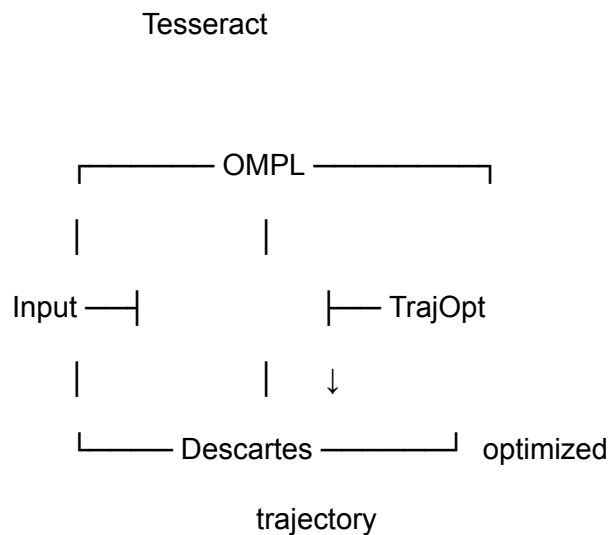
### Free space

OMPL→TrajOpt

### Process path

Descartes→TrajOpt

So:



Therefore the fundamental distinction becomes:

### HPP

## Compose behavior inside the planner

Planner+Steering+Constraint

## Tesseract

## Compose different planners/optimizers into a pipeline

Planner→Planner/Optimizer

That's a much stronger architectural comparison.

---

# 10. Revised scoring

Given the additional LAAS capabilities you've described, I'd revise the scoring to:

| Capability                | IRT   | LAAS |
|---------------------------|-------|------|
| Sampling-based algorithms | 5/5   | 5/5  |
| RRT/RRT*                  | 5/5   | 5/5  |
| Bidirectional planning    | 5/5   | 5/5  |
| PRM                       | 5/5   | 5/5  |
| Graph search              | 4.5/5 | 5/5  |
| A*/Dijkstra options       | 4/5   | 5/5  |

|                               |            |            |
|-------------------------------|------------|------------|
| Potential-field option        | 3/5        | <b>5/5</b> |
| Configurable steering         | 3.5/5      | <b>5/5</b> |
| Hermite steering              | 3/5        | <b>5/5</b> |
| Kinodynamic steering          | 4/5        | <b>5/5</b> |
| Constraint-aware planning     | 4/5        | <b>5/5</b> |
| Constraint projection         | 3.5/5      | <b>5/5</b> |
| Cartesian planning            | <b>5/5</b> | <b>5/5</b> |
| Dense discrete Cartesian path | <b>5/5</b> | 4.5/5      |
| Continuous EE trajectory      | 4.5/5      | <b>5/5</b> |
| Trajectory optimization       | <b>5/5</b> | 4–4.5/5    |
| Planner chaining              | <b>5/5</b> | 4.5/5      |
| Custom planning research      | 4.5/5      | <b>5/5</b> |

So with the additional information you've given me, I would now say **LAAS/HPP actually has the richer set of planning primitives and customization possibilities**, while **IRT/Tesseract has the stronger packaged multi-stage industrial planning/optimization pipeline**.

That distinction fits much better with the rest of your IRT–LAAS comparison.

Does IRT trying to provide the complete stack -->?

like perception (but not fully, only part identification) + path planning + execution (with ros2 as the software middle ware )

previously we had REACT stack --> in which we use Visp (by inria) for localization and referencing and feature detection etc (whole perception) + path planner (with HPP) and control (stack of tasks for task optimization )

agimus or dynamic behaviour tree for the over all orchestrator

also HPP is introducing something called path memory optimization --> meaning in case of birrt\* path generated may be different at different instance , this feature saves the path and do some analysis and gives the best optimized if no change in env

Can you do comparison on above points?

Yes. With those additional points, the comparison becomes broader than **HPP vs Tesseract**. You are really comparing **two robotics software-stack philosophies**:

- **IRT JV**: an industrialized, ROS 2-centered stack integrating perception/part identification → tool-path generation → motion planning/optimization → robot execution.
- **LAAS / previous REACT approach**: a more modular robotics stack where perception, planning, control, execution and orchestration are separate interchangeable components.

That distinction is important. Publicly, IRT describes its robotics expertise as covering **perception, automatic programming, process-based control and industrial integration**, and specifically mentions “3D perception of complex surfaces and definition of robotic trajectories.”

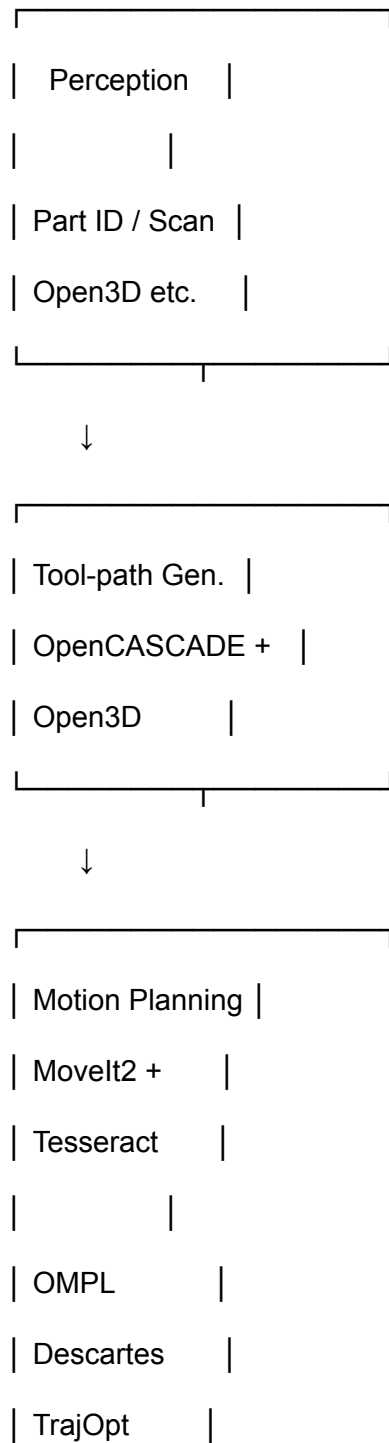
One caveat: I could not find public documentation for the specific **HPP “path memory optimization”** feature you described. So I would treat that as a **new LAAS/custom HPP capability communicated to your team**, rather than claiming it is currently a standard documented HPP feature.

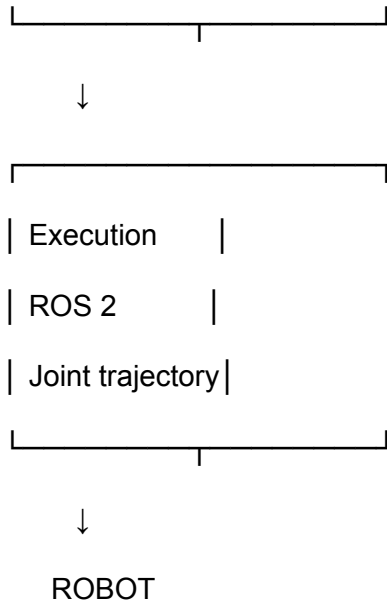
## 1. Overall stack comparison



I would draw the two architectures like this:

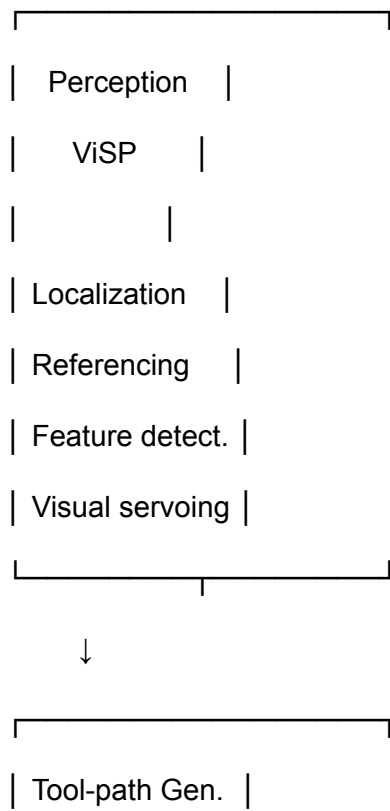
### IRT JV APPROACH

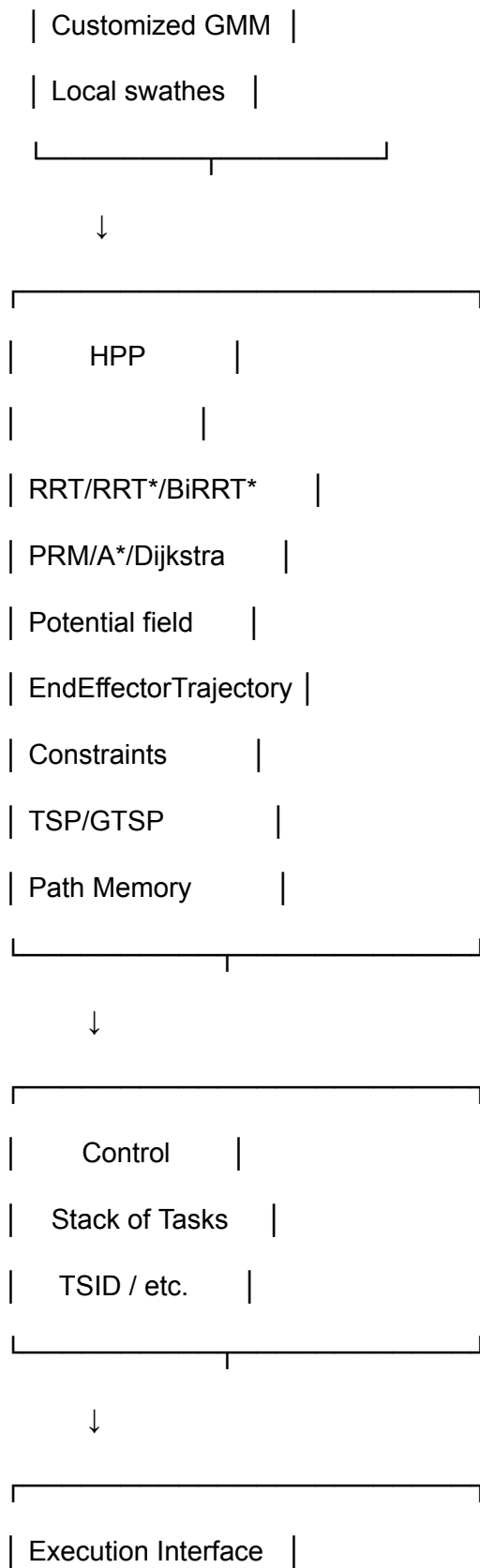


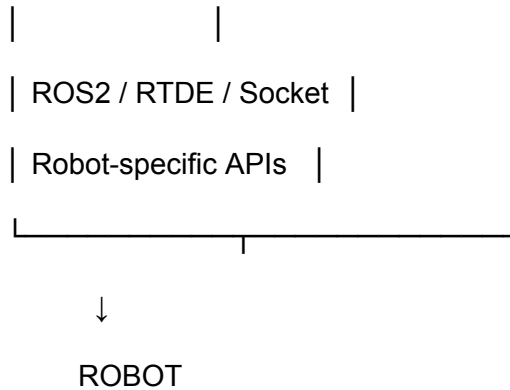


versus:

#### LAAS / REACT APPROACH







AGIMUS / Behaviour Tree

orchestrates the workflow

LAAS publicly describes its broader robotics activity as spanning **perception, decision, movement generation and communication**, and its historical architecture uses supervisors over distributed functional modules.

---

## 2. Is IRT trying to provide a “complete stack”?

Yes, based on what you have learned from IRT, I think that is a reasonable interpretation — with an important qualification.

I would call it:

**an integrated industrial robot-programming stack**

rather than:

**a complete autonomous robotics stack.**

Why?

Because the current IRT architecture appears strong from:

Part information→Toolpath→Robot trajectory→Execution

but its perception layer, according to your discussions, does not currently provide everything that the previous REACT perception stack provided.

IRT itself says it develops/integrates technological bricks and delivers industrial demonstrators rather than claiming one monolithic framework.

---

### 3. Perception comparison

This is therefore an important difference.

#### IRT

From your discussion:

Perception ≈ part identification / geometric acquisition

with Open3D being used for point-cloud/3D processing.

Then Open CASCADE provides the CAD/geometric side.

This supports:

Part → Identify/Register → Surface → Toolpath

#### REACT / LAAS

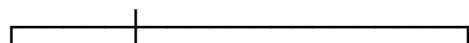
ViSP provides a much broader robotics vision capability.

Conceptually:

Camera / Sensor



ViSP



Feature Localization   Referencing

detect.



Pose estimation



Visual servoing /

feedback

So the perception stack can participate not only at initialization but during robot behavior.

## Assessment

| Perception capability    | IRT current                    | REACT / LAAS                    |
|--------------------------|--------------------------------|---------------------------------|
| Part identification      | <b>Strong</b>                  | Strong                          |
| Point-cloud processing   | <b>Strong</b>                  | Depends on integrated component |
| CAD geometry integration | <b>Very strong</b>             | Requires integration            |
| Localization             | Partial/current implementation | <b>Strong</b>                   |
| Referencing              | Partial                        | <b>Strong</b>                   |
| Feature tracking         | Limited/current scope          | <b>Strong with ViSP</b>         |
| Pose estimation          | Application dependent          | <b>Strong</b>                   |

|                                 |                           |                             |
|---------------------------------|---------------------------|-----------------------------|
| Visual servoing                 | Not central current stack | <b>Core ViSP capability</b> |
| Closed perception-control loop  | Limited                   | <b>Strong</b>               |
| Overall perception completeness | 3.5/5                     | <b>5/5</b>                  |

So I'd give **REACT/LAAS** the **advantage for complete perception**, while IRT's combination is strong specifically for **3D geometry** → **industrial toolpath**.

---

## 4. Planning

We've already established that this is more balanced.

### IRT

OMPL

+

Descartes

+

TrajOpt

Excellent industrial pipeline.

### LAAS

RRT / RRT\* / BiRRT\*

PRM

A\* / Dijkstra

Potential Field

+

Steering Methods

+

Constraints

+

EndEffectorTrajectory

+

TSP/GTSP

The LAAS architecture has greater customization and mission-level capability in the implementation you're evaluating.

Therefore:

| Planning dimension                    | Advantage |
|---------------------------------------|-----------|
| Standard free-space planning          | ≈         |
| Industrial trajectory optimization    | IRT       |
| Continuous constrained Cartesian path | LAAS      |
| Complex constraints                   | LAAS      |
| Planner/steering customization        | LAAS      |



100+ defect sequencing      **LAAS**

Local defect scan      **LAAS**

Packaged industrial planning pipeline      **IRT**

---

## 5. Control is where REACT has another major architectural layer

This should be separated from trajectory execution.

A planner generates:

$q_d(t)$

or:

$T_d(t)$

but control asks:

**How do I continuously drive the robot toward the desired task while respecting priorities and feedback?**

With Stack of Tasks you can formulate something conceptually like:

Task 1: maintain process TCP Task 2: avoid joint limits Task 3: preferred posture

using task priorities.

Conceptually:

HPP

|

| Desired motion

↓

Stack of Tasks

|

|— TCP task

|— orientation task

|— joint-limit task

|— posture task

|— sensor-feedback task

|

↓

Robot command

This gives REACT a genuine:

Planning → Control

architecture.

---

## 6. IRT currently appears more trajectory-execution centric

Based on what IRT told you:

Tesseract

↓

Time-parameterized

joint trajectory

↓

ROS2



Robot controller

This is a perfectly valid industrial architecture.

But it is different from:

HPP desired trajectory



Task-level controller



Sensor feedback



Dynamic task optimization



Robot

So I'd characterize them as:

## **IRT**

Plan → Optimize → Execute

## **REACT**

Plan → Control → Execute

with the possibility of perception feedback into control.

That's an important difference.

---

# 7. Orchestration — AGIMUS / Behavior Tree

This is another area that should appear in the comparison.

The orchestrator is not the planner.

It answers:

What should the robot do next?

For example:

Initialize



Localize part



Generate inspection path



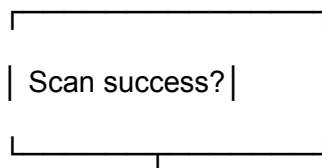
Plan approach



Execute approach



Start scan



Yes | No



| Replan

| ↓

| Retry

↓

Move to next defect

This is exactly where a Behavior Tree / AGIMUS-style supervisor becomes valuable.

AGIMUS's stated objective includes generating methods for **coordinating, scheduling and controlling manipulators**, especially for less repetitive and more modular industrial tasks.

---

## 8. Why orchestration matters for your 100-defect example

Suppose TSP gives:

D37→D82→D11→...

The orchestrator then handles:

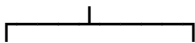
Get next defect

↓

Plan approach

↓

Plan successful?



Yes      No

↓

↓

Execute    Try alternative

↓

configuration

Scan



Scan successful?



Update mission



Next defect

TSP doesn't do this.

HPP doesn't do this.

The Behavior Tree/supervisor does.

That's why the REACT architecture is closer to a **robot autonomy architecture**, rather than simply a planning stack.

---

## 9. What is the corresponding IRT component?

This is where I would ask IRT a specific question:

**“What is the supervisory/orchestration layer above Tesseract/MoveIt2 that manages perception → planning → execution → recovery/replanning?”**

Tesseract itself has **Task Composer**, which can create task graphs/pipelines.

So IRT could potentially have:

Task Composer

Perception



Toolpath



Planning



TrajOpt



Validation



Execution

But Task Composer should not automatically be considered equivalent to a **dynamic Behavior Tree/autonomy supervisor**.

You need to establish whether their implementation supports:

- conditional branching,
- execution feedback,
- failure recovery,
- re-localization,
- replanning,
- retry,
- task cancellation,
- dynamic target selection.

That would be a very useful question for your next IRT discussion.

---

## 10. Path Memory Optimization

The HPP feature you mentioned is particularly interesting.

Let's say you use Bi-RRT\*.

Run 1 produces:

P1

Run 2:

P2

Run 3:

P3

because sampling-based planners are stochastic.

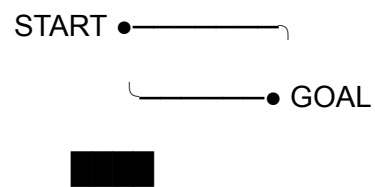
Even with identical:

qstart,qgoal,Environment

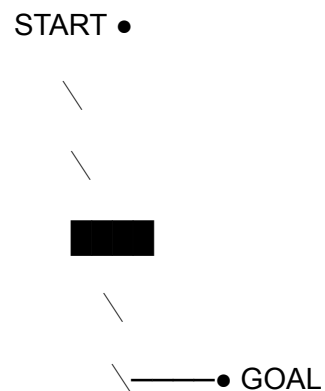
you may obtain different solutions.

For example:

RUN 1

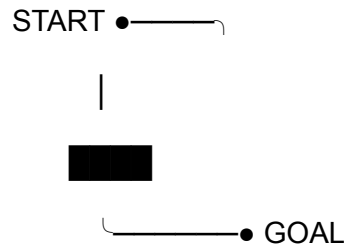


RUN 2





RUN 3



All three may be valid.

But they can differ in:

PathLength ExecutionTime JointMotion Clearance Manipulability

etc.

---

## 11. Traditional behavior

Normally you might simply take:

$P_{current}$

optimize it and execute it.

The next planning call starts again.

That means previous planning computation is largely discarded.

---

## 12. Path-memory concept

Based on how LAAS explained the feature to you, it becomes:

Bi-RRT\* run



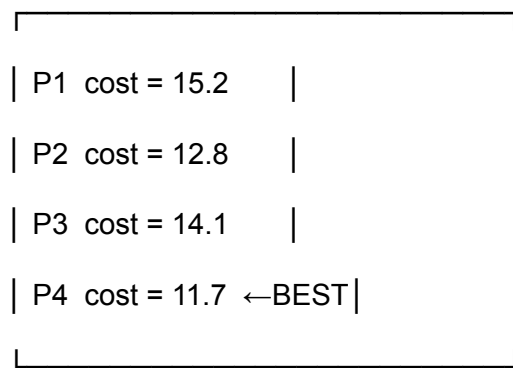
Candidate path



Path quality analysis



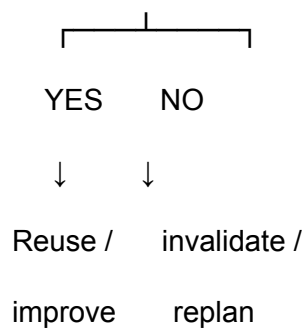
Path Memory



Check environment



Environment unchanged?



Best-known path

This introduces something very useful:

Planning experience

---

## 13. Cost can be multi-objective

You don't necessarily want the shortest geometric path.

Define:

$$J(P) = w_{LL}(P) + w_{TT}(P) + w_{QQ}(P) + w_{CC}(P) + w_{SS}(P)$$

where, for example:

- L = path length
- T = execution time
- Q = total joint displacement
- C = clearance penalty
- S = singularity/manipulability penalty.

Then:

$$P^* = \arg \min_{P_i \in \text{Memory}} J(P_i)$$

This could give a much better robot path than simply accepting the latest Bi-RRT\* result.

---

## 14. Why this is very relevant industrially

Industrial cells often have a mostly **static environment**.

For example:

Robot

+

Rail

+

Aircraft panel

+

Fixtures

may remain unchanged for thousands of cycles.

Therefore repeatedly calculating:

$qA \rightarrow qB$

from scratch wastes useful information.

Instead:

Environment unchanged+Same/similar query

could allow:

reuse best-known validated path

This could improve:

- repeatability,
- planning latency,
- path quality,
- predictability.

---

## 15. This also addresses a weakness of sampling planners

Sampling planners inherently have stochastic variability.

Without memory:

Run1=Run2=Run3

With path memory:

Run1,Run2,...,RunN

can contribute toward:

Pbest

So instead of each run being independent:

$P_i = f(\text{random samples})$

you gradually accumulate useful planning experience:

$P_{\text{bestk}+1} = \min(P_{\text{bestk}}, P_{\text{new}})$

according to the selected quality metric.

This is a very interesting concept for industrializing RRT-family planning.

---

## 16. But environment change detection is critical

You correctly said:

**if there is no change in environment.**

That's essential.

A cached path:

$P_{\text{cached}}$

cannot simply be executed blindly.

You need:

$\text{Environment}_{\text{current}} = ? \text{Environment}_{\text{cached}}$

If unchanged:

$P_{\text{cached}} \rightarrow \text{Validation} \rightarrow \text{Execution}$

If changed:

$P_{\text{cached}} \rightarrow \text{INVALID}$

and:

Replan

A robust architecture would therefore be:

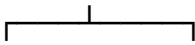
Planning request



Query Path Memory



Candidate exists?



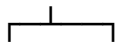
YES NO



Check env Plan



unchanged?



Yes No



Validate Replan



Compare cached vs new



Best path



Execute

I would **always retain path validation before execution**, even when the environment signature appears unchanged.

---

## 17. Does IRT/Tesseract have the same capability?

I would currently mark this as:

**Not demonstrated in the IRT implementation you have evaluated.**

Rather than:

“Tesseract cannot do it.”

Tesseract has environment representation, planning, collision checking and trajectory optimization, and its core is actually ROS-agnostic despite IRT's current ROS 2 execution choice.

But I haven't found evidence that the standard Tesseract planning workflow provides the exact **cross-run path-memory analysis and best-path reuse mechanism** LAAS described to you.

So this becomes a potentially meaningful **LAAS differentiator**.

---

## 18. Path Memory vs TrajOpt

Interestingly, these solve different problems.

### TrajOpt

Given:

P<sub>initial</sub>

find:

P<sub>optimized</sub>

during the current planning request.

### Path Memory

Given historical paths:

P1,P2,...,PN

retain/analyze:

Pbest

across planning instances.

So:

TrajOpt=intra-runoptimization

whereas:

PathMemory=inter-runoptimization/experiencereuse

They are complementary.

An even stronger architecture could theoretically be:

BiRRT→PathMemory→BestSeed→TrajOpt

---

## 19. Complete stack-level comparison

I would now add this table to your study.

| Stack component     | IRT JV current stack         | LAAS / REACT                   | Advantage |
|---------------------|------------------------------|--------------------------------|-----------|
| Part identification | Strong                       | Strong                         | ≈         |
| Complete perception | Partial                      | ViSP-based complete perception | LAAS      |
| Localization        | Partial/application specific | ViSP                           | LAAS      |



|                                   |                              |                                                 |                     |
|-----------------------------------|------------------------------|-------------------------------------------------|---------------------|
| <b>Referencing</b>                | Partial/application specific | <b>ViSP</b>                                     | LAAS                |
| <b>Feature detection/tracking</b> | Limited current scope        | <b>ViSP / perception modules</b>                | LAAS                |
| <b>3D point-cloud processing</b>  | <b>Open3D</b>                | Custom/perception stack                         | IRT                 |
| <b>CAD processing</b>             | <b>Open CASCADE</b>          | Less central                                    | IRT                 |
| <b>Toolpath generation</b>        | <b>OCC + Open3D</b>          | GMM                                             | Depends             |
| <b>Robot path planning</b>        | <b>Movel2 + Tesseract</b>    | <b>HPP</b>                                      | Both strong         |
| <b>Planner diversity</b>          | <b>OMPL ecosystem</b>        | <i>RRT/RRT*/BiRRT/PRM/A*/Dijkstra/PF etc.**</i> | ≈                   |
| <b>Cartesian planning</b>         | <b>Descartes</b>             | <b>EndEffectorTrajectory</b>                    | Different strengths |
| <b>Constraint planning</b>        | TrajOpt/planner constraints  | <b>HPP first-class architecture</b>             | LAAS                |
| <b>Trajectory optimization</b>    | <b>TrajOpt</b>               | HPP PathOptimizer                               | IRT                 |

|                                    |                                       |                                     |                       |
|------------------------------------|---------------------------------------|-------------------------------------|-----------------------|
| <b>100+ defect sequencing</b>      | Not current capability                | <b>TSP/GTSP</b>                     | <b>LAAS</b>           |
| <b>Local defect scanning</b>       | Not current capability                | <b>Supported</b>                    | <b>LAAS</b>           |
| <b>Path-memory optimization</b>    | Not demonstrated                      | <b>New LAAS/HPP capability</b>      | <b>LAAS</b>           |
| <b>Task-level control</b>          | Robot/ROS controller-centric          | <b>Stack of Tasks / TSID</b>        | <b>LAAS</b>           |
| <b>Execution</b>                   | ROS 2 + joint trajectory currently    | <b>ROS2/RTDE/socket/custom</b>      | LAAS flexibility      |
| <b>Cartesian execution</b>         | Not current IRT implementation        | <b>Supported in your LAAS stack</b> | LAAS                  |
| <b>Execution sample rate</b>       | Not currently configurable            | <b>Configurable</b>                 | LAAS                  |
| <b>Orchestration</b>               | Need to clarify IRT supervisory layer | <b>AGIMUS / Behavior Tree</b>       | <b>LAAS currently</b> |
| <b>Failure recovery/replanning</b> | Need clarification                    | <b>Supervisor/BT can manage it</b>  | LAAS                  |

|                               |                       |                         |             |
|-------------------------------|-----------------------|-------------------------|-------------|
| <b>ROS2 standardization</b>   | <b>Very strong</b>    | Integration dependent   | <b>IRT</b>  |
| <b>Industrial packaging</b>   | <b>Major strength</b> | More integration effort | <b>IRT</b>  |
| <b>Research/customization</b> | Good                  | <b>Very high</b>        | <b>LAAS</b> |

---

## 20. The architectural conclusion is becoming quite strong

I would characterize **IRT** as:

Integrated industrial robot programming pipeline

with emphasis on:

3D/CAD→Automatic toolpath→Motion planning→Trajectory optimization→ROS2 execution

That aligns well with IRT's public objective of transferring robotics research into robust, quickly deployable industrial demonstrators.

And I would characterize **REACT / LAAS** as:

Modular autonomous robotics architecture

with:

Perception↔Planning↔Control↔Execution

under:

Supervisor / Behavior Tree

and with much deeper customization at each level.

That also fits LAAS's broader research philosophy: their robotics department explicitly spans perception, decision, movement generation and communication, while their decisional-architecture research covers functional components through acting-level skills.

So I wouldn't say **IRT has replaced what REACT was doing**. They overlap strongly in the middle—particularly motion planning—but the objectives appear different:

**IRT is pushing toward a standardized, industrially deployable Scan/Plan/Execute pipeline, whereas the LAAS/REACT architecture provides a broader perception–planning–control–supervision framework with more customization, online behavior and autonomy.**

The **Path Memory** concept strengthens the LAAS side further for repetitive industrial cells because it adds an experience/reuse layer that neither classical HPP planning nor a conventional Tesseract planning request provides by itself. I would make “**Planning Experience / Path Memory**” a separate row in your final architecture comparison, rather than hiding it under trajectory optimization.

Yes. Based on everything we have discussed, I would make the **final comparison at the stack/architecture level**, rather than calling it simply “HPP vs Tesseract.”

The two sides are:

- **IRT JV:** Open3D + Open CASCADE + MoveIt2/OMPL + Tesseract/Descartes/TrajOpt + ROS2 execution.
- **LAAS / REACT:** ViSP + customized GMM/local swathe generation + HPP + TSP/GTSP + Stack of Tasks/TSID + AGIMUS/Behavior Tree + flexible robot execution.

Also, wherever something is based specifically on what **IRT or LAAS told you**, I mark it as **current implementation** rather than incorrectly claiming it is an inherent limitation/capability of Tesseract or HPP.

## Final IRT JV vs LAAS/REACT comparison

| #                              | Factor / Feature   | IRT JV approach                                           | LAAS / REACT approach                                                  | Assessment          |
|--------------------------------|--------------------|-----------------------------------------------------------|------------------------------------------------------------------------|---------------------|
| <b>A. Overall architecture</b> |                    |                                                           |                                                                        |                     |
| 1                              | Overall philosophy | Integrated industrial Scan/Plan/Optimize/Execute pipeline | Modular perception–planning–control–execution–supervision architecture | Different strengths |
| 2                              | Primary focus      | Industrialization, standardization, process planning      | Flexibility, autonomy, constraint-rich/custom robotics                 | Different           |

|                      |                                |                                                                      |                                                                              |                                        |
|----------------------|--------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------|----------------------------------------|
| 3                    | Middleware                     | <b>ROS2-centric</b>                                                  | ROS/ROS2 possible but components can remain middleware/interface independent | IRT standardization / LAAS flexibility |
| 4                    | Component coupling             | More integrated pipeline                                             | Highly modular                                                               | <b>LAAS flexibility</b>                |
| 5                    | Industrial packaging           | <b>Strong</b>                                                        | Requires more application integration                                        | <b>IRT</b>                             |
| 6                    | Research customization         | Good                                                                 | <b>Very high</b>                                                             | <b>LAAS</b>                            |
| 7                    | Robot/application independence | Good through ROS ecosystem                                           | <b>High through modular adapters/components</b>                              | LAAS slight                            |
| <b>B. Perception</b> |                                |                                                                      |                                                                              |                                        |
| 8                    | Perception scope               | Currently mainly part/geometric identification in evaluated workflow | <b>Broader perception stack</b>                                              | <b>LAAS</b>                            |

|    |                        |                                         |                                                 |             |
|----|------------------------|-----------------------------------------|-------------------------------------------------|-------------|
| 9  | Main libraries         | Open3D + other IRT components           | <b>ViSP + application perception components</b> | Different   |
| 10 | Point-cloud processing | <b>Open3D strong</b>                    | Can be integrated                               | <b>IRT</b>  |
| 11 | Part identification    | Supported                               | Supported                                       | ≈           |
| 12 | Localization           | Partial/application-specific currently  | <b>ViSP-based localization</b>                  | <b>LAAS</b> |
| 13 | Referencing            | Partial/application-specific            | <b>Global/local referencing supported</b>       | <b>LAAS</b> |
| 14 | Feature detection      | Not complete perception stack currently | <b>Supported through perception stack</b>       | <b>LAAS</b> |
| 15 | Feature tracking       | Not central currently                   | <b>ViSP strength</b>                            | <b>LAAS</b> |
| 16 | Pose estimation        | Available depending on implementation   | <b>Strong ViSP integration</b>                  | LAAS        |
| 17 | Visual servoing        | Not central current architecture        | <b>ViSP / control integration</b>               | <b>LAAS</b> |

|    |                               |                               |                                                      |             |
|----|-------------------------------|-------------------------------|------------------------------------------------------|-------------|
| 18 | Perception → control feedback | Limited in evaluated pipeline | <b>Architecture supports closed-loop integration</b> | <b>LAAS</b> |
|----|-------------------------------|-------------------------------|------------------------------------------------------|-------------|

### C. CAD / geometric processing

|    |                             |                                       |                                              |            |
|----|-----------------------------|---------------------------------------|----------------------------------------------|------------|
| 19 | CAD processing              | <b>Open CASCADE</b>                   | Custom/application-specific                  | <b>IRT</b> |
| 20 | B-Rep/surface processing    | <b>Strong Open CASCADE capability</b> | Requires corresponding custom implementation | <b>IRT</b> |
| 21 | Mesh/point-cloud processing | <b>Open3D</b>                         | Custom/integrated library                    | <b>IRT</b> |
| 22 | Generic geometry processing | <b>Strong</b>                         | More use-case customized                     | <b>IRT</b> |

### D. Tool-path generation

|    |                    |                              |                                       |           |
|----|--------------------|------------------------------|---------------------------------------|-----------|
| 23 | Tool-path approach | <b>Open3D + Open CASCADE</b> | Customized GMM/local surface approach | Different |
|----|--------------------|------------------------------|---------------------------------------|-----------|



|    |                                    |                                                       |                  |             |
|----|------------------------------------|-------------------------------------------------------|------------------|-------------|
| 24 | Generic CAD-derived toolpath       | <b>Strong</b>                                         | More customized  | <b>IRT</b>  |
| 25 | Surface-based path generation      | Strong                                                | Strong           | ≈           |
| 26 | Swathe/raster generation           | Supported for predefined process geometry             | <b>Supported</b> | ≈           |
| 27 | Localized defect-centered swathe   | <b>Not currently supported in evaluated IRT stack</b> | <b>Supported</b> | <b>LAAS</b> |
| 28 | Dynamically selected ROI scanning  | Not current functionality                             | <b>Supported</b> | <b>LAAS</b> |
| 29 | Scan only selected defect area     | Not current implementation                            | <b>Yes</b>       | <b>LAAS</b> |
| 30 | Continuous full-surface processing | <b>Strong</b>                                         | Supported        | IRT slight  |

**E.  
Cartesian  
robot path  
planning**

|    |                           |                                                  |                                                            |                                   |
|----|---------------------------|--------------------------------------------------|------------------------------------------------------------|-----------------------------------|
| 31 | Cartesian planning        | <b>Descartes/process planning</b>                | <b>EndEffectorTrajectory</b>                               | Both strong                       |
| 32 | Cartesian representation  | Typically dense/sample process waypoints         | <b>Continuous EE trajectory can be represented</b>         | HPP different strength            |
| 33 | Linear TCP path           | Supported                                        | <b>Supported</b>                                           | ≈                                 |
| 34 | Curved TCP path           | Supported through sampled/generated toolpath     | <b>Supported by EE trajectory approach</b>                 | ≈ / HPP continuous representation |
| 35 | Spline TCP path           | Can be represented/discretized                   | <b>Supported in LAAS EE trajectory implementation</b>      | LAAS                              |
| 36 | Polynomial TCP path       | Can be represented through trajectory processing | <b>Different polynomial path representations available</b> | LAAS                              |
| 37 | Continuous EE curve       | Typically discretized into process poses         | <b>Strong EndEffectorTrajectory concept</b>                | <b>LAAS</b>                       |
| 38 | Dense Cartesian waypoints | <b>Descartes strength</b>                        | Supported                                                  | <b>IRT</b>                        |

## F. IK and redundancy

|    |                                   |                                        |                                                         |             |
|----|-----------------------------------|----------------------------------------|---------------------------------------------------------|-------------|
| 39 | IK support                        | Strong                                 | Strong                                                  | ≈           |
| 40 | Multiple IK solutions             | <b>Supported</b>                       | <b>Supported</b>                                        | ≈           |
| 41 | Redundant robot handling          | Strong                                 | <b>Very strong with constraints/steering</b>            | LAAS slight |
| 42 | IK sequence across Cartesian path | <b>Descartes layered graph</b>         | EE trajectory + constraint solver                       | Different   |
| 43 | Global IK candidate sequence      | <b>Descartes strength</b>              | Different constraint-driven approach                    | <b>IRT</b>  |
| 44 | Free redundant DOF                | Process tolerances/candidate solutions | <b>Masks/partial constraints naturally exploit it</b>   | <b>LAAS</b> |
| 45 | Posture customization             | Optimization terms                     | <b>Constraint/task architecture highly customizable</b> | LAAS        |

## G. Planner availability

|    |                   |                                                            |                                          |                     |
|----|-------------------|------------------------------------------------------------|------------------------------------------|---------------------|
| 46 | RRT               | ✓ OMPL                                                     | ✓                                        | Tie                 |
| 47 | RRT*              | ✓ OMPL                                                     | ✓                                        | Tie                 |
| 48 | Bidirectional RRT | ✓                                                          | ✓                                        | Tie                 |
| 49 | Bi-RRT*           | Depends on selected OMPL implementation                    | <b>Available in described LAAS stack</b> | LAAS implementation |
| 50 | PRM               | ✓                                                          | ✓                                        | Tie                 |
| 51 | PRM variants      | <b>Large OMPL ecosystem</b>                                | Depends on HPP/LAAS implementation       | IRT ecosystem       |
| 52 | A*                | Graph contexts, not primary Tesseract free-space interface | <b>Available in LAAS stack</b>           | LAAS                |
| 53 | Dijkstra          | Graph contexts                                             | <b>Available</b>                         | LAAS                |
| 54 | Potential Field   | Not central Tesseract planner                              | <b>Available</b>                         | LAAS                |

|    |                         |                  |                                     |                                   |
|----|-------------------------|------------------|-------------------------------------|-----------------------------------|
| 55 | Cartesian graph planner | <b>Descartes</b> | Different EE trajectory formulation | <b>IRT distinctive capability</b> |
|----|-------------------------|------------------|-------------------------------------|-----------------------------------|

|    |                      |                                |                          |            |
|----|----------------------|--------------------------------|--------------------------|------------|
| 56 | Trajectory optimizer | <b>TrajOpt / TrajOpt-IFOPT</b> | HPP PathOptimizer/custom | <b>IRT</b> |
|----|----------------------|--------------------------------|--------------------------|------------|

**H. Steering methods**

|    |                                |                          |                                       |            |
|----|--------------------------------|--------------------------|---------------------------------------|------------|
| 57 | Planner vs steering separation | Less central abstraction | <b>Explicit architectural concept</b> | <b>HPP</b> |
|----|--------------------------------|--------------------------|---------------------------------------|------------|

|    |                       |                                          |                  |           |
|----|-----------------------|------------------------------------------|------------------|-----------|
| 58 | Interpolated steering | Available through planners/interpolation | <b>Supported</b> | $\approx$ |
|----|-----------------------|------------------------------------------|------------------|-----------|

|    |                  |                                   |                  |            |
|----|------------------|-----------------------------------|------------------|------------|
| 59 | Hermite steering | Not central Tesseract abstraction | <b>Supported</b> | <b>HPP</b> |
|----|------------------|-----------------------------------|------------------|------------|

|    |                      |                            |                                      |            |
|----|----------------------|----------------------------|--------------------------------------|------------|
| 60 | Kinodynamic steering | Depends on backend/planner | <b>Supported in HPP architecture</b> | <b>HPP</b> |
|----|----------------------|----------------------------|--------------------------------------|------------|

|    |                            |                         |                              |           |
|----|----------------------------|-------------------------|------------------------------|-----------|
| 61 | Cartesian EE steering/path | Descartes sampled graph | <b>EndEffectorTrajectory</b> | Different |
|----|----------------------------|-------------------------|------------------------------|-----------|

|    |                        |                              |                                 |            |
|----|------------------------|------------------------------|---------------------------------|------------|
| 62 | Custom steering method | Possible through development | <b>Core extension mechanism</b> | <b>HPP</b> |
|----|------------------------|------------------------------|---------------------------------|------------|

|    |                           |                                |                           |            |
|----|---------------------------|--------------------------------|---------------------------|------------|
| 63 | Constraint-aware steering | Optimization/planner dependent | <b>Strong integration</b> | <b>HPP</b> |
|----|---------------------------|--------------------------------|---------------------------|------------|

**I.  
Constraint  
handling**

|    |                     |            |            |     |
|----|---------------------|------------|------------|-----|
| 64 | Equality constraint | <b>Yes</b> | <b>Yes</b> | Tie |
|----|---------------------|------------|------------|-----|

|    |                       |            |            |     |
|----|-----------------------|------------|------------|-----|
| 65 | Inequality constraint | <b>Yes</b> | <b>Yes</b> | Tie |
|----|-----------------------|------------|------------|-----|

|    |                               |          |                                |            |
|----|-------------------------------|----------|--------------------------------|------------|
| 66 | Implicit nonlinear constraint | Possible | <b>First-class HPP concept</b> | <b>HPP</b> |
|----|-------------------------------|----------|--------------------------------|------------|

|    |                     |                                    |                            |            |
|----|---------------------|------------------------------------|----------------------------|------------|
| 67 | Explicit constraint | Not a central planning abstraction | <b>First-class concept</b> | <b>HPP</b> |
|----|---------------------|------------------------------------|----------------------------|------------|

|    |                                |             |                                                                |            |
|----|--------------------------------|-------------|----------------------------------------------------------------|------------|
| 68 | Explicit variable substitution | Not central | <b>Supported conceptually through HPP constraint framework</b> | <b>HPP</b> |
|----|--------------------------------|-------------|----------------------------------------------------------------|------------|

|    |                       |                   |                          |            |
|----|-----------------------|-------------------|--------------------------|------------|
| 69 | Constraint projection | Planner dependent | <b>Core HPP strength</b> | <b>HPP</b> |
|----|-----------------------|-------------------|--------------------------|------------|

|    |                              |                                      |                        |            |
|----|------------------------------|--------------------------------------|------------------------|------------|
| 70 | Constraint manifold planning | Possible through specialized methods | <b>Core philosophy</b> | <b>HPP</b> |
|----|------------------------------|--------------------------------------|------------------------|------------|

|                                       |                                      |                                            |                                         |            |
|---------------------------------------|--------------------------------------|--------------------------------------------|-----------------------------------------|------------|
| 71                                    | Partial Cartesian constraint         | <b>Supported</b>                           | <b>Supported via masks/components</b>   | Tie        |
| 72                                    | Free Cartesian DOF                   | Supported through tolerances/IK freedom    | <b>Natural with masking</b>             | HPP slight |
| 73                                    | Hierarchical constraints             | Can be modeled through optimization/stages | <b>Hierarchical solver architecture</b> | <b>HPP</b> |
| 74                                    | Hard constraint                      | <b>Strong</b>                              | <b>Strong</b>                           | Tie        |
| 75                                    | Soft constraint/cost                 | <b>TrajOpt major strength</b>              | Possible but less central               | <b>IRT</b> |
| 76                                    | Mixed cost + constraint optimization | <b>Excellent</b>                           | More constraint-centric                 | <b>IRT</b> |
| <b>J. Example process constraints</b> |                                      |                                            |                                         |            |
| 77                                    | TCP follows exact swathe             | Cartesian waypoint constraint              | <b>EE trajectory equality</b>           | Both       |
| 78                                    | Standoff = 100 mm                    | Equality                                   | Equality                                | Tie        |

|    |                                |                                   |                                            |                                   |
|----|--------------------------------|-----------------------------------|--------------------------------------------|-----------------------------------|
| 79 | Standoff 98–102 mm             | Inequality/tolerance              | <b>Superior + Inferior type constraint</b> | Tie                               |
| 80 | Tool normal $\pm 5^\circ$      | Tolerance/optimization constraint | Inequality/partial orientation constraint  | Tie                               |
| 81 | Fixture clearance $\geq 50$ mm | <b>Collision constraint/cost</b>  | Collision/path constraint                  | IRT slight optimization advantage |
| 82 | Free rotation about tool Z     | Cartesian tolerance               | <b>Mask/unconstrained component</b>        | HPP elegant                       |
| 83 | Coupled mechanisms             | Custom constraints                | <b>Explicit/implicit constraints</b>       | <b>HPP</b>                        |

#### K. Collision handling

|    |                                   |                                    |                           |            |
|----|-----------------------------------|------------------------------------|---------------------------|------------|
| 84 | Collision checking                | <b>Strong</b>                      | <b>Strong</b>             | Tie        |
| 85 | Discrete collision checking       | Yes                                | Yes                       | Tie        |
| 86 | Continuous/contact-aware checking | <b>Strong Tesseract capability</b> | Path validation available | <b>IRT</b> |



|    |                             |                         |              |            |
|----|-----------------------------|-------------------------|--------------|------------|
| 87 | Collision as optimizer term | <b>TrajOpt strength</b> | Less central | <b>IRT</b> |
|----|-----------------------------|-------------------------|--------------|------------|

|    |                                |                                            |                    |            |
|----|--------------------------------|--------------------------------------------|--------------------|------------|
| 88 | Minimum clearance optimization | <b>Natural cost/constraint formulation</b> | Can be implemented | <b>IRT</b> |
|----|--------------------------------|--------------------------------------------|--------------------|------------|

|    |                                       |           |                               |           |
|----|---------------------------------------|-----------|-------------------------------|-----------|
| 89 | Collision during constrained planning | Supported | <b>Strong HPP integration</b> | $\approx$ |
|----|---------------------------------------|-----------|-------------------------------|-----------|

#### L. Singularity handling

|    |                       |                                                     |                        |           |
|----|-----------------------|-----------------------------------------------------|------------------------|-----------|
| 90 | Singularity detection | Available through kinematic/manipulability analysis | Available/customizable | $\approx$ |
|----|-----------------------|-----------------------------------------------------|------------------------|-----------|

|    |                                 |                                                    |                                          |                                    |
|----|---------------------------------|----------------------------------------------------|------------------------------------------|------------------------------------|
| 91 | Singularity avoidance objective | <b>TrajOpt exposes singularity-avoidance terms</b> | Can use constraints/cost/custom planning | <b>IRT out-of-box optimization</b> |
|----|---------------------------------|----------------------------------------------------|------------------------------------------|------------------------------------|

|    |                                       |     |                                                  |            |
|----|---------------------------------------|-----|--------------------------------------------------|------------|
| 92 | Exploit redundancy around singularity | Yes | <b>Strong via unconstrained DOFs/constraints</b> | HPP slight |
|----|---------------------------------------|-----|--------------------------------------------------|------------|

#### M. Path validation

|                                   |                               |                               |                                |            |
|-----------------------------------|-------------------------------|-------------------------------|--------------------------------|------------|
| 93                                | Joint-limit validation        | Yes                           | Yes                            | Tie        |
| 94                                | Collision validation          | Yes                           | Yes                            | Tie        |
| 95                                | Constraint validation         | Yes                           | <b>Deeply integrated</b>       | HPP        |
| 96                                | Cartesian path validation     | Yes                           | Yes                            | Tie        |
| 97                                | Custom validator              | Extensible                    | <b>Highly customizable</b>     | HPP slight |
| <b>N. Trajectory optimization</b> |                               |                               |                                |            |
| 98                                | Whole-trajectory optimization | <b>TrajOpt major strength</b> | Available but less central     | <b>IRT</b> |
| 99                                | Smoothness optimization       | <b>Strong</b>                 | Path optimization available    | <b>IRT</b> |
| 100                               | Velocity cost                 | <b>Supported</b>              | Can optimize/time parameterize | IRT        |

|                               |                              |                                 |                                         |                       |
|-------------------------------|------------------------------|---------------------------------|-----------------------------------------|-----------------------|
| 101                           | Acceleration cost            | <b>Supported</b>                | Available through processing            | IRT                   |
| 102                           | Jerk cost                    | <b>Supported</b>                | Application dependent                   | <b>IRT</b>            |
| 103                           | Collision optimization       | <b>Strong</b>                   | Less optimization-centric               | <b>IRT</b>            |
| 104                           | Cartesian error optimization | <b>Strong</b>                   | Primarily constraint satisfaction       | <b>IRT</b>            |
| 105                           | Multi-objective optimization | <b>Strong</b>                   | Possible/custom                         | <b>IRT</b>            |
| 106                           | Initial path refinement      | <b>OMPL/Descartes → TrajOpt</b> | Planner → PathOptimizer                 | IRT packaged pipeline |
| <b>O. Planner composition</b> |                              |                                 |                                         |                       |
| 107                           | Planner chaining             | <b>Very strong</b>              | Possible                                | <b>IRT</b>            |
| 108                           | OMPL → optimizer             | <b>OMPL → TrajOpt</b>           | Planner → optimizer equivalent possible | IRT packaged          |

|     |                                |                                   |                           |              |
|-----|--------------------------------|-----------------------------------|---------------------------|--------------|
| 109 | Cartesian → optimizer          | <b>Descartes</b> → <b>TrajOpt</b> | EE trajectory → optimizer | IRT packaged |
| 110 | Planner + steering composition | Less central                      | <b>Major HPP strength</b> | <b>HPP</b>   |
| 111 | Planner + constraint solver    | Supported but planner dependent   | <b>Core architecture</b>  | <b>HPP</b>   |

**P.  
Multi-defect  
sequencing /  
mission  
planning**

|     |                                |                                       |                                             |             |
|-----|--------------------------------|---------------------------------------|---------------------------------------------|-------------|
| 112 | Multiple defect visits         | Sequence can be supplied              | <b>Supported workflow</b>                   | LAAS        |
| 113 | Automatic optimal defect order | Not current IRT capability            | <b>TSP</b>                                  | <b>LAAS</b> |
| 114 | 100+ defects                   | Requires additional sequencing module | <b>Supported by sequencing architecture</b> | <b>LAAS</b> |
| 115 | Multiple candidates per defect | Requires custom layer                 | <b>GTSP</b>                                 | <b>LAAS</b> |

|     |                                       |                                 |                                                |              |
|-----|---------------------------------------|---------------------------------|------------------------------------------------|--------------|
| 116 | Optimize defect + robot configuration | Not current functionality       | <b>GTSP can jointly select candidate state</b> | <b>LAAS</b>  |
| 117 | Euclidean routing                     | Can be added externally         | Supported                                      | LAAS current |
| 118 | Robot-aware route cost                | Requires additional integration | <b>Can use HPP transition costs</b>            | <b>LAAS</b>  |
| 119 | Collision-aware transition            | Tesseract/MoveIt                | HPP                                            | Tie          |
| 120 | Mission-level optimization            | Missing currently               | <b>TSP/GTSP</b>                                | <b>LAAS</b>  |
| 121 | Local scan after target selection     | Not current capability          | <b>Integrated</b>                              | <b>LAAS</b>  |

**Q. Task composition / orchestration**

|     |                  |                                                          |                   |                   |
|-----|------------------|----------------------------------------------------------|-------------------|-------------------|
| 122 | Task composition | <b>Task Composer / MoveIt Task Constructor ecosystem</b> | Custom/supervisor | <b>IRT strong</b> |
|-----|------------------|----------------------------------------------------------|-------------------|-------------------|

|     |                               |                                             |                                   |              |
|-----|-------------------------------|---------------------------------------------|-----------------------------------|--------------|
| 123 | Task-sequence optimization    | Not equivalent to Task Composer             | <b>TSP/GTSP</b>                   | <b>LAAS</b>  |
| 124 | Overall supervisor            | Need confirmation of IRT implementation     | <b>AGIMUS / Behavior Tree</b>     | LAAS current |
| 125 | Conditional branching         | Depends on IRT orchestration implementation | <b>Behavior Tree natural fit</b>  | LAAS         |
| 126 | Failure handling              | Needs implementation confirmation           | <b>BT/supervisor can manage</b>   | LAAS         |
| 127 | Retry/replan                  | Needs implementation confirmation           | <b>Supervisor can orchestrate</b> | LAAS         |
| 128 | Dynamic next-target selection | Requires custom layer                       | <b>TSP/GTSP + supervisor</b>      | LAAS         |
| 129 | Re-localization workflow      | Needs perception/orchestrator integration   | <b>ViSP + supervisor</b>          | LAAS         |

## R. Control

|     |                                   |                                                       |                              |             |
|-----|-----------------------------------|-------------------------------------------------------|------------------------------|-------------|
| 130 | Explicit task-level control layer | Current pipeline appears execution/controller-centric | <b>Stack of Tasks / TSID</b> | <b>LAAS</b> |
|-----|-----------------------------------|-------------------------------------------------------|------------------------------|-------------|

|     |                             |                                      |                                           |             |
|-----|-----------------------------|--------------------------------------|-------------------------------------------|-------------|
| 131 | Task-priority control       | Not central current IRT architecture | <b>Stack of Tasks strength</b>            | <b>LAAS</b> |
| 132 | TCP control objective       | Robot controller/ROS interface       | <b>Can be explicit task</b>               | LAAS        |
| 133 | Secondary posture objective | Primarily planner/optimizer          | <b>Task hierarchy/null-space control</b>  | LAAS        |
| 134 | Sensor-feedback control     | Requires dedicated controller        | <b>Can integrate with task controller</b> | LAAS        |
| 135 | Online task optimization    | Not central current workflow         | <b>SoT/TSID architecture</b>              | LAAS        |

#### **S. Trajectory execution**

|     |                              |                             |                            |            |
|-----|------------------------------|-----------------------------|----------------------------|------------|
| 136 | Current execution middleware | <b>ROS2</b>                 | ROS2 / robot-native/custom | Different  |
| 137 | ROS2 support                 | <b>Strong</b>               | Supported/integratable     | <b>IRT</b> |
| 138 | UR RTDE                      | Not current execution route | <b>Can use adapter</b>     | LAAS       |

|     |                                   |                                           |                                                    |                  |
|-----|-----------------------------------|-------------------------------------------|----------------------------------------------------|------------------|
| 139 | FANUC socket                      | Not current execution route               | <b>Can use adapter</b>                             | LAAS             |
| 140 | Custom robot protocol             | Less central                              | <b>Supported architecture</b>                      | LAAS             |
| 141 | Joint trajectory execution        | <b>Yes</b>                                | <b>Yes</b>                                         | Tie              |
| 142 | TCP trajectory execution          | Not current IRT implementation            | <b>Supported in described LAAS execution layer</b> | <b>LAAS</b>      |
| 143 | Output sample rate                | Not currently configurable/exposed        | <b>Configurable in execution layer</b>             | <b>LAAS</b>      |
| 144 | Robot-controller independence     | ROS-driver dependent                      | <b>Adapter based</b>                               | LAAS flexibility |
| 145 | Standardized interface            | <b>Excellent</b>                          | Depends on adapter                                 | <b>IRT</b>       |
| 146 | Deterministic joint configuration | <b>Strong joint-trajectory approach</b>   | Supported                                          | Tie              |
| 147 | Online TCP correction potential   | Requires appropriate controller/interface | <b>Higher with Cartesian/custom interfaces</b>     | LAAS             |



**T. Path  
Memory /  
experience reuse**

|     |                                |                                                |                                         |               |
|-----|--------------------------------|------------------------------------------------|-----------------------------------------|---------------|
| 148 | Stochastic path variation      | Sampling planners can vary                     | Sampling planners can vary              | Tie           |
| 149 | Remember previous paths        | Not demonstrated in current IRT implementation | <b>New LAAS/HPP Path Memory concept</b> | <b>LAAS</b>   |
| 150 | Compare historical paths       | Not demonstrated                               | <b>Yes, as described by LAAS</b>        | <b>LAAS</b>   |
| 151 | Select best historical path    | Not demonstrated                               | <b>Yes</b>                              | <b>LAAS</b>   |
| 152 | Multi-metric path analysis     | TrajOpt current-run objective                  | <b>Can analyze stored path cost</b>     | Different     |
| 153 | Environment-aware reuse        | Would require caching logic                    | <b>Planned Path Memory behavior</b>     | LAAS          |
| 154 | Reuse in unchanged environment | Not demonstrated                               | <b>Key intended use</b>                 | LAAS          |
| 155 | Path revalidation before reuse | Can be implemented                             | <b>Should always be performed</b>       | Both possible |

|                                      |                                             |                                  |                              |             |
|--------------------------------------|---------------------------------------------|----------------------------------|------------------------------|-------------|
| 156                                  | Cross-run optimization                      | Not current feature demonstrated | <b>Path Memory</b>           | <b>LAAS</b> |
| 157                                  | Intra-run optimization                      | <b>TrajOpt excellent</b>         | Path optimizer               | <b>IRT</b>  |
| <b>U.<br/>Industrial suitability</b> |                                             |                                  |                              |             |
| 158                                  | Standard ROS ecosystem                      | <b>Excellent</b>                 | Good                         | <b>IRT</b>  |
| 159                                  | Standard deployment pipeline                | <b>Strong</b>                    | More customized              | <b>IRT</b>  |
| 160                                  | Maintainability through standard interfaces | <b>Strong</b>                    | More custom code to maintain | <b>IRT</b>  |
| 161                                  | Highly specialized industrial cell          | Strong                           | <b>Very strong</b>           | LAAS        |
| 162                                  | Research experimentation                    | Good                             | <b>Excellent</b>             | LAAS        |
| 163                                  | Complex constrained robot                   | Strong                           | <b>Excellent</b>             | LAAS        |

|     |                                     |                                |                                   |             |
|-----|-------------------------------------|--------------------------------|-----------------------------------|-------------|
| 164 | Repetitive CAD-defined process      | <b>Excellent</b>               | Strong                            | <b>IRT</b>  |
| 165 | Sparse defect-driven inspection     | Current gaps                   | <b>Excellent fit</b>              | <b>LAAS</b> |
| 166 | Adaptive/autonomous inspection      | Requires additional components | <b>Architecture better suited</b> | <b>LAAS</b> |
| 167 | Turnkey industrialization direction | <b>Stronger</b>                | Requires integration/engineering  | <b>IRT</b>  |

## The biggest differentiators

Rather than presenting 167 rows to management, I would reduce the final decision to about **10 architectural differentiators**:

| Area                    | IRT JV                       | LAAS / REACT                                          | Winner for your inspection use case   |
|-------------------------|------------------------------|-------------------------------------------------------|---------------------------------------|
| Complete perception     | Partial current scope        | <b>ViSP-based perception/localization/referencing</b> | <b>LAAS</b>                           |
| CAD/geometry processing | <b>Open3D + Open CASCADE</b> | Customized GMM                                        | <b>IRT for generic CAD processing</b> |

|                           |                                 |                                                |                       |
|---------------------------|---------------------------------|------------------------------------------------|-----------------------|
| Local defect swathe       | Not current capability          | <b>Dynamic localized swathe</b>                | <b>LAAS</b>           |
| Free-space planning       | <b>OMPL/Tesseract</b>           | <b>HPP multiple planners</b>                   | ≈                     |
| Cartesian planning        | <b>Descartes</b>                | <b>EndEffectorTrajectory</b>                   | Different strengths   |
| Complex constraints       | TrajOpt/constraints             | <b>Implicit/explicit + projection/manifold</b> | <b>LAAS</b>           |
| Trajectory optimization   | <b>TrajOpt</b>                  | HPP optimizers                                 | <b>IRT</b>            |
| 100+ defect sequencing    | Missing currently               | <b>TSP/GTSP</b>                                | <b>LAAS</b>           |
| Control                   | ROS/controller-centric          | <b>Stack of Tasks / TSID</b>                   | <b>LAAS</b>           |
| Orchestration             | Needs clarification             | <b>AGIMUS / Behavior Tree</b>                  | <b>LAAS currently</b> |
| Execution standardization | <b>ROS2</b>                     | Multiple custom interfaces                     | <b>IRT</b>            |
| Execution flexibility     | ROS2 joint trajectory currently | <b>TCP/joint + ROS2/RTDE/socket</b>            | <b>LAAS</b>           |

|                       |                  |                                          |             |
|-----------------------|------------------|------------------------------------------|-------------|
| Experience/path reuse | Not demonstrated | <b>Path Memory</b>                       | <b>LAAS</b> |
| Industrial packaging  | <b>Strong</b>    | Highly customizable but more integration | <b>IRT</b>  |

## Overall interpretation

For a **standard industrial process where the CAD/process area is known beforehand**, the IRT architecture is extremely attractive:

CAD/3D→Toolpath→Descartes/OMPL→TrajOpt→ROS2→Robot

Its major strengths are **standardization, industrial packaging, geometry processing and trajectory optimization**.

For your **autonomous NDT inspection scenario with 100+ dynamically detected defects**, the LAAS architecture is currently more complete:

Perception→Localization/Referencing→Defect Detection→Local Swathe→TSP/GTSP→HPP→SoT/TSID→Execution

with:

AGIMUS/Behavior Tree

providing supervision and potentially:

Path Memory

providing planning-experience reuse.

## The most important conclusion

I would **not conclude that HPP is simply “better than Tesseract.”**

A more technically defensible conclusion is:

**IRT JV provides the stronger standardized industrial process-planning pipeline, particularly for CAD/geometry-driven tool-path generation, planner chaining and whole-trajectory optimization. LAAS/REACT provides the more complete and customizable autonomous robotics architecture, particularly**

**for perception-driven inspection, complex constrained planning, mission-level sequencing, task-level control, flexible execution and supervisory autonomy.**

And specifically for your **100+ defect NDT inspection scenario**, I would currently favor the **LAAS/REACT architecture**, because the requirement is not merely:

Generate a good trajectory

but:

Perceive→decide where to inspect→optimize inspection order→generate local scan→plan→control→execute→recover/replan

That end-to-end autonomy requirement is where the architectural difference becomes much more significant than comparing individual RRT or trajectory-optimization algorithms.